

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ  
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»  
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ  
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

## **ПОЯСНЮВАЛЬНА ЗАПИСКА**

до кваліфікаційної роботи освітнього ступеня «бакалавр»  
за спеціальністю 121 «Інженерія програмного забезпечення»  
(освітня програма «Інженерія програмного забезпечення»)

на тему:

**«Система для розвитку гармонічного слуху»**

Виконала студентка групи ІПЗк-23-1  
ШАНУРІНА Дар'я Сергіївна

Керівник роботи:  
САВІЦЬКИЙ Роман Святославович

Рецензент:  
ГОДЛЕВСЬКИЙ Юрій Олександрович

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ  
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»  
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ  
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

**«ЗАТВЕРДЖУЮ»**

Завідувач кафедри інженерії  
програмного забезпечення

Тетяна ВАКАЛЮК

«26» лютого 2025 р.

**ЗАВДАННЯ**  
**на кваліфікаційну роботу**

Здобувач вищої освіти: **ШАНУРІНА Дар'я Сергіївна**

Керівник роботи: **САВІЦЬКИЙ Роман Святославович**

Тема роботи: **«Система для розвитку гармонічного слуху»**,

затверджена Наказом закладу вищої освіти від **«14» лютого 2025 р., №61/с**

Вихідні дані для роботи: Об'єктом дослідження є процес створення системи для тренування гармонічного слуху користувачів, предметом дослідження є розробка системи для розвитку гармонічного слуху з використанням нейронної мережі для розпізнавання музичних конструкцій.

Консультанти з бакалаврської кваліфікаційної роботи із зазначенням розділів, що їх стосуються:

| Розділ | Консультант    | Завдання видав | Завдання прийняв |
|--------|----------------|----------------|------------------|
| 1      | Савіцький Р.С. | 27.02.2025р.   | 27.02.2025р.     |
| 2      | Савіцький Р.С. | 27.02.2025р.   | 27.02.2025р.     |
| 3      | Савіцький Р.С. | 27.02.2025р.   | 27.02.2025р.     |

## РЕФЕРАТ

Кваліфікаційна робота освітнього ступеня «бакалавр» складається з програмної системи для розвитку гармонічного слуху та пояснювальної записки. Пояснювальна записка до кваліфікаційної роботи містить 105 сторінок, 50 ілюстрацій та 14 таблиць.

Метою роботи є розробка вебзастосунку для тренування та тестування гармонічного слуху користувачів із використанням нейронних мереж. Реалізовано систему на основі клієнт-серверної архітектури із REST API. Серверна частина створена за допомогою Django з використанням бібліотек TensorFlow, Librosa та SoundFile, клієнтська – за допомогою React.js.

У межах роботи проведено аналіз аналогів, обґрунтовано вибір архітектури та технологічного стеку, розроблено модулі автентифікації, тренування, тестування та розпізнавання інтервалів, реалізовано віртуальне фортепіано та систему накопичення балів досвіду. Протестовано роботу основних модулів, проведено оцінку ефективності розпізнавання інтервалів.

**КЛЮЧОВІ СЛОВА:** ГАРМОНІЧНИЙ СЛУХ, ІНТЕРВАЛ, ВЕБЗАСТОСУНОК, NEURAL NETWORK, DJANGO, REACT, ML, SPA.

|           |      |                  |        |      |                                                                     |                                             |      |        |
|-----------|------|------------------|--------|------|---------------------------------------------------------------------|---------------------------------------------|------|--------|
|           |      |                  |        |      | ІПЗ.КР.Б – 121 – 25 – ПЗ                                            |                                             |      |        |
| Змн.      | Арк. | № докум.         | Підпис | Дата |                                                                     |                                             |      |        |
| Розроб.   |      | Шануріна Д.С.    |        |      | Система для розвитку гармонічного слуху<br><br>Пояснювальна записка | Літ.                                        | Арк. | Аркуші |
| Керівник  |      | Савіцький Р.С.   |        |      |                                                                     |                                             | 3    | 105    |
|           |      |                  |        |      |                                                                     | Житомирська політехніка,<br>група ІПЗк-23-1 |      |        |
| Рецензент |      | Годлевський Ю.О. |        |      |                                                                     |                                             |      |        |
| Зав. каф. |      | Вакалюк Т.А.     |        |      |                                                                     |                                             |      |        |

## ABSTRACT

The bachelor's qualification project consists of a software system for developing harmonic hearing and an explanatory paper. The explanatory paper includes 103 pages, 50 illustrations, and 14 tables.

The objective of the project is to develop a web application for training and testing users' harmonic hearing using neural networks. The system is implemented using a client-server architecture based on REST API. The backend is developed with Django using TensorFlow, Librosa, and SoundFile libraries; the frontend is built with React.js.

The work includes analysis of existing solutions, justification of architectural and technological choices, and the implementation of authentication, training, testing, and interval recognition modules. A virtual piano and an experience point system were developed. The performance of the main modules and the accuracy of interval recognition were tested.

KEYWORDS: HARMONIC HEARING, INTERVAL, WEB APPLICATION, NEURAL NETWORK, DJANGO, REACT, ML, SPA.

## ЗМІСТ

|                                                                                             |            |
|---------------------------------------------------------------------------------------------|------------|
| <b>РЕФЕРАТ .....</b>                                                                        | <b>3</b>   |
| <b>ABSTRACT .....</b>                                                                       | <b>4</b>   |
| <b>ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ .....</b>                                                      | <b>6</b>   |
| <b>ВСТУП.....</b>                                                                           | <b>8</b>   |
| <b>РОЗДІЛ 1. АНАЛІЗ НАПРЯМКІВ ВИКОРИСТАННЯ ІНФОРМАЦІЙНИХ<br/>ТЕХНОЛОГІЙ .....</b>           | <b>10</b>  |
| 1.1 Постановка задачі .....                                                                 | 10         |
| 1.2 Аналіз аналогів програмного продукту .....                                              | 12         |
| 1.3 Вибір архітектури моделі розпізнавання системи для розвитку<br>гармонічного слуху ..... | 19         |
| 1.4 Обґрунтування вибору інструментальних засобів .....                                     | 23         |
| Висновки до першого розділу.....                                                            | 27         |
| <b>РОЗДІЛ 2. ПРОЄКТУВАННЯ СИСТЕМИ ДЛЯ РОЗВИТКУ<br/>ГАРМОНІЧНОГО СЛУХУ .....</b>             | <b>29</b>  |
| 2.1 Визначення варіантів використання та структури системи .....                            | 29         |
| 2.2 Проєктування та реалізація алгоритмів роботи системи .....                              | 42         |
| 2.3 Реалізація створення датасету .....                                                     | 52         |
| 2.4 Реалізація створення моделі розпізнавання .....                                         | 67         |
| Висновки до другого розділу.....                                                            | 72         |
| <b>РОЗДІЛ 3. ІНТЕРФЕЙС ТА ПОРЯДОК РОБОТИ З СИСТЕМОЮ<br/>ДОДАТКУ .....</b>                   | <b>74</b>  |
| 3.1 Структура інтерфейсу та порядок взаємодії з системою.....                               | 74         |
| 3.2 Тестування датасету та моделі .....                                                     | 83         |
| 3.3 Тестування роботи програмного продукту .....                                            | 95         |
| Висновки до третього розділу .....                                                          | 98         |
| <b>ВИСНОВКИ .....</b>                                                                       | <b>100</b> |
| <b>СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ .....</b>                                                     | <b>103</b> |
| <b>ДОДАТКИ.....</b>                                                                         | <b>105</b> |

## ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

ADSR — Attack, Decay, Sustain, Release — фази огинаючої звукового сигналу.

API — Application Programming Interface — інтерфейс прикладного програмування.

CORS — Cross-Origin Resource Sharing — механізм обміну ресурсами між різними джерелами.

CSS — Cascading Style Sheets — каскадні таблиці стилів.

DC — Direct Current — постійний струм.

DOM — Document Object Model — об'єктна модель документа.

ES-модулі — ECMAScript Modules — модулі в JavaScript за стандартом ECMAScript.

FFT — Fast Fourier Transform — швидке перетворення Фур'є.

HTML — HyperText Markup Language — мова гіпертекстової розмітки.

HTTP — HyperText Transfer Protocol — протокол передавання гіпертексту.

JWT — JSON Web Token — токен для передачі захищеної інформації.

ORM — Object-Relational Mapping — об'єктно-реляційне відображення.

POST — метод POST — метод HTTP-запиту для надсилання даних.

REST — Representational State Transfer — архітектурний стиль вебсервісів.

RMS — Root Mean Square — середньоквадратична енергія сигналу.

SPA — Single Page Application — односторінковий вебзастосунок.

STFT — Short-Time Fourier Transform — короткочасне перетворення Фур'є.

URL — Uniform Resource Locator — уніфікований локатор ресурсу.

UUID — Universally Unique Identifier — універсальний унікальний ідентифікатор.

WAV — Waveform Audio File Format — формат звукового файлу для зберігання звукових даних у нестиснутому вигляді.

JWT — JSON Web Token — формат для обміну захищеною інформацією у вебi.

ПК — персональний комп'ютер.

|  |  |  |  |  |                          |   |
|--|--|--|--|--|--------------------------|---|
|  |  |  |  |  | ІПЗ.КР.Б – 121 – 25 – ІЗ |   |
|  |  |  |  |  |                          |   |
|  |  |  |  |  |                          | 7 |

## ВСТУП

**Актуальність теми.** Звук — це більше, ніж просто механічна хвиля коливання. Наше сприйняття світу формується на основі постійного зіставлення мозком отриманих сигналів з очікуваними моделями попереднього досвіду, прогножуючи та роблячи певні висновки щодо джерела, характеристик та значення донесених у вигляді нервових імпульсів звукових хвиль. Чим більший набір із уявлень та шаблонів сприйняття свідомість має, тим точніше можна інтерпретувати та розпізнавати звукові сигнали.

Звуки, які органи слуху здатні сприймати, в основному поділяються на музичні (у яких чітко можна визначити висоту тону, яка прямо залежить від частоти коливань у звуковій хвилі) та шумові (з невпорядкованим частотним спектром). Якщо під час прослуховування музики зосередитися на взаємодії тонів між собою, то можна помітити, що вони складаються в різноманітні структури та композиції. Одні звуки мають вище звучання, інші — нижче; десь звучать дві, три чи більше ноти одночасно, а в інших місцях вони можуть утворювати послідовну мелодію. Після такого спостереження мимоволі виникає питання, чи спроможний мозок навчитися глибше аналізувати аудіосигнали та самотійно розділяти його на структури.

Насправді це є можливим завдяки гармонічному слуху. Гармонічним слухом називають здатність сприймати та розрізняти одночасне звучання двох і більше звуків, розпізнавати послідовності акордів і сприймати багатоголосну музику. Формується він на основі звуковисотного та ладового відчуття, дозволяючи не лише чути гармонійну структуру, а й осмислювати окремі ноти, з яких вона складається.

Кожна людина має гармонічний слух, проте рівень його розвитку залежить від індивідуальних особливостей та практикування. Наразі розвиток гармонічного слуху є можливим завдяки доступним методам і засобам навчання, таким як музичні школи, приватні заняття або самотійне навчання. Однак не всі мають змогу скористатися цими можливостями: музичні заклади часто не

приймають дорослих учнів, індивідуальні заняття потребують значних фінансових витрат, а самостійний пошук закінчується відсутністю якісних навчальних матеріалів. Крім того, класичні підходи до навчання нерідко не враховують індивідуальних особливостей учнів, їхнього темпу засвоєння матеріалу та специфічних потреб. У цьому контексті система для розвитку гармонічного слуху є ефективним рішенням, що поєднує доступність, гнучкість та адаптивність. Її можуть використовувати як початківці, так і досвідчені музиканти, які прагнуть удосконалити свої навички.

**Метою** є розробка системи для розвитку гармонічного слуху.

Для реалізації поставленої мети визначено наступні **завдання**:

1. Проведення аналізу аналогів з метою дослідження функціоналу та визначення переваг і недоліків.
2. Визначення архітектури системи.
3. Проектування та вибір технологій та інструментів реалізації.
4. Розробка алгоритмів системи.
5. Реалізація функціоналу системи.
6. Тестування та перевірка на відповідність вимогам.

**Об'єктом дослідження** є процес створення системи для тренування гармонічного слуху користувачів.

**Предметом дослідження** є розробка системи для розвитку гармонічного слуху з використанням нейронної мережі для розпізнавання музичних конструкцій.

# РОЗДІЛ 1. АНАЛІЗ НАПРЯМКІВ ВИКОРИСТАННЯ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

## 1.1 Постановка задачі

Розвиток гармонічного слуху є одним із найважливіших аспектів музичної освіти, що визначає здатність людини сприймати, аналізувати та відтворювати складні музичні структури. Гармонічний слух дозволяє розрізняти інтервали, акорди та їх послідовності, що є досить важливим для успішної музичної діяльності на будь-якому рівні професійності.

Традиційні методи розвитку гармонічного слуху характеризуються рядом суттєвих обмежень. По-перше, вони значною мірою залежать від постійної присутності кваліфікованого викладача, який здійснює контроль та оцінку результатів навчання. По-друге, студенти мають обмежені можливості для самостійної практики поза аудиторними заняттями, що суттєво уповільнює процес засвоєння матеріалу. По-третє, оцінювання прогресу часто носить суб'єктивний характер і не забезпечує точних кількісних показників розвитку навичок. Крім того, існуючі підходи не дозволяють адаптувати темп навчання до індивідуальних потреб студентів, що знижує ефективність освітнього процесу.

Метою даної роботи є розробка комплексної вебсистеми для розвитку гармонічного слуху, яка забезпечить ефективне самостійне навчання користувачів через поєднання практичних вправ та автоматизованого аналізу результатів. Система повинна надавати можливість користувачам розвивати гармонічний слух за допомогою різноманітних видів діяльності, що охоплюють як теоретичне засвоєння матеріалу, так і практичні вправи.

Для встановлення переліку функціоналу потрібно визначити основні впливові фактори, які сприятимуть навчанню. Найбільший вплив на розвиток гармонічного слуху мають такі види діяльності як прослуховування прикладів звучання інтервалів для формування асоціацій, проведення тестування на розпізнавання звучання музичних структур, а також самостійне відтворення або повторення голосом чи інструментом звукового прикладу музичної конструкції.

Беручи до уваги визначені чинники, система для розвитку гармонічного слуху повинна містити такі модулі, як модуль розпізнавання, модуль тренування та модуль тестування. Модуль розпізнавання передбачений для того, щоб користувач мав можливість за допомогою вебдодатка розпізнавати музичні конструкції, які звучать навколо нього. Цей модуль буде особливо корисним для тих, хто тільки починає розвивати гармонічний слух, оскільки самостійне осмислювання структури звуку є доволі складною задачею. Його можна використовувати як допоміжний засіб під час практикування слухової чутливості біля музичного інструмента або для полегшення процесу підбирання мелодії на слух.

Модуль тренування являтиме собою лабораторію генерування звучання музичних конструкцій, у якій користувач після налаштування критеріїв генерації матиме змогу відтворювати звучання інтервалів. Система має передбачати обирання категорії музичних структур та базової ноти, від якої будуватимуться аудіоприклади. Важливою вимогою є реалізація прикладів звучання у двох режимах з цілісним (одночасним) звучанням нот і з поступовим відтворенням нот для кращого розуміння структури і відстані між звуками.

Модуль тестування призначено для забезпечення комфортної перевірки рівня розвитку гармонічного слуху. Проведення даного виду діяльності реалізовано за допомогою проходження користувачем тесту із завданнями на обирання назви музичної конструкції, звучання якої буде надане у завданні. За кожну правильну відповідь користувач після проходження тесту отримуватиме бали досвіду, які своєю кількістю визначатимуть рівень розвинутості гармонічного слуху.

Система орієнтована на широке коло користувачів, включаючи студентів музичних навчальних закладів, музикантів-початківців та аматорів, викладачів музично-теоретичних дисциплін, а також осіб, що займаються самостійним вивченням музики. Окрім основних модулів, система повинна мати функціонал можливість перегляду набраних балів досвіду і рівня користувача.

Наукова новизна роботи полягає у комплексному підході до розвитку гармонічного слуху через інтеграцію машинного навчання та вебтехнологій, розробці моделі для розпізнавання музичних інтервалів на основі спектральних ознак і характеристик, а також створенні адаптивної системи навчання з персоналізованим підходом. Практична значущість визначається забезпеченням доступності якісної музичної освіти незалежно від географічного розташування, можливістю індивідуального темпу навчання та об'єктивного оцінювання прогресу.

Сформульовані вимоги сприятимуть забезпеченню ефективного функціонування системи та реалізації її ключових завдань.

## **1.2 Аналіз аналогів програмного продукту**

На даному етапі для кращого розуміння аспектів визначених функціональних вимог варто здійснити аналіз існуючих аналогів програмного продукту. Оцінювання сильних і слабких сторін наявних рішень дозволить врахувати їх у процесі розробки, а також остаточно визначити оптимальний функціонал на основі виявлених недоліків та потреб цільової аудиторії. До того ж це сприятиме створенню унікальних переваг, які відрізнятимуть розроблювану систему від аналогів.

Отже, проведемо виявлення доступних програмних продуктів, які мають частково схожий функціонал або вирішують подібні завдання.

Першим розглянемо мобільний додаток Simply Sing [1]. Програмний продукт призначено для вдосконалення чистоти співу. Після створення акаунту додаток пропонує визначити тип голосу шляхом виконання завдань на оцінку вокальних можливостей. Завдяки адаптації всіх пісень до індивідуального вокального діапазону користувач може комфортно виконувати їх, незалежно від тональності, у якій вони були записані оригінальним виконавцем.

На головній сторінці (рис. 1.1) надається доступ до особистих даних, щоденного навчального завдання з порадами для покращення контролю голосом та до пісень для виконання.

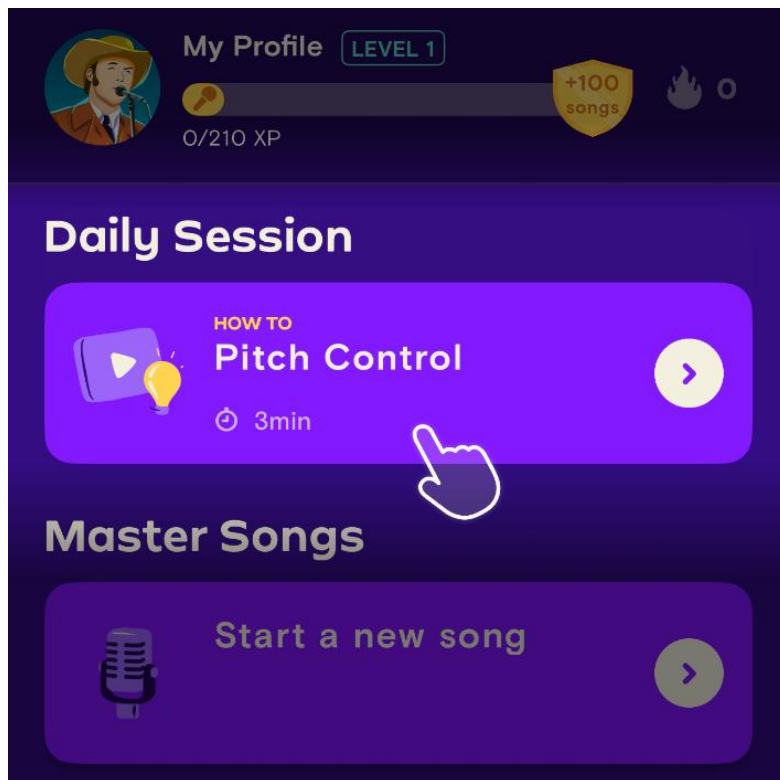


Рисунок 1.1 – Фрагмент головного екрана додатка Simply Sing

Під час відтворення користувачем вокальної партії додаток аналізує голос на чистоту співу та у режимі реального часу надає візуальну зворотну реакцію. Якщо виконання є правильним, то доріжка підсвічується іншим кольором, а вказівник залишатиметься в її межах (рис. 1.2).

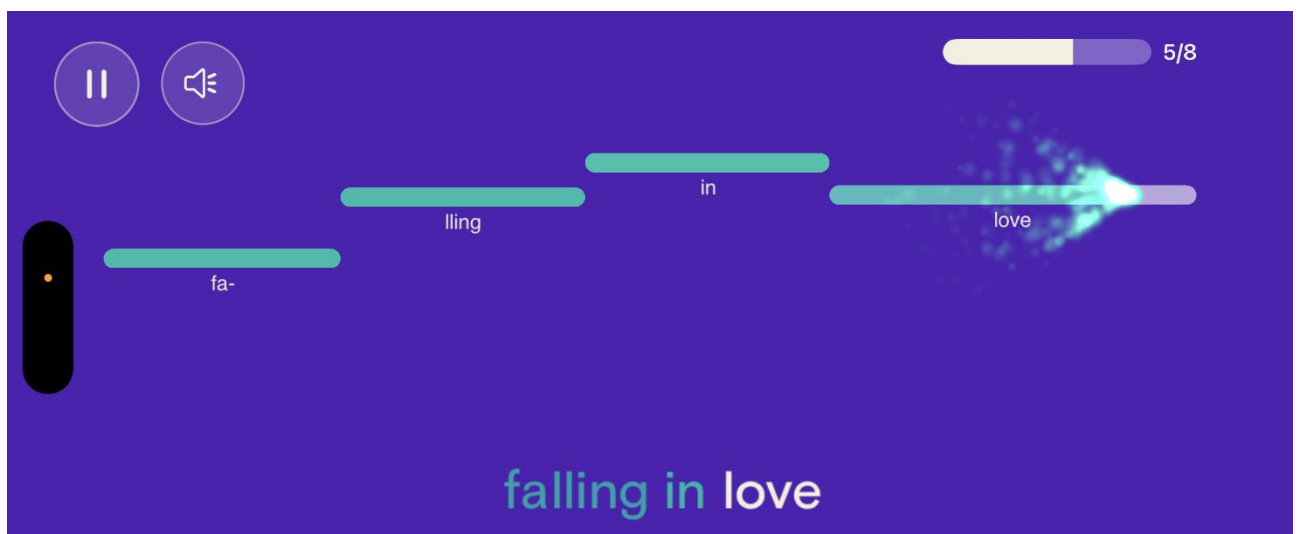


Рисунок 1.2 – Приклад потрапляння в ноти пісні в додатку  
Simply Sing

При неправильному відтворенні голосом партії програма реагує зміною положення вказівника, який не буде фіксуватися на потрібній висоті, що призводить до відсутності зафарбовування доріжки (рис. 1.3).



Рисунок 1.3 – Реакція додатку Simply Sing на підвищення голосу за межі норми

За результатами тестування функціональності додатку можна стверджувати, що ідентифікація звукових змін здійснюється на належному рівні: будь-які коливання висоти голосу миттєво фіксуються та відображаються, а точність розпізнавання нот є високою.

Однак, попри можливість розпізнавання голосу чи звучання музичного інструменту за допомогою запатентованого механізму ідентифікації акустичних нот, додаток не підтримує аналіз музичних конструкцій, які відтворюються з одночасним звучанням нот. Цей аспект заплановано до реалізації в наступних етапах розробки системи.

Наступним аналогом розглянемо сайт Musicca [2]. Даний сайт містить в собі різноманітні вправи для набуття музичних навичок та вдосконалення вмінь визначати, записувати та відтворювати ноти, ритми, інтервали, акорди тощо. У розділі «Інструменти» додатково надано можливість грати на віртуальних музичних інструментах. Для тренування слуху потрібно обрати категорію музичних конструкцій та вид тренування (рис. 1.4).

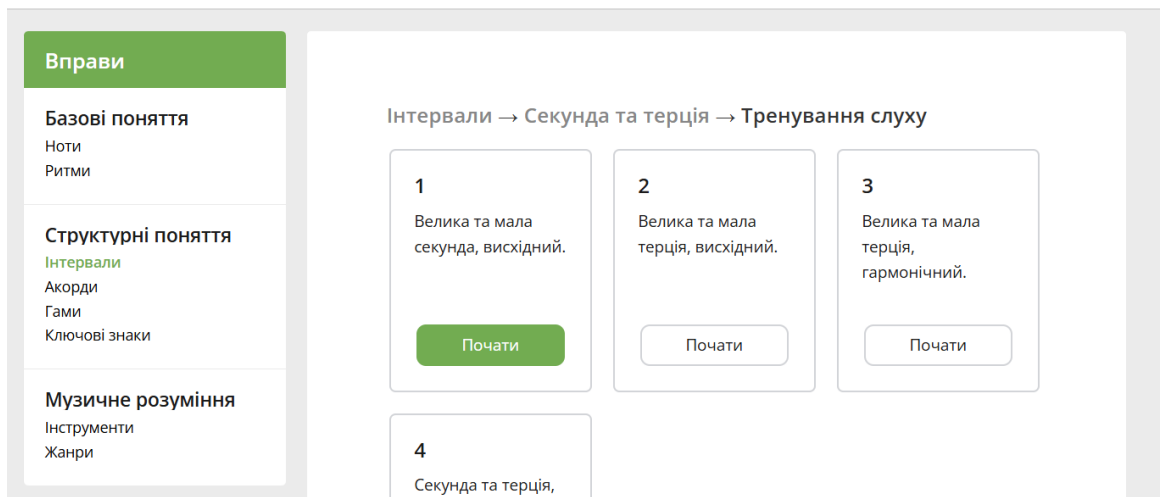


Рисунок 1.4 – Обирання категорії «Секунда та терція» та висхідного виду прослуховування

Після запуску тесту користувач бачить перед собою екран з нотним станом, варіантами відповіді і індикатором прогресу (рис. 1.5). Відбувається послідовне відтворення звучання нот інтервалу.

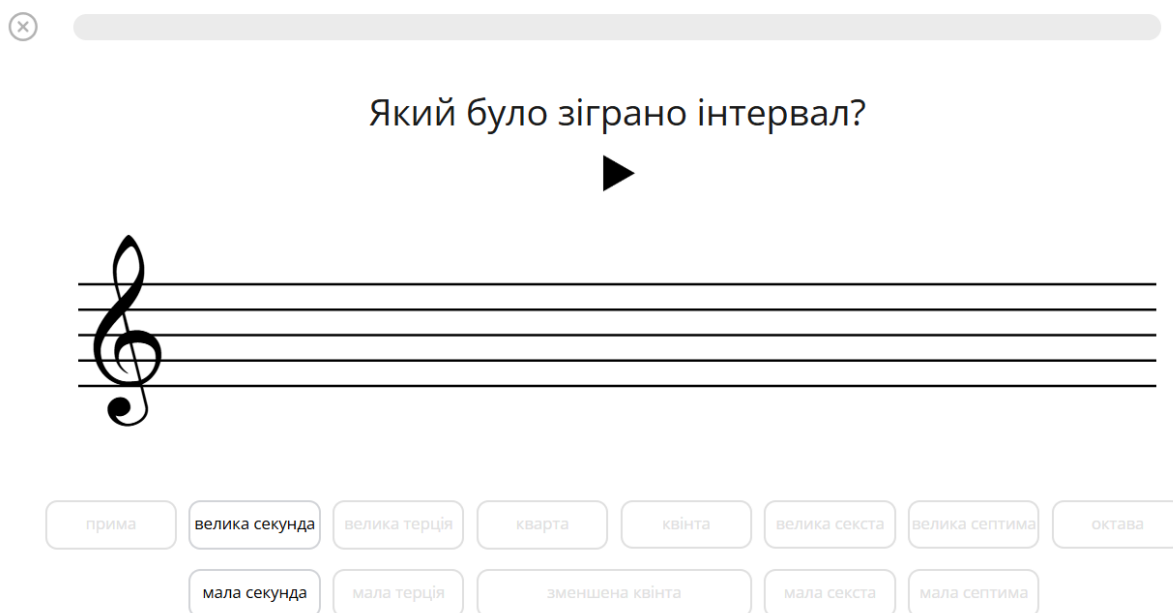


Рисунок 1.5 – Екран тесту перевірки рівня гармонічного слуху

Якщо було обрано правильну відповідь, то вона підсвічується зеленим кольором (рис. 1.6, а), у разі вибору хибної відповіді — зафарбовується червоним (рис. 1.6, б) і надається змога переобрати правильний варіант.

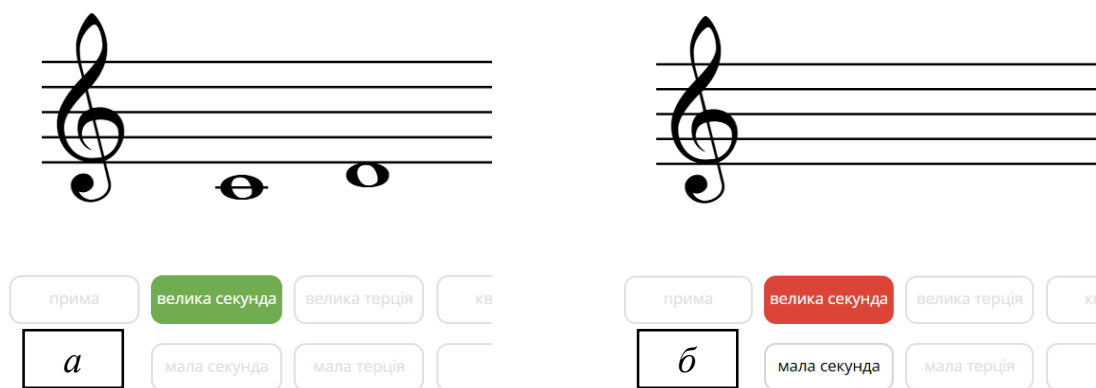


Рисунок 1.6 – Реакція модулю тренування на: обирання правильної відповіді (а), обирання хибної відповіді (б)

Після проходження всіх завдань на екран виводиться результат (рис. 1.7), в якому зазначається відсоткова точність.

## Вправу виконано!

Увійти або Зареєструватися, щоб зберегти ваші результати.



Рисунок 1.7 – Виведення результату проходження тесту на сайті Musicca

Розглянемо функціонування віртуального фортепіано (рис. 1.8).



Рисунок 1.8 – Натиснення клавіші, яка відповідає ноті «Ля»

У ході тестування віртуального фортепіано було виявлено низку базових функціональних можливостей та обмежень. Інструмент дозволяє користувачеві відтворювати звук фортепіано шляхом натискання на віртуальні клавіші. При натисканні на клавішу нота звучить протягом кількох секунд, однак одночасне звучання кількох нот не підтримується: при переході до нової ноти попередня миттєво затихає. Обрати кількість октав або змінити їхній діапазон неможливо — користувач має доступ лише до фіксованого набору нот. Позитивним аспектом є наявність підписів нот, що спрощує орієнтування, проте загальна реалізація виглядає обмеженою та не передбачає розширених освітніх можливостей.

З огляду на ідею використання віртуального фортепіано як інструменту для підтримки розвитку гармонічного слуху, під час проєктування власної системи було прийнято рішення врахувати цю функціональність у більш розширеному вигляді. Зокрема, віртуальне фортепіано буде доступне у вигляді інтерактивного елемента на сторінці тренування гармонічного слуху, що дозволить користувачеві самостійно відтворювати ноти, слухати їхнє звучання, проводити аналогії, запам'ятовувати та аналізувати інтервальні відстані, що сприятиме кращому засвоєнню матеріалу.

Реалізована клавіатура охоплюватиме діапазон від субконтроктави до п'ятої октави, з можливістю відображення позначень нот і номерів октав. Одночасне звучання нот підтримуватиметься з такими правилами:

- при звичайному натисканні звук триває 1 секунду;
- при утриманні — до 3,5 секунд в залежності від тривалості затискання;
- при активованій педалі — 7 секунд при натисканні чи затисканні.

Крім того, передбачено кнопку, яка дозволить зупинити всі активні звуки. Фортепіано розміщуватиметься на місці футера та слугуватиме важливою частиною інтерфейсу на сторінці тренування інтервалів, дозволяючи паралельно слухати звучання і відтворювати ноти вручну (на відміну від простішої реалізації в протестованому аналогові). З метою наочного порівняння функціональності

віртуального фортепіано аналогу та розробленої реалізації нижче наведено порівняльну таблицю (табл. 1.1).

Таблиця 1.1

Порівняння характеристик віртуального фортепіано

| № з/п | Характеристика                               | Musissa                     | Розроблювана система                              |
|-------|----------------------------------------------|-----------------------------|---------------------------------------------------|
| 1     | Звучання нот                                 | Одна нота одночасно         | Можливе накладання кількох нот                    |
| 2     | Тривалість звучання при натисканні           | 3 секунди                   | 1 секунда при кліку, максимум 3,5 — при утриманні |
| 3     | Підтримка педалі сустейну (подовження звуку) | Відсутня                    | Так, 7 секунд звучання при активній педалі        |
| 4     | Можливість зупинити всі звуки                | Відсутня                    | Є спеціальна кнопка зупинки                       |
| 5     | Підпис клавіш                                | Є (позначення нот)          | Є (позначення нот та октав)                       |
| 6     | Діапазон октав                               | Обмежений, не змінюється    | Від субконтроктави до п'ятої октави               |
| 7     | Можливість змінювати октави                  | Відсутня                    | Так                                               |
| 8     | Використання як навчального інструменту      | Обмежене                    | Так                                               |
| 9     | Роль у системі                               | Окремий музичний інструмент | Функціональний елемент в освітньому процесі       |

Для комплексного узагальнення результатів аналізу протестованих аналогів та визначення переваг і відмінностей розроблюваної системи нижче представлено порівняльну таблицю (табл. 1.2). Особливу увагу було приділено аспектам, які безпосередньо впливають на навчальний ефект, зокрема: підтримці гармонічного слуху, інтервального мислення та можливості взаємодії з віртуальним інструментом у реальному часі.

Порівняння особливостей аналогів та розроблюваної системи для  
розвитку гармонічного слуху

| Особливості                                                                               | Програмний продукт |         |                      |
|-------------------------------------------------------------------------------------------|--------------------|---------|----------------------|
|                                                                                           | Simply Sing        | Musicca | Розроблювана система |
| Запис аудіо з мікрофону                                                                   | Так                | Ні      | Так                  |
| Розпізнавання звучання інтервалів                                                         | Ні                 | Ні      | Так                  |
| Наявність модулю для ручного синтезування інтервалів та акордів згідно обраних параметрів | Ні                 | Ні      | Так                  |
| Наявне віртуальне фортепіано                                                              | Ні                 | Так     | Так                  |
| Тестування гармонічного слуху прослуховуванням музичних конструкцій                       | Ні                 | Так     | Так                  |
| Наявні бали досвіду                                                                       | Так                | Ні      | Так                  |
| Рекомендації після проходження тесту                                                      | Так                | Ні      | Так                  |
| Доступно для використання на ПК                                                           | Ні                 | Так     | Так                  |

Аналіз порівняльної таблиці 1.2 свідчить про те, що розроблювана система для розвитку гармонічного слуху має низку переваг у порівнянні з існуючими аналогами, поєднуючи в собі найбільш ефективні функціональні рішення та розширюючи їх можливості у навчальному контексті.

### 1.3 Вибір архітектури моделі розпізнавання системи для розвитку гармонічного слуху

Від вибору архітектурної моделі залежить не лише загальна продуктивність та масштабованість системи, а й ефективність та точність роботи алгоритмів машинного навчання при розпізнаванні музичних структур. Обрані технічні рішення мають сприяти швидкій та узгодженій взаємодії між компонентами, забезпечувати ефективну обробку аудіоданих у режимі

реального часу та створювати підґрунтя для подальшого розширення функціональності.

При виборі загальної архітектури системи розглядалися три основні підходи: монолітна архітектура, мікросервісна архітектура та класична клієнт-серверна архітектура з REST API. Простий у початковій реалізації монолітний підхід був відхилений через складність масштабування та підтримки, особливо з урахуванням різноманітності компонентів системи від вебінтерфейсу до машинного навчання. Мікросервісна архітектура була розглянута як сучасна та масштабована альтернатива, однак її впровадження вимагає значних ресурсів для розгортання та управління, що не виправдано для системи середнього масштабу.

Оптимальним рішенням стала класична клієнт-серверна архітектура із REST API як оптимальний баланс між складністю реалізації та функціональними можливостями. Такий підхід забезпечує чітке розмежування відповідальностей між презентаційним шаром та бізнес-логікою, дозволяє незалежно розвивати клієнтську та серверну частини і надає можливості для майбутнього масштабування через додавання додаткових серверних інстансів.

Вибір технологічного стеку обумовлений низкою ключових технічних та функціональних чинників. Насамперед, інтеграція компонентів машинного навчання з іншими модулями системи вимагала використання інструментів, сумісних із середовищем Python, що обумовило відмову від фреймворків на базі інших мов програмування.

Для реалізації серверної логіки обрано фреймворк Django, який забезпечує безперебійну взаємодію з такими бібліотеками, як TensorFlow, Keras, NumPy та іншими інструментами для наукових обчислень, що є необхідним для системи зі складовими штучного інтелекту. Окрім цього, Django пропонує зрілу інфраструктуру REST API (через Django REST Framework), вбудовані механізми автентифікації та безпеки, ORM для абстрагування роботи з базою даних, а також систему міграцій для версіонування її схеми [3].

Клієнтська частина реалізована у форматі Single Page Application із використанням React.js [4]. Такий підхід обрано через високі вимоги до швидкої взаємодії з аудіоінтерфейсами: запис, відтворення та аналіз звуку мають здійснюватися без затримок і перезавантажень сторінок. Архітектура SPA дозволяє динамічно оновлювати інтерфейс у відповідь на дії користувача, що особливо важливо у застосунках такого типу, де будь-яке переривання може завадити навчанню. Компонентна модель React забезпечує гнучке створення складних інструментів — від віртуальної клавіатури до системи тестування — з постійним збереженням контексту сесії.

Поточна реалізація ґрунтується на легкій локальній архітектурі з файловою базою даних SQLite, що забезпечує автономну роботу застосунку без необхідності налаштування зовнішнього серверного середовища. SQLite обрано для етапу прототипування завдяки її простоті інтеграції, відсутності сторонніх залежностей та можливості використання як вбудованої бази даних. Це рішення дає змогу запускати систему у форматі standalone-додатка для індивідуального використання.

У подальшому передбачено перехід до серверної архітектури з повнофункціональною базою даних PostgreSQL, що дозволить масштабувати систему для потреб музичних освітніх установ, забезпечити підтримку багатьох користувачів, розподілене зберігання даних і високу доступність.

При виборі архітектури моделі машинного навчання для розпізнавання музичних інтервалів було розглянуто два принципово різних підходи, кожен з яких має свої переваги та обмеження залежно від специфіки задачі класифікації аудіосигналів.

Перший підхід базується на використанні згорткових нейронних мереж для обробки спектрограм (рис. 1.9), згенерованих за допомогою короткочасного перетворення Фур'є [5]. У цьому методі аудіосигнал перетворюється у двовимірне зображення спектрограми, де вісь часу та вісь частот формують матрицю інтенсивностей. Згорткові нейронні мережі демонструють високу

ефективність при роботі з такими зображеннями завдяки здатності виявляти локальні патерни та ієрархічні ознаки. Для задач з невеликою кількістю класів, наприклад бінарної класифікації, цей підхід показує відмінні результати з високою точністю класифікації.

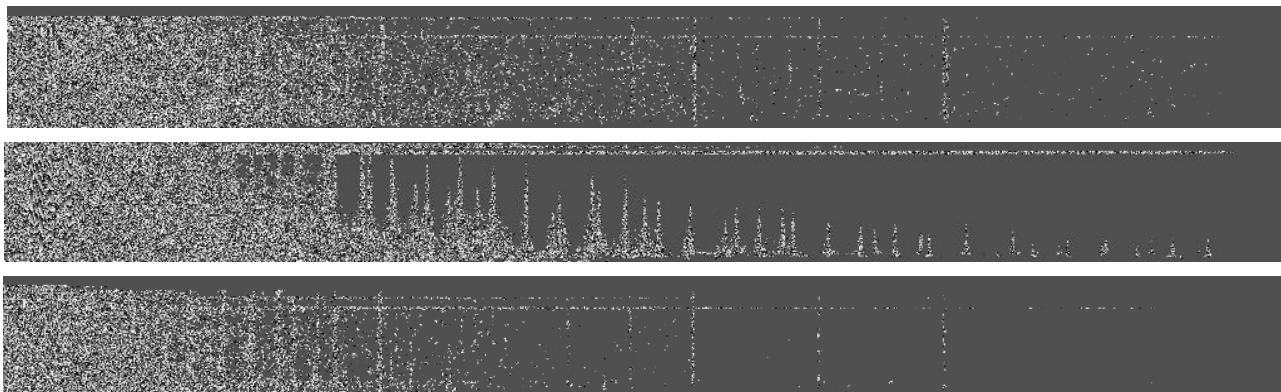


Рисунок 1.9 – Зразки згенерованих спектрограм з аудіоприкладів

Однак при збільшенні кількості класів до 12, що відповідає кількості основних музичних інтервалів, згорткові мережі стикаються з явищем плато навчання, коли покращення точності фактично припиняється на ранніх етапах тренування. Це пов'язано з тим, що спектрограми різних музичних інтервалів демонструють високий ступінь візуальної схожості, особливо за наявності варіацій тембру, гучності або гармонічного контексту. У таких умовах модель не здатна ефективно виокремити розрізняльні ознаки, що призводить до слабкої диференціації між класами і, як наслідок, точність класифікації стабілізується на рівні 10 – 11%, що практично відповідає випадковому вгадуванню.

Другий підхід полягає у використанні глибокої нейронної мережі з повнозв'язними шарами, яка працює з попередньо обчисленими спектральними ознаками та характеристиками аудіосигналу. Замість подачі сирих спектрограм на вхід нейронної мережі, система виконує комплексне виділення ознак за допомогою швидкого перетворення Фур'є та додаткових алгоритмів аналізу сигналу. Цей підхід виявився значно більш ефективним для задачі розпізнавання музичних інтервалів з великою кількістю класів.

Ключова перевага глибокої нейронної мережі з попередньою обробкою полягає в тому, що короткочасне перетворення Фур'є бере на себе основну

роботу з виділення релевантних спектральних характеристик, дозволяючи створити відносно компакту архітектуру нейронної мережі. Система виділяє такі ознаки як спектральний центроїд, спектральну ширину, спектральний rolloff, характеристики піків у частотній області, співвідношення між гармоніками, RMS енергію, zero crossing rate та інші музично-релевантні параметри. Такий підхід дозволяє мережі зосередитися на вивченні взаємозв'язків між цими високорівневими ознаками замість пошуку елементарних патернів у сирих даних.

#### **1.4 Обґрунтування вибору інструментальних засобів**

Вибір інструментальних засобів для розробки системи розвитку гармонічного слуху визначає не лише технічну реалізацію, а й продуктивність, масштабованість та подальший розвиток проєкту. Кожен компонент технологічного стеку був ретельно обраний з урахуванням специфічних вимог до обробки аудіоданих, машинного навчання та створення інтерактивного веб-інтерфейсу, а також на основі порівняльного аналізу альтернативних рішень.

Фреймворк Django було обрано як основу серверної частини на основі кількох факторів, що робить його оптимальним вибором для проєктів з компонентами машинного навчання [6]. Головною перевагою Django є його нативна інтеграція з екосистемою Python, що дозволяє безпосередню роботу з бібліотеками машинного навчання без необхідності створення окремих сервісів чи API обгортки.

Як альтернатива розглядався FastAPI — сучасний асинхронний фреймворк, відомий своєю високою продуктивністю, однак орієнтований він переважно на розробку API з мінімальним шаблоном функціональності [7]. FastAPI не надає з коробки інтегрованих механізмів автентифікації, управління користувачами чи адміністративного інтерфейсу, тому реалізація подібної логіки вимагала б підключення сторонніх бібліотек і розробки індивідуальних рішень. Хоча екосистема FastAPI активно зростає, вона менш інтегрована порівняно з

Django, що може ускладнити реалізацію комплексної серверної логіки в межах одного фреймворку.

Flask також розглядався як мінімалістична альтернатива, що надає максимальну гнучкість в архітектурних рішеннях [8]. Водночас, така гнучкість вимагала б значних витрат на налаштування базової інфраструктури, оскільки Flask не містить вбудованих засобів ORM, системи міграцій, засобів автентифікації та адміністративного інтерфейсу.

Django у поєднанні з Django REST Framework забезпечує потужні засоби для створення API з мінімальними зусиллями, включаючи автоматичну серіалізацію моделей, валідацію даних, пагінацію та фільтрацію. Вбудована підтримка різних форматів автентифікації, включаючи JWT токени через пакет djangorestframework-simplejwt, робить Django ідеальним вибором для створення безпечних API для Single Page Applications.

React.js обрано для створення клієнтської частини на основі специфічних вимог до роботи з аудіоконтентом та інтерактивністю користувацького інтерфейсу [9]. Ключовою перевагою React є його ефективна система управління станом та віртуальний DOM, що забезпечує плавну роботу з динамічним контентом без перезавантаження сторінок.

Vue.js розглядався як альтернатива через свою простоту вивчення та прогресивний підхід до впровадження [10]. Однак Vue має менш активну екосистему готових компонентів для роботи з Web Audio API та аудіоінтерфейсами, що обмежує можливості швидкої інтеграції спеціалізованих бібліотек. React має кращу підтримку специфічних бібліотек як Tone.js для синтезу звуку та більше готових компонентів для створення аудіоінтерфейсів [11].

Angular відхилено через його складність та надмірність для проєкту середнього масштабу. TypeScript, хоча й забезпечує строгішу типізацію, є обов'язковим у Angular, що підвищує вхідний поріг і складність початкової

розробки без критичних переваг для невеликих проєктів з фокусом на клієнтський інтерфейс [12].

Для збирання клієнтської частини обрано Vite — сучасний інструмент, що забезпечує швидкий запуск і оновлення інтерфейсу під час розробки [13]. У порівнянні з традиційними рішеннями, такими як Create React App, Vite працює значно швидше завдяки використанню новітніх можливостей браузера, зокрема підтримки модулів JavaScript (ES-модулів), які дозволяють підвантажувати лише змінені частини коду без повторної збірки всього проєкту. Це дає змогу миттєво бачити зміни в інтерфейсі без перезавпуску всього застосунку.

TensorFlow обрано як основний фреймворк для машинного навчання на основі кількох факторів. Першочерговою перевагою є зрілість платформи та її готовність до продакшн використання. TensorFlow має розвинену екосистему інструментів для розгортання моделей, включаючи TensorFlow Serving для високопродуктивного інференсу та TensorFlow Lite для мобільних пристроїв, що забезпечує шляхи для майбутнього розширення системи [14].

PyTorch розглядався як альтернатива через його динамічний граф обчислень та зручність для дослідницьких задач [15]. Однак PyTorch має менш централізовану інфраструктуру для розгортання моделей у промисловому середовищі, а підтримка інтеграції з вебдодатками вимагає додаткової конфігурації порівняно з рішеннями в екосистемі TensorFlow. TorchServe, хоча й підтримується спільнотою та компанією AWS, поступається TensorFlow Serving за рівнем документації, кількістю готових інтеграцій і гнучкістю налаштувань. Крім того, екосистема TensorFlow має кращу сумісність із фреймворками на Python, зокрема Django, завдяки стабільним API, простій серіалізації моделей і широкій підтримці інструментів для серверної інтеграції.

Keras API у складі TensorFlow 2.x забезпечує високорівневий інтерфейс для швидкого прототипування та створення моделей, одночасно зберігаючи можливості для низькорівневих оптимізацій [16]. Eager execution режим робить

розробку та відлагодження моделей значно простішими порівняно з графовим режимом попередніх версій TensorFlow.

Scikit-learn обрано для допоміжних задач машинного навчання через його стабільність, обширну документацію та інтеграцію з NumPy екосистемою. StandardScaler забезпечує ефективну нормалізацію ознак, що важливо для стабільності навчання нейронних мереж [17]. Joblib використовується для серіалізації моделей та трансформерів завдяки його оптимізації для роботи з NumPy масивами та кращій продуктивності порівняно зі стандартним pickle.

Librosa обрано як основну бібліотеку для аналізу аудіосигналів через її спеціалізацію на музичних та аудіоданих [18]. На відміну від загальних бібліотек обробки сигналів, Librosa надає готові реалізації музично-орієнтованих алгоритмів, оптимізована для роботи з музичними сигналами та має обширну документацію з прикладами використання для задач класифікації музичного контенту.

PyAudio розглядався для прямої роботи з аудіообладнанням, але був відхилений через складність встановлення та залежність від системних бібліотек, зокрема PortAudio, що створює труднощі з підтримкою на різних операційних системах [19]. Librosa забезпечує більш високорівневий API та краще інтегрується з NumPy екосистемою без прямих залежностей від аудіодрайверів.

SoundFile та audioread забезпечують надійне читання різних форматів аудіофайлів. Ці бібліотеки обрані через їх стабільність та підтримку широкого спектру форматів з мінімальними залежностями та стабільною підтримкою популярних форматів аудіофайлів. SoundFile особливо ефективний для читання WAV файлів, що є основним форматом для обробки в системі.

NumPy та SciPy є фундаментальними компонентами для всіх математичних операцій та алгоритмів обробки сигналів. NumPy забезпечує ефективні операції з багатовимірними масивами, що корисно для продуктивності при обробці аудіоданих [20]. SciPy розширює можливості NumPy

спеціалізованими алгоритмами для обробки сигналів, включаючи фільтрацію та спектральний аналіз [21].

JWT токени через `django-rest-framework-simplejwt` обрані для автентифікації через їх stateless природу, що важливо для масштабованості Single Page Applications. На відміну від session-based автентифікації, JWT не потребують серверного зберігання стану та можуть бути валідовані незалежно кожним серверним інстансом.

CORS middleware через `django-cors-headers` необхідний для безпечної взаємодії між клієнтською та серверною частинами під час розробки та у продакшн середовищі з різними доменами [22]. Правильне налаштування CORS важливо для безпеки вебдодатків та для запобігання небезпечним запитам з непередбачених доменів, які можуть сприяти атакам типу Cross-Site Request Forgery при неправильному налаштуванні.

Sass використовується для стилізації через можливості вкладених селекторів, змінних та міксинів, що робить CSS код більш організованим та підтримуваним. Код на Sass компілюється у звичайний CSS, що забезпечує сумісність з усіма браузерами й не потребує додаткових клієнтських залежностей.

Обраний набір інструментальних засобів формує узгоджену та взаємодоповнювану екосистему, в якій кожен компонент посилює загальну ефективність розробки, тестування та розгортання системи розвитку гармонічного слуху, забезпечуючи її масштабованість і готовність до подальшого функціонального розширення.

### **Висновки до першого розділу**

У першому розділі було проведено всебічний аналіз вимог до системи для розвитку гармонічного слуху, що поєднує інтерфейс користувача, обробку аудіосигналів та використання нейронних мереж. Обґрунтовано вибір стеку технологій відповідно до функціональних і нефункціональних вимог. Django у

поєднанні з Django REST Framework забезпечує стабільну основу для побудови backend-частини, завдяки вбудованим механізмам автентифікації, ORM та гнучкій API-архітектурі. React.js обрано як основу клієнтської частини за здатність ефективно працювати з аудіоконтентом у реальному часі та забезпечувати високий рівень інтерактивності.

TensorFlow з Keras API надає потужні інструменти для створення, навчання та розгортання згорткових нейронних мереж, що дозволяє реалізувати точне розпізнавання музичних інтервалів із перспективою подальшого масштабування. Допоміжні бібліотеки, такі як Librosa, SoundFile, NumPy та SciPy, створюють основу для якісного аналізу аудіосигналів. Інструменти конфігурації та підтримки як Vite, ESLint, Sass, dotenv і JWT/CORS middleware забезпечують зручність у розробці, тестуванні та безпечному розгортанні SPA-додатку.

Сформована архітектура дозволяє досягти високої модульності, масштабованості та продуктивності системи, а обґрунтований вибір кожного інструмента підтверджує її готовність до використання у реальних умовах навчання музики та подальшого науково-технічного розвитку.

## РОЗДІЛ 2. ПРОЄКТУВАННЯ СИСТЕМИ ДЛЯ РОЗВИТКУ ГАРМОНІЧНОГО СЛУХУ

### 2.1 Визначення варіантів використання та структури системи

При аналізі системи для розвитку гармонічного слуху було виявлено двох основних типів акторів. Кожен тип має чітко визначені права доступу та можливості взаємодії з різними модулями системи. Неавторизовані користувачі отримують доступ виключно до процедур автентифікації, що забезпечує контрольований вхід до системи та захист персональних даних. Авторизовані користувачі, натомість, отримують повний спектр навчальних інструментів та можливості персоналізації досвіду. Короткий опис акторів представлено в табл. 2.1.

Таблиця 2.1

Виявлення акторів

| Актор                    | Короткий опис                                                                                                          | Основні можливості                                                                                                                                                                                                                                                                                                                             |
|--------------------------|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Гість                    | Неавторизований користувач системи, який має обмежений доступ лише до процедур входу в систему                         | Має доступ виключно до форм автентифікації: може створити новий обліковий запис через форму реєстрації або увійти в систему через форму входу з існуючими обліковими даними. Не має доступу до жодної функціональності системи до моменту успішної автентифікації                                                                              |
| Авторизований користувач | Зареєстрований та автентифікований користувач системи, який має повний доступ до всіх функціональних модулів платформи | Має доступ до всіх основних функцій системи: запис аудіо через мікрофон, автоматичне розпізнавання музичних інтервалів за допомогою ML-алгоритмів, повнофункціональне тренування з усіма доступними інтервалами, використання клавіатури віртуального фортепіано, проходження структурованих тестів з детальною аналітикою та отриманням балів |

Для повноцінного моделювання функціональних вимог системи необхідно систематизувати всі можливі сценарії взаємодії користувачів з платформою. У табл. 2.2 – 2.8 представлено структурований перелік виявлених варіантів використання, які слугуватимуть основою для візуалізації поведінкових аспектів системи та подальшого архітектурного проєктування.

Таблиця 2.2

Варіант використання «Реєстрація»

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Варіант використання   | UC-01: Реєстрація                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Актор                  | Гість                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Мета                   | Створення нового облікового запису для отримання доступу до функціональності системи                                                                                                                                                                                                                                                                                                                                                                                                     |
| Передумови             | Користувач не має облікового запису в системі                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Тригер                 | Натискання кнопки «Зареєструватися»                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Основний сценарій      | <ol style="list-style-type: none"> <li>1. Гість переходить на сторінку реєстрації.</li> <li>2. Заповнює форму з полями «Логін» та «Пароль».</li> <li>3. Система виконує валідацію введених даних.</li> <li>4. При натисканні «Зареєструватися» дані передаються на сервер для обробки.</li> <li>5. Система перевіряє унікальність логіна та створює новий обліковий запис.</li> <li>6. Користувач перенаправляється на сторінку входу з повідомленням про успішну реєстрацію.</li> </ol> |
| Альтернативні сценарії | <ol style="list-style-type: none"> <li>1. У разі, якщо обраний логін уже використовується іншим користувачем, система відображає повідомлення з вимогою вказати інший логін.</li> <li>2. Якщо введений пароль не відповідає вимогам, система відображає відповідне повідомлення.</li> <li>3. У разі виникнення мережевої помилки система зберігає введені користувачем дані та пропонує повторити спробу після відновлення з'єднання.</li> </ol>                                         |
| Постумови              | В системі створено новий обліковий запис з базовим профілем користувача                                                                                                                                                                                                                                                                                                                                                                                                                  |

Таблиця 2.3

## Варіант використання «Вхід в систему»

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Варіант використання   | UC-02: Вхід в систему                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Актор                  | Гість                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Мета                   | Автентифікація для отримання доступу до функціональності                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Передумови             | Користувач має дійсний обліковий запис                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Тригер                 | Натискання кнопки "Увійти"                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Основний сценарій      | <ol style="list-style-type: none"> <li>1. Гість вводить логін та пароль на сторінці входу.</li> <li>2. Система виконує базову валідацію заповненості полів.</li> <li>3. Облікові дані передаються на сервер, система перевіряє відповідність даних записам в базі даних.</li> <li>4. При успішній автентифікації генеруються токени безпеки для підтримки сесії.</li> <li>5. Токени зберігаються у браузері користувача.</li> <li>6. Користувач перенаправляється на головну сторінку з повним доступом до функцій.</li> </ol> |
| Альтернативні сценарії | <ol style="list-style-type: none"> <li>1. У разі введення неправильного логіна або пароля система відображає загальне повідомлення про помилку автентифікації.</li> <li>2. У випадку внутрішньої помилки сервера система повідомляє про збій та пропонує повторити спробу входу пізніше.</li> </ol>                                                                                                                                                                                                                            |
| Постумови              | Користувач автентифікований, має доступ до захищених маршрутів                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

Таблиця 2.4

## Варіант використання «Розпізнавання музичних конструкцій»

|                      |                                                                                                     |
|----------------------|-----------------------------------------------------------------------------------------------------|
| Варіант використання | UC-03: Розпізнавання музичних конструкцій                                                           |
| Актор                | Авторизований користувач                                                                            |
| Мета                 | Автоматичне визначення музичних інтервалів з аудіозапису за допомогою технологій машинного навчання |
| Передумови           | Користувач автентифікований, має доступ до мікрофона, система розпізнавання готова до роботи        |

| Тригер                 | Перехід на сторінку розпізнавання аудіо                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Основний сценарій      | <ol style="list-style-type: none"> <li>1. Користувач переходить на сторінку аудіорозпізнавання.</li> <li>2. Система перевіряє готовність модуля машинного навчання.</li> <li>3. Запитується дозвіл на доступ до мікрофона користувача.</li> <li>4. Користувач розпочинає запис натисканням відповідної кнопки, система ініціалізує аудіозапис.</li> <li>5. Під час запису відображається таймер, який відраховує довжину запису.</li> <li>6. Користувач зупиняє запис та може прослухати записаний фрагмент.</li> <li>7. Натисканням кнопки «Розпізнати інтервал» ініціюється процес розпізнавання.</li> <li>8. Система обробляє аудіосигнал та повертає результати з рівнями достовірності.</li> </ol> |
| Альтернативні сценарії | <ol style="list-style-type: none"> <li>1. Якщо модуль розпізнавання тимчасово недоступний, система інформує користувача про технічні роботи та просить спробувати пізніше.</li> <li>2. У разі відсутності доступу до мікрофона система відображає повідомлення.</li> </ol>                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Постумови              | Користувач отримав результат розпізнавання з детальною аналітикою                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

Таблиця 2.5

## Варіант використання «Тренування гармонічного слуху»

|                      |                                                                                |
|----------------------|--------------------------------------------------------------------------------|
| Варіант використання | УС-04: Тренування гармонічного слуху                                           |
| Актор                | Авторизований користувач                                                       |
| Мета                 | Розвиток навичок розпізнавання музичних інтервалів через інтерактивну практику |
| Передумови           | Параметри генерації налаштовані та валідовані                                  |
| Тригер               | Натискання кнопки генерації навчальних матеріалів                              |
| Основний сценарій    | 1. Система отримує встановлені налаштування користувача для генерації.         |

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Основний сценарій      | <p>2. Для кожного обраного інтервалу розраховуються точні частоти звучання.</p> <p>3. Здійснюється синтез аудіосигналів.</p> <p>4. Створюються два варіанти звучання: цілісний та поступовий.</p> <p>5. Аудіофайли організовуються в структуровану систему з контрольованим доступом.</p> <p>6. Відображається інтерфейс з картками інтервалів для інтерактивного навчання.</p> <p>7. Користувач може прослуховувати матеріали в довільному порядку.</p> |
| Альтернативні сценарії | 1. У разі виникнення помилки під час синтезу аудіо система інформує користувача про збій та надає можливість повторити генерацію.                                                                                                                                                                                                                                                                                                                        |
| Постумови              | Створені навчальні матеріали готові для інтерактивного використання                                                                                                                                                                                                                                                                                                                                                                                      |

Таблиця 2.6

## Варіант використання «Використання клавіатури фортепіано»

|                      |                                                                                                                                                                                                                                                  |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Варіант використання | UC-05: Використання клавіатури фортепіано                                                                                                                                                                                                        |
| Актор                | Авторизований користувач                                                                                                                                                                                                                         |
| Мета                 | Надання додаткового інтерактивного музичного інструменту для практики та експериментування з музичними конструкціями                                                                                                                             |
| Передумови           | Користувач автентифікований, перебуває на сторінці тренування                                                                                                                                                                                    |
| Тригер               | Активація віртуального фортепіано зі сторінки тренування гармонічного слуху                                                                                                                                                                      |
| Основний сценарій    | <p>1. Користувач натискає кнопку активації віртуального музичного інструменту.</p> <p>2. Система ініціалізує аудіоконтекст та завантажує звукові семпли.</p> <p>3. Відображається інтерактивна клавіатура з налаштовуваним діапазоном октав.</p> |

|                        |                                                                                                                                                                                                                                                                                                                                                                |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Основний сценарій      | <p>4. Користувач може взаємодіяти з клавішами за допомогою миші або сенсорного введення.</p> <p>5. Кожне натискання генерує реалістичний звук з природною динамікою та тембром, доступна функція педалі сустейну для продовження звучання та функція зупинки всіх звучань.</p> <p>6. Користувач може налаштовувати відображення назв нот та номерів октав.</p> |
| Альтернативні сценарії | 1. У разі повільного завантаження звукових файлів система відображає індикатор прогресу та інформує користувача про очікування.                                                                                                                                                                                                                                |
| Постумови              | Користувач має доступ до повнофункціонального віртуального музичного інструменту для практики та навчання                                                                                                                                                                                                                                                      |

Таблиця 2.7

## Варіант використання «Тестування гармонічного слуху»

|                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Варіант використання | UC-06: Тестування гармонічного слуху                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Актор                | Авторизований користувач                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Мета                 | Комплексне оцінювання рівня розвитку гармонічного слуху з детальною аналітикою                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Передумови           | Параметри тестування повністю встановлені та успішно пройшли системну валідацію                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Тригер               | Натискання кнопки початку тестування                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Основний сценарій    | <p>1. Система створює індивідуальну тестову сесію з унікальними параметрами та ідентифікатором.</p> <p>2. Генеруються питання відповідно до налаштованої конфігурації з випадковими варіаціями.</p> <p>3. Для кожного тестового завдання створюються унікальні аудіозразки звучання інтервалу.</p> <p>4. Ініціалізується інтерфейс тестування з елементами навігації та індикаторами прогресу.</p> <p>5. Питання подаються послідовно з можливістю прослуховування цілісних та поступових варіантів.</p> |

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Основний сценарій      | 6. При завершенні всіх питань або досягненні часового ліміту тест автоматично завершується.<br>7. Обчислюються комплексні метрики та генерується звіт з персоналізованими рекомендаціями для подальшого розвитку.                                                                                                                                                                                                                                                       |
| Альтернативні сценарії | 1. У разі, якщо користувач самостійно перериває тестування, система автоматично зберігає досягнутий прогрес для можливого подальшого відновлення.<br>2. У разі виникнення технічних збоїв система забезпечує надійне автоматичне збереження поточного стану та відновлення сесії після усунення проблем.<br>3. Після завершення відведеного часу система автоматично завершує тестування та обчислює фінальні результати на основі всіх виконаних користувачем завдань. |
| Постумови              | Результати збережені в профілі користувача, досвід коректно нарахований, статистика актуалізована, згенеровані персональні рекомендації                                                                                                                                                                                                                                                                                                                                 |

Таблиця 2.8

## Варіант використання «Вихід із системи»

|                      |                                                                                                                                                                                                                                                                             |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Варіант використання | UC-07: Вихід із системи                                                                                                                                                                                                                                                     |
| Актор                | Авторизований користувач                                                                                                                                                                                                                                                    |
| Мета                 | Безпечне завершення робочої сесії користувача з надійним збереженням усіх важливих даних                                                                                                                                                                                    |
| Передумови           | Користувач автентифікований та активно використовує функціональність системи                                                                                                                                                                                                |
| Тригер               | Свідомий вибір опції виходу з системи                                                                                                                                                                                                                                       |
| Основний сценарій    | 1. Користувач свідомо ініціює процедуру безпечного завершення робочої сесії.<br>2. Очищуються токени безпеки та конфіденційні персональні дані з локального сховища браузера.<br>3. Сесія користувача інвалідується на рівні сервера, зупиняються всі активні аудіопроцеси. |

|                        |                                                                                                                                                                                        |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Основний сценарій      | 4. Користувач перенаправляється на сторінку входу.<br>5. Статус користувача змінюється на неавторизований з відповідними обмеженими правами доступу.                                   |
| Альтернативні сценарії | 1. У разі виникнення мережевої помилки під час виходу із системи здійснюється гарантоване локальне очищення критичних даних та коректне перенаправлення користувача на сторінку входу. |
| Постумови              | Сесія користувача повністю та безпечно завершена, доступ до персональних функцій та даних надійно обмежений                                                                            |

Для повного розуміння функціональної архітектури системи необхідно також розглянути допоміжні та розширювальні варіанти використання, які забезпечують детальну взаємодію з основними модулями платформи (табл. 2.9). Ці варіанти використання реалізують специфічні підпроцеси, розширюють базову функціональність та створюють зв'язки між різними компонентами системи.

Таблиця 2.9

## Пов'язані варіанти використання системи

| № з/п | Модуль        | Найменування                                   | Формулювання                                                                                                                      |
|-------|---------------|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| 1     | Розпізнавання | Перегляд стану готовності моделі               | Автоматична перевірка готовності системи машинного навчання до обробки аудіозапитів з відображенням поточного статусу користувачу |
| 2     | Розпізнавання | Запис звучання музичного інтервалу з мікрофона | Захоплення аудіосигналу через мікрофон користувача з оптимальними параметрами для подальшого аналізу                              |

|   |               |                                                 |                                                                                                                                 |
|---|---------------|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| 3 | Розпізнавання | Відтворення звучання записаного фрагменту       | Можливість прослуховування записаного користувачем аудіофрагменту для перевірки якості перед ініціацією процесу розпізнавання   |
| 4 | Розпізнавання | Видалення запису                                | Очищення поточного аудіозапису з метою створення нового або скасування процесу розпізнавання                                    |
| 5 | Розпізнавання | Виведення результату розпізнавання              | Представлення результатів автоматичного аналізу у зрозумілому форматі з рівнями достовірності та альтернативними варіантами     |
| 6 | Тренування    | Налаштування параметрів генерації               | Конфігурація параметрів для створення персоналізованих навчальних аудіоматеріалів з вибором базової ноти та музичних інтервалів |
| 7 | Тренування    | Налаштування параметрів відображення клавіатури | Персоналізація віртуального фортепіано через вибір діапазону октав, відображення назв нот та номерів відображених октав         |
| 8 | Тренування    | Відтворення нот на клавіатурі                   | Генерація реалістичного звуку при взаємодії з віртуальними клавішами з підтримкою поліфонії та педалі сустейну                  |
| 9 | Тренування    | Виведення панелі згенерованих інтервалів        | Організоване відображення створених навчальних аудіозразків у вигляді інтерактивних карток для зручного прослуховування         |

|    |                          |                                                 |                                                                                                                                  |
|----|--------------------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| 10 | Тренування та Тестування | Відтворення цілісного звучання                  | Прослуховування гармонічного варіанту музичного інтервалу, де обидві ноти звучать одночасно                                      |
| 11 | Тренування та Тестування | Відтворення поступового звучання                | Прослуховування мелодичного варіанту музичного інтервалу, де ноти відтворюються послідовно одна за одною                         |
| 12 | Тренування               | Очистити панель згенерованих інтервалів         | Видалення поточних навчальних матеріалів та підготовка інтерфейсу до створення нової тренувальної сесії                          |
| 13 | Тестування               | Налаштування параметрів тесту                   | Конфігурація параметрів для проведення структурованого оцінювання музичних навичок з налаштуванням складності та типу тестування |
| 14 | Тестування               | Проходження тесту на визначення назви інтервалу | Основний процес формального оцінювання здатності користувача розпізнавати різні типи музичних інтервалів                         |
| 15 | Тестування               | Виведення результатів проходження тесту         | Комплексний аналіз та представлення результатів тестування з детальною статистикою та рекомендаціями                             |
| 16 | Тестування               | Отримання балів досвіду                         | Нарахування винагороди у вигляді балів досвіду за успішне виконання навчальних завдань та проходження тестів                     |

Проведений детальний аналіз функціональних вимог та специфікація варіантів використання створили основу для візуального моделювання взаємодії користувачів з системою. На рис. 2.1 наведено діаграму варіантів використання системи для розвитку гармонічного слуху, яка наочно демонструє повний спектр функціональних можливостей. Діаграма відображає зв'язки між варіантами використання та розподіл відповідальності між акторами системи.

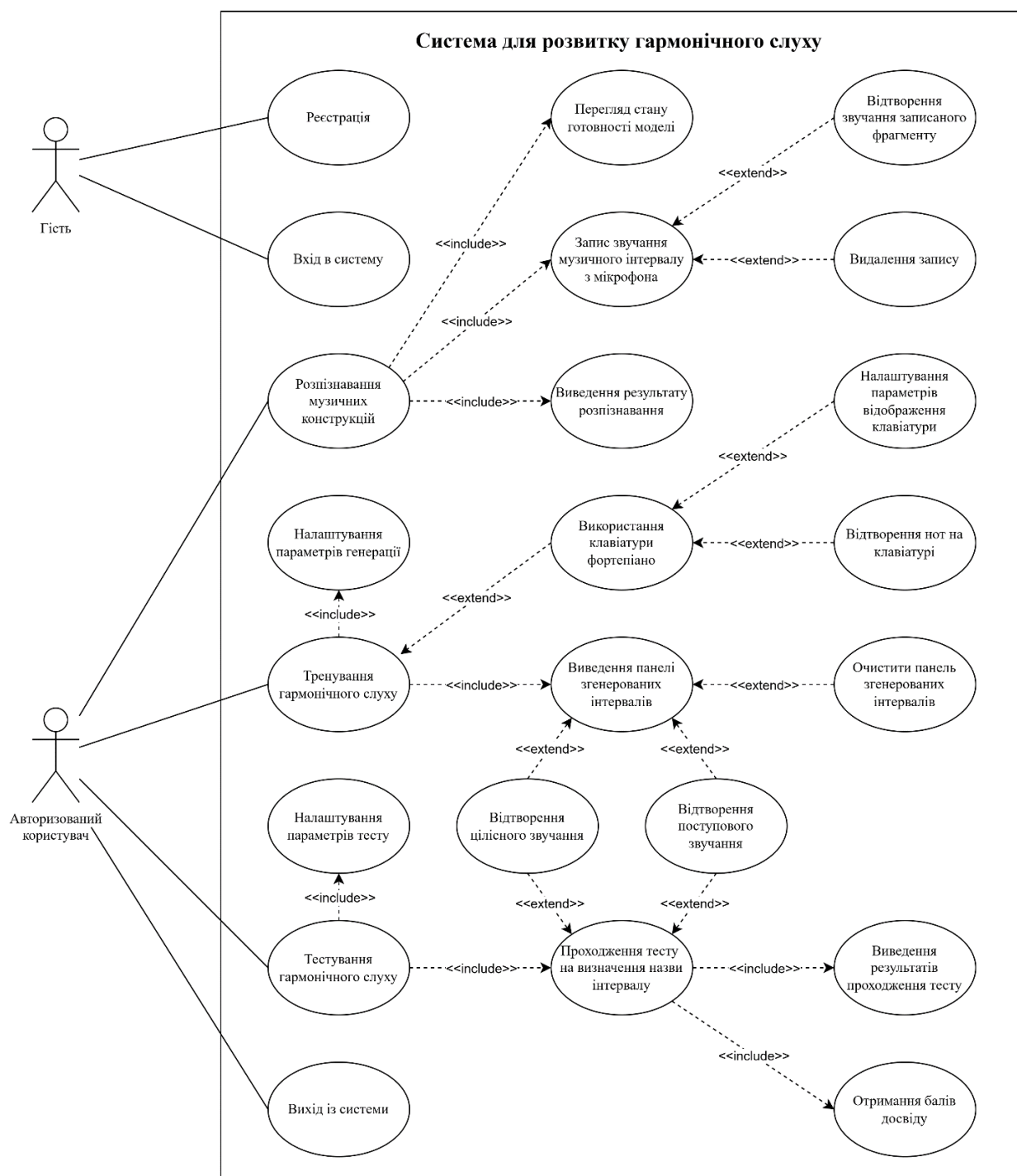


Рисунок 2.1 – Діаграма варіантів використання системи

Система реалізована за принципом клієнт-серверної архітектури з чітким розподілом відповідальності між компонентами. Основою серверної логіки є кілька ключових модулів. Модуль автентифікації та авторизації відповідає за безпечний доступ користувачів до системи, використовуючи JWT токени для підтримки сесій. Модуль управління користувачами зберігає інформацію про профілі, досягнення та прогрес навчання. Центральним компонентом є модуль розпізнавання музичних інтервалів, який інтегрує нейронну мережу для аналізу аудіосигналів. Структура системи наведена на рис. 2.2.

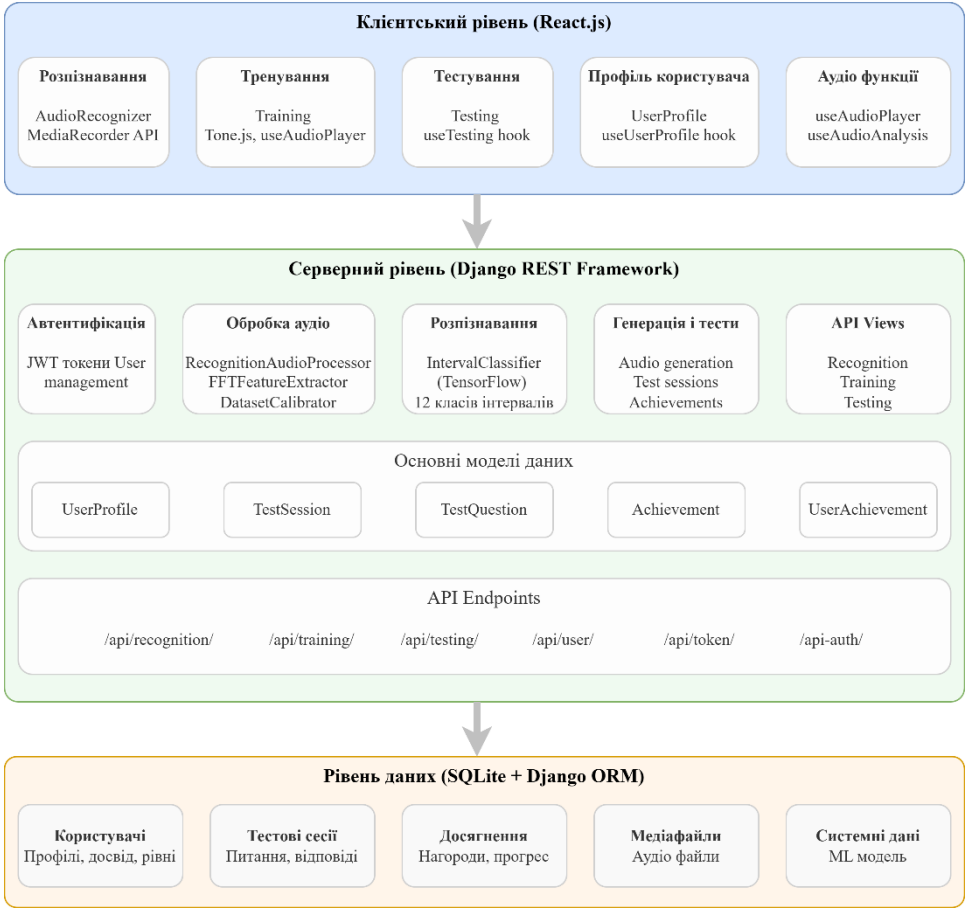


Рисунок 2.2 – Структура системи розпізнавання музичних інтервалів

Модуль обробки аудіо реалізований з використанням бібліотек librosa та scipy для цифрової обробки сигналів. Цей компонент відповідає за нормалізацію вхідних аудіоданих, видалення шумів та підготовку сигналу для подальшого аналізу. Модуль виділення ознак використовує швидке перетворення Фур'є для отримання спектральних характеристик аудіосигналу, які потім передаються нейронній мережі.

Нейронна мережа для розпізнавання інтервалів реалізована на базі TensorFlow та складається з повнозв'язних шарів з dropout-регуляризацією для запобігання перенавчанню. Модель навчена на великому датасеті музичних інтервалів і показує високу точність розпізнавання.

Модуль генерації навчальних матеріалів створює синтетичні аудіо інтервали з використанням математичних моделей музичних звуків. Цей компонент забезпечує створення якісних навчальних зразків з різними тембрами та динамічними характеристиками.

Система тестування включає модуль створення персоналізованих тестів, який адаптує складність завдань відповідно до рівня користувача. Модуль аналізу результатів обчислює детальну статистику виконання тестів та надає рекомендації для подальшого навчання.

Клієнтська частина системи розроблена з використанням React, який обрано через його компонентний підхід, що дозволяє створювати повторно використовувані елементи інтерфейсу та ефективно управляти станом додатка.

Архітектура фронтенду побудована за принципом розділення на компоненти згідно з функціональними областями. Компоненти розпізнавання відповідають за запис аудіо, його відправлення на сервер та відображення результатів аналізу. Компоненти тренування забезпечують інтерфейс для вибору навчальних матеріалів та їх відтворення. Компоненти тестування реалізують логіку проходження тестів та відображення результатів.

Для управління станом додатка використовуються React hooks, які забезпечують ефективну синхронізацію між компонентами. Система маршрутизації реалізована за допомогою React Router, що дозволяє створювати односторінковий додаток з плавними переходами між розділами.

Аудіо функціональність на клієнті реалізована з використанням Web Audio API та MediaRecorder API для запису та відтворення звуку безпосередньо в браузері. Для синтезу звуків фортепіано використовується бібліотека Tone.js, яка надає потужні інструменти для генерації та маніпуляції аудіосигналами.

Схема бази даних включає таблиці користувачів, сесій тестування, питань та відповідей, досягнень та статистики прогресу. Використання ORM Django забезпечує безпечну роботу з базою даних та автоматичну міграцію схеми при оновленнях. Система кешування реалізована на рівні додатка для оптимізації продуктивності при роботі з часто запитуваними даними, такими як аудіо файли та результати аналізу.

Розглянута структура системи створює необхідні передумови для детального проєктування алгоритмів роботи окремих компонентів та їх практичної реалізації.

## **2.2 Проєктування та реалізація алгоритмів роботи системи**

Для повноцінного функціонування системи було реалізовано три ключові програмні модулі, які покривають базові сценарії використання.

Центральне місце у системі займає модуль, що відповідає за автоматичне розпізнавання музичних інтервалів за вхідним аудіосигналом. Його реалізація поєднує інструменти цифрової обробки сигналів з методами машинного навчання, що дозволяє досягати прийнятної точності навіть за умов невисокої якості звуку.

Основу модуля становить багат шаровий перцептрон із чотирма послідовно з'єднаними повнозв'язними шарами, у яких кількість нейронів зменшується від 512 до 64. Для покращення стабільності навчання застосовано ReLU-активацію та механізм вирівнювання розподілу (batch normalization), а для уникнення перенавчання використано регуляризацию Dropout (від 0.3 до 0.5).

Попередня обробка аудіо виконується автоматично: нормалізується гучність сигналу, обрізається зайвий частотний діапазон та проводиться загальна калібровка відповідно до параметрів, використаних під час тренування моделі. Виділення спектральних ознак відбувається за допомогою Швидкого перетворення Фур'є (FFT) з вікном у 2048 відліків. Аналіз обмежується частотами від 80 до 4000 Гц, що охоплюють діапазон більшості музичних

інтервалів. Крім того, система обчислює додаткові дескриптори, зокрема спектральний центроїд, ширину спектру та параметри гармонік.

Для уточнення вхідних характеристик модуль використовує процедуру виділення пікових частот і формує на їх основі вектор ознак, що подається до нейронної мережі. Результатом роботи модуля є ймовірнісна оцінка належності сигналу до одного з визначених класів інтервалів. Користувачеві відображаються три найвірогідніші варіанти з відповідними значеннями довіри.

Алгоритм розпізнавання музичного інтервалу наведено на рис. 2.3.

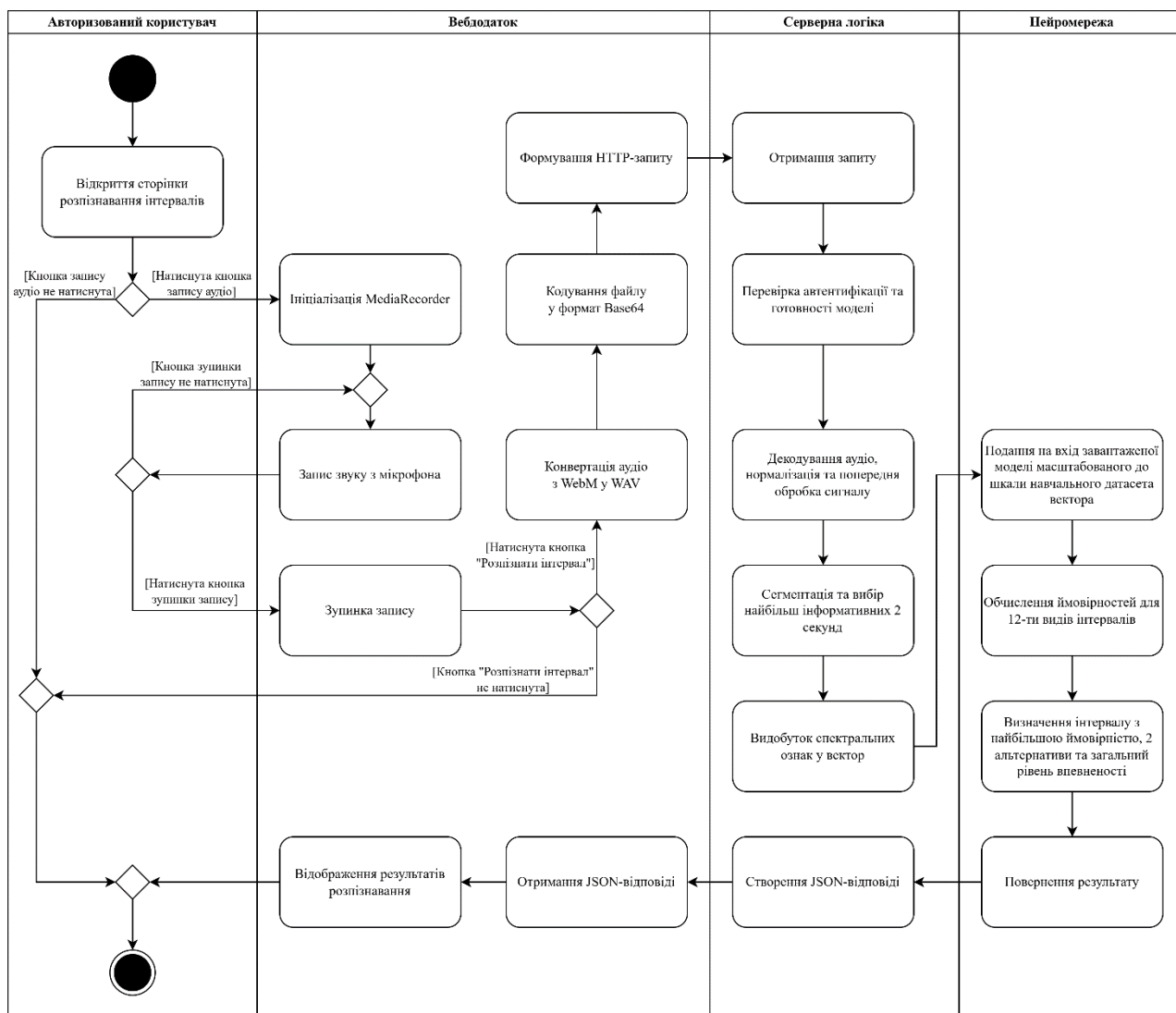


Рисунок 2.3 – Діаграма активності процесу розпізнавання інтервалу в аудіо

Початковий етап ініціюється на стороні клієнта: користувач натискає кнопку запису, після чого активується MediaRecorder API з частотою дискретизації 44100 Гц і монофонічним каналом.

Після завершення запису аудіодані конвертуються з формату WebM у WAV, а потім кодується в Base64. Разом із метаданими (формат, параметри запису) сформований запит надсилається на сервер через HTTP методом POST.

На сервері запит обробляється у класі IntervalRecognitionView. Система перевіряє доступність моделі, декодує аудіо та завантажує його з використанням librosa, автоматично нормалізуючи частоту дискретизації до 22050 Гц.

Модуль попередньої обробки застосовує калібрування аудіо до стандартів навчального датасету. Система видаляє DC зміщення, застосовує фільтрацію високих та низьких частот, та нормалізує RMS рівень до цільового значення. Tanh компресія забезпечує м'яке обмеження амплітуди для запобігання цифровим артефактам.

Сегментація аудіо відбувається шляхом вибору найінформативнішого фрагменту тривалістю 2 секунди. Алгоритм аналізує енергетичні характеристики різних частин запису та обирає сегмент з найвищою щільністю корисного сигналу.

FFTFeatureExtractor виконує швидке перетворення Фур'є з вікном Ханна для отримання частотного спектру. Система виділяє діапазон від 80 до 4000 Гц та нормалізує амплітудний спектр в логарифмічній шкалі. Додатково обчислюються спектральні характеристики, такі як центроїд, ширина смуги та rolloff частота.

Аналіз гармонічної структури включає пошук спектральних піків та обчислення їх співвідношень. Система ідентифікує основну частоту та її гармоніки, формуючи вектор ознак, що характеризує музичний інтервал.

IntervalClassifier завантажує попередньо навчену модель TensorFlow та scaler для нормалізації ознак. Вектор ознак масштабується відповідно до параметрів навчального датасету та подається на вхід нейронної мережі.

Нейронна мережа обчислює розподіл ймовірностей для всіх 12 класів інтервалів. Система визначає найбільш ймовірний інтервал та обчислює рівень

довіри до передбачення. Додатково формується список топ-3 альтернативних варіантів з відповідними ймовірностями.

Результат аналізу упаковується в JSON відповідь, що включає найкращий варіант розпізнавання, альтернативні варіанти, час обробки та метрики якості аналізу. Сервер повертає відповідь клієнту через HTTP.

На клієнті `RecognitionResults` компонент отримує результат та відображає його у зручному для користувача форматі. Система показує українські назви інтервалів, візуалізує рівень впевненості через прогрес-бари та надає додаткову інформацію про альтернативні варіанти.

Модуль тренування гармонічного слуху призначений для систематичного розвитку музичних здібностей користувачів через інтерактивні вправи з розпізнавання інтервалів. Основою модуля є система генерації синтетичних музичних інтервалів. Генератор звуку реалізований на основі адитивного синтезу з використанням гармонічних рядів. Кожна нота формується як сума синусоїдальних коливань основної частоти та її обертонів з відповідними амплітудними коефіцієнтами. ADSR огибаюча забезпечує природну динаміку атаки, спаду, підтримки та загасання звуку.

Система тренування підтримує два режими подачі інтервалів: цілісний (гармонічний), коли обидві ноти звучать одночасно, та поступовий (мелодичний), коли ноти відтворюються послідовно. Це дозволяє розвивати різні аспекти музичного слуху та адаптувати навчання до різних музичних стилів.

Модуль включає розширені можливості налаштування генерування. Користувач може обирати базову ноту для побудови інтервалів, що дозволяє сконцентруватися на певних тональностях.

Інтерактивна клавіатура фортепіано інтегрована в модуль тренування для надання додаткових можливостей дослідження музичних співвідношень. Клавіатура реалізована з використанням `Tone.js` та підтримує відтворення нот в реальному часі з налаштованими параметрами відображення клавіатури та тривалості звучання.

Процес генерації навчальних аудіоматеріалів продемонстровано на рис. 2.4.

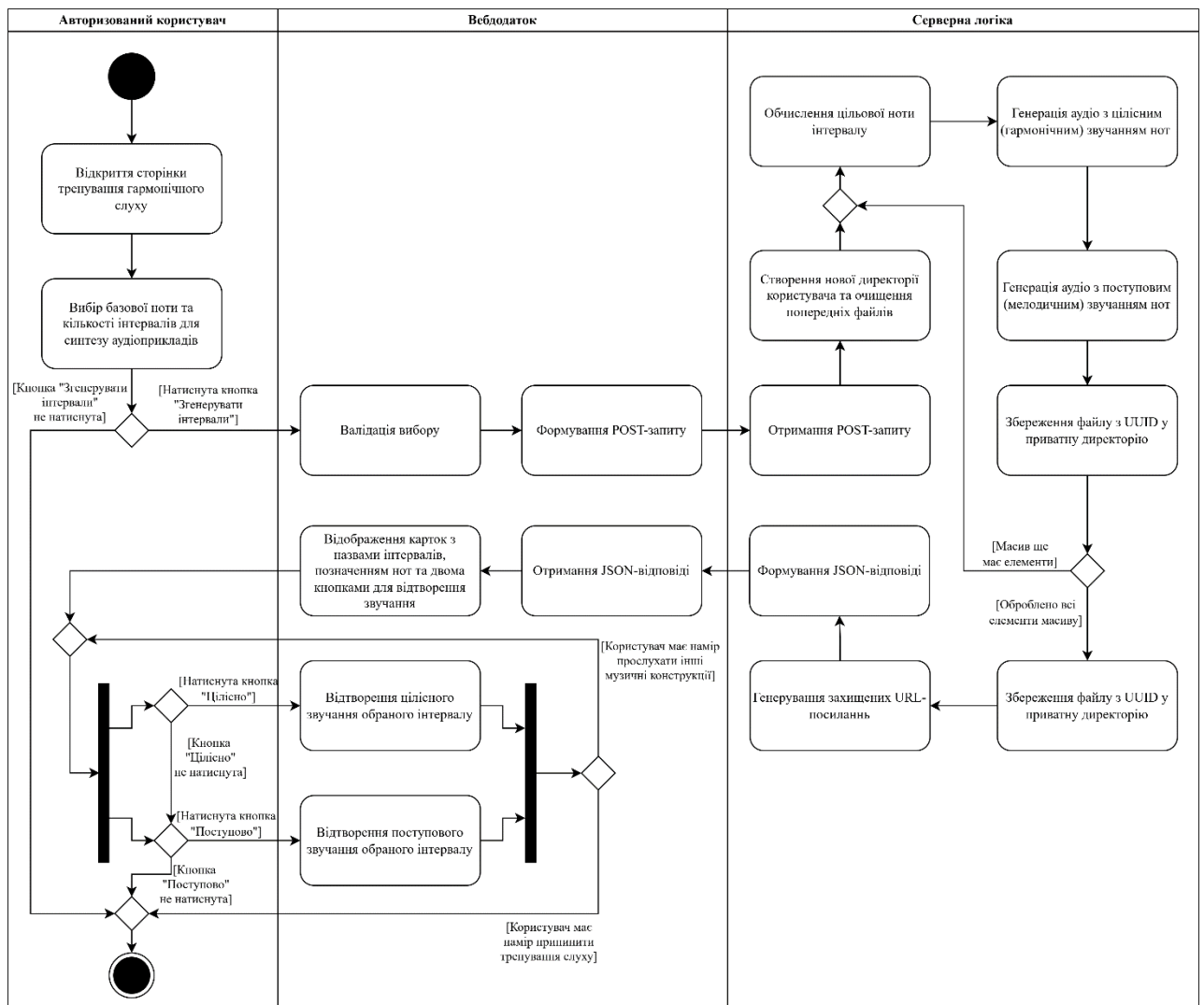


Рисунок 2.4 – Діаграма активності процесу генерації аудіоматеріалів та тренування гармонічного слуху

Алгоритм починається з вибору користувачем параметрів тренування через інтерфейс TrainingSettings компонента. Користувач обирає базову ноту з дванадцяти можливих хроматичних варіантів та вказує список інтервалів для генерації. Система валідує вибір, перевіряючи наявність принаймні одного інтервалу для створення навчального матеріалу.

При натисканні кнопки генерації клієнтський додаток формує POST запит до ендпоінту generation intervals з параметрами базової ноти та списком обраних

інтервалів. Запит передається через Axios HTTP клієнт з автентифікацією JWT токеном.

На сервері `GenerateIntervalsView` отримує запит та ініціалізує генератор аудіо з попередньо визначеними частотами нот та співвідношеннями інтервалів. Система створює унікальну робочу директорію для кожного користувача, очищуючи попередні файли для оптимізації використання дискового простору.

Для кожного обраного інтервалу алгоритм обчислює цільову ноту шляхом додавання відповідної кількості півтонів до базової ноти. Система використовує циклічну арифметику по модулю 12 для коректної обробки переходів через октаву.

Генератор створює два варіанти аудіо для кожного інтервалу: `wskscybyq` (гармонічний) та поступовий (мелодичний). Гармонічна версія формується шляхом одночасного синтезу базової та цільової нот з подальшим міксуванням сигналів. Мелодична версія створюється через конкатенацію послідовно згенерованих нот.

Синтез кожної ноти використовує адитивну модель з основною частотою та чотирма гармоніками з амплітудними коефіцієнтами 0.3, 0.2, 0.1 та 0.05 відповідно. ADSR огибаюча формує природну динаміку звуку з швидкою атакою протягом 0.1 секунди, експоненційним спадом до рівня 0.7 за 0.3 секунди та плавним загасанням протягом 0.5 секунди.

Згенеровані аудіосигнали нормалізуються до максимальної амплітуди 0.7 для запобігання цифровому кліпуванню та конвертуються в 16-бітний WAV формат з частотою дискретизації 44100 Гц. Система застосовує дітеринг для мінімізації квантизаційних шумів.

Кожен аудіофайл зберігається з унікальним UUID ідентифікатором в захищеній користувацькій директорії. Система генерує захищені URL посилання, що включають ідентифікатор користувача та забезпечують доступ тільки автентифікованим користувачам до їх власних файлів.

Відповідь сервера містить масив згенерованих інтервалів з метаданими, включаючи тип інтервалу, базову та цільову ноти, кількість півтонів та URL посилання на гармонічні та мелодичні версії. JSON структура оптимізована для ефективною серіалізації та передачі по мережі.

На клієнті GeneratedIntervals компонент отримує дані та відображає інтерактивні картки інтервалів. Кожна картка містить назву інтервалу українською мовою, музичні позначення нот та кнопки для відтворення гармонічної та мелодичної версій.

Система відтворення використовує useAudioPlayer хук для управління аудіо ресурсами та запобігання конфліктам при одночасному відтворенні кількох файлів. При натисканні кнопки відтворення клієнт завантажує аудіофайл через захищений ендпоінт та створює локальний blob URL для відтворення через HTML5 Audio API.

Візуальні індикатори відображають стан завантаження та відтворення кожного аудіофайлу. Система автоматично керує ресурсами, звільняючи пам'ять після завершення відтворення та запобігаючи накопиченню невикористаних audio об'єктів.

Модуль тестування гармонічного слуху реалізує комплексну систему оцінки музичних здібностей користувачів через структуровані тести з адаптивною складністю. Цей модуль забезпечує об'єктивну оцінку прогресу навчання та надає детальну аналітику результатів.

Система тестування підтримує тип тесту на розпізнавання звучання інтервалів.

Модуль створення тестів використовує алгоритми псевдовипадкової генерації для забезпечення різноманітності питань при збереженні контрольованого рівня складності. Система балансує розподіл питань між різними типами інтервалів відповідно до обраних параметрів складності.

Адаптивна система складності автоматично підбирає набір інтервалів для тестування відповідно до трьох попередньо налаштованих рівнів. Легкий рівень

включає основні консонантні інтервали, середній додає секундарні інтервали та сексти, а складний рівень охоплює всі дванадцять типів інтервалів включно з найскладнішими септимами та тритоном.

Алгоритм обчислення досвіду нараховує базові бали за правильні відповіді та додаткові бонуси за високу точність та стабільність результатів.

Модуль аналітики результатів генерує детальні звіти з візуалізацією прогресу по кожному типу інтервалу. Система відстежує динаміку покращення результатів та ідентифікує проблемні області, що потребують додаткової уваги.

Модуль підтримує збереження прогресу тестування та можливість повернення до незавершених сесій. Система автоматично зберігає стан тесту після кожної відповіді та дозволяє відновити роботу з того місця, де була припинена сесія.

Алгоритм створення тесту наведений на рис. 2.5.

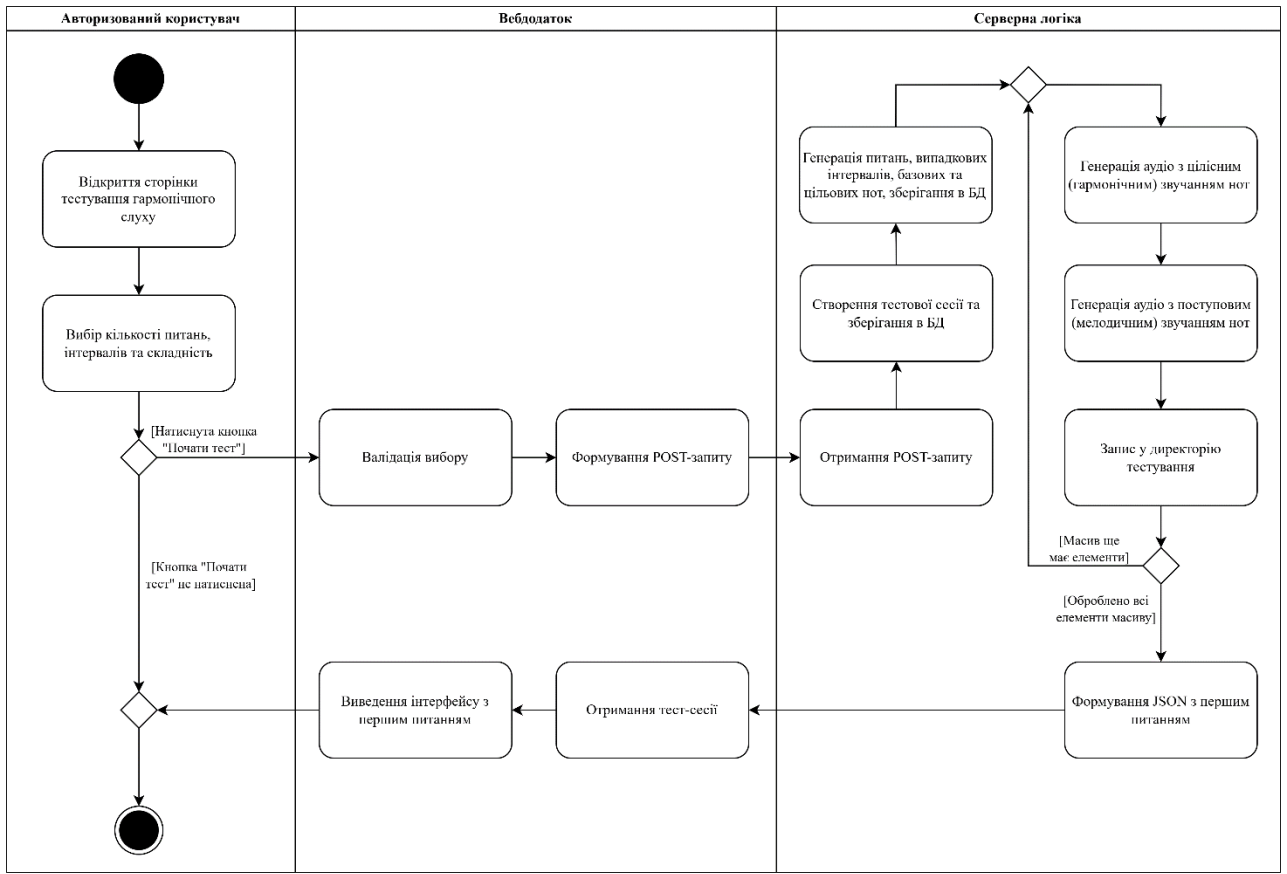


Рисунок 2.5 – Діаграма активності процесу створення тесту

Алгоритм створення персоналізованого тесту демонструє комплексну взаємодію між налаштуваннями користувача, серверною логікою генерації питань та системою збереження результатів. Процес починається з налаштування користувачем параметрів через TestSettings компонент.

Користувач встановлює кількість питань від 5 до 20, обирає рівень складності та опціонально налаштовує часові обмеження. Система валідує налаштування та перевіряє наявність принаймні одного інтервалу для тестів розпізнавання.

При натисканні кнопки початку тесту клієнт викликає createTestSession функцію з useTesting хука, яка формує POST запит до ендпоінту create session з повним набором параметрів тестування. Запит включає тип тесту, кількість питань, список обраних інтервалів та додаткові налаштування.

На сервері CreateTestSessionView отримує запит та створює новий об'єкт TestSession в базі даних з унікальним ідентифікатором та часовою міткою початку. Система зберігає всі параметри тестування для подальшого аналізу результатів.

Для тестів розпізнавання інтервалів генерується визначена кількість питань з випадковим вибором інтервалів з обраного набору. Алгоритм забезпечує рівномірний розподіл питань між різними типами інтервалів для об'єктивної оцінки знань.

Кожне питання включає випадково обрану базову ноту з діапазону C3-B5 та обчислену цільову ноту відповідно до типу інтервалу. Система зберігає метадані питання в базі даних та ініціює генерацію відповідних аудіофайлів.

Процес генерації аудіо для тестування використовує той самий алгоритм синтезу, що й модуль тренування, але створює файли в окремій директорії для ізоляції тестових матеріалів. Система генерує як гармонічні, так і мелодичні версії кожного інтервалу.

Створена тестова сесія серіалізується через TestSessionSerializer та повертається клієнту разом з першим питанням та всіма необхідними

метаданими. JSON відповідь оптимізована для швидкого завантаження та відображення питань.

На клієнті `TestSession` компонент отримує дані сесії та ініціалізує інтерфейс тестування з прогрес-баром, таймером та навігаційними елементами. Система встановлює початковий стан з першим питанням та готується до отримання відповідей користувача.

`IntervalRecognitionTest` компонент відображає поточне питання з аудіо контролами та варіантами відповідей. Система динамічно генерує дистрактори з інших обраних інтервалів та переміщує варіанти відповідей.

При виборі відповіді користувачем клієнт викликає `submitAnswer` функцію, яка відправляє POST запит до відповідного ендпоінту з ідентифікатором питання, обраною відповіддю та часом реакції. Система записує часову мітку відповіді для аналізу швидкості мислення.

Сервер обробляє відповідь через `SubmitAnswerView`, перевіряє правильність відповіді порівнянням з збереженим типом інтервалу та оновлює статистику сесії. Система обчислює поточну точність та перевіряє умови завершення тесту.

При завершенні всіх питань обчислюються фінальні результати, включаючи загальну точність, кількість правильних відповідей та нараховані бали досвіду.

Профіль користувача оновлюється новим досвідом через метод `add experience`, який автоматично перевіряє умови підвищення рівня відповідно до прогресивної формули нарахування. Система розраховує новий рівень на основі накопиченого досвіду з урахуванням зростаючих вимог для кожного наступного рівня.

Завершена сесія серіалізується з повною інформацією про результати, включаючи детальну статистику по кожному питанню та часові. Відповідь сервера містить всі дані, необхідні для відображення екрану результатів.

На клієнті система отримує результати та переходить до відображення TestResults компонента з комплексною аналітикою виконання. Інтерфейс показує загальну оцінку результату, детальну статистику точності, розподіл результатів по типах інтервалів та персоналізовані рекомендації для подальшого навчання.

Компонент результатів обчислює рівень виконання тесту відповідно до відсотка правильних відповідей та відображає відповідні візуальні індикатори успіху. Система надає детальний аналіз помилок з рекомендаціями щодо проблемних інтервалів.

Розглянуті модулі розпізнавання музичних інтервалів, тренування та тестування гармонічного слуху забезпечують комплексне функціонування системи навчання. Кожен з компонентів виконує чітко визначену роль у загальній архітектурі, формуючи послідовний цикл від формування навчального контенту та його інтерактивного використання до збору, аналізу та збереження результатів користувача.

### 2.3 Реалізація створення датасету

Дослідження розпізнавання музичних інтервалів потребує створення репрезентативного датасету, який містить достатню кількість прикладів кожного з 12 основних музичних інтервалів. У рамках цього дослідження було здійснено запис 24 аудіофайлів, структурованих за принципом покриття всіх можливих варіацій інтервалів з урахуванням динамічних особливостей виконання.

Структура початкових записів базується на музично-теоретичних принципах та включає 12 інтервалів: малу секунду (minor\_2nd), велику секунду (major\_2nd), малу терцію (minor\_3rd), велику терцію (major\_3rd), чисту кварту (perfect\_4th), тритон (tritone), чисту квінту (perfect\_5th), малу сексту (minor\_6th), велику сексту (major\_6th), малу септиму (minor\_7th), велику септиму (major\_7th) та чисту октаву (perfect\_8th). Кожен інтервал представлений у двох варіантах динамічного виконання: piano (м'яке натискання клавіш) та forte (сильне натискання), що забезпечує врахування тембральних відмінностей, які

виникають внаслідок різної сили атаки звуку. Структура збору аудіоданих зображена на рис. 2.6.

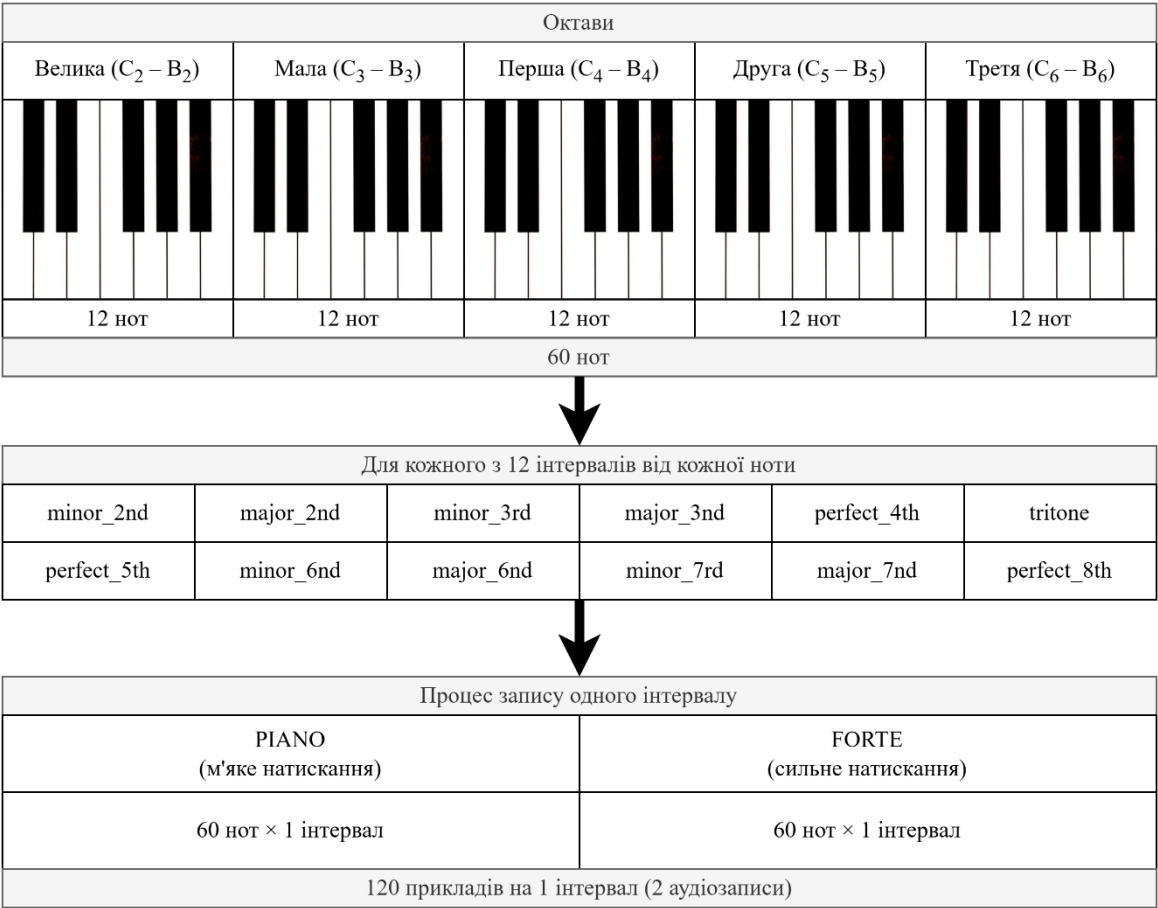


Рисунок 2.6 – Діаграма структури збору аудіоданих

Кожен з 24 аудіофайлів містить в собі по 60 відтворень інтервалів, побудованих від кожної ноти обраних октав, які звучать приблизно по 2 секунди з паузами між ними у 1–3 секунди. Загальна кількість початкових аудіоприкладів становить 1440 (24 файли × 60 прикладів), що створює основу для подальшого розширення датасету через методи аугментації.

Система обробки аудіоданих реалізована як набір взаємопов'язаних модулів, кожен з яких виконує специфічні функції у пайплайні підготовки датасету. Архітектурне рішення базується на принципах модульності та розділення відповідальності, що забезпечує можливість незалежного тестування та модифікації окремих компонентів системи.

На рис. 2.7 зображено діаграму класів модуля нормалізації аудіо і модуля обробки та сегментації.

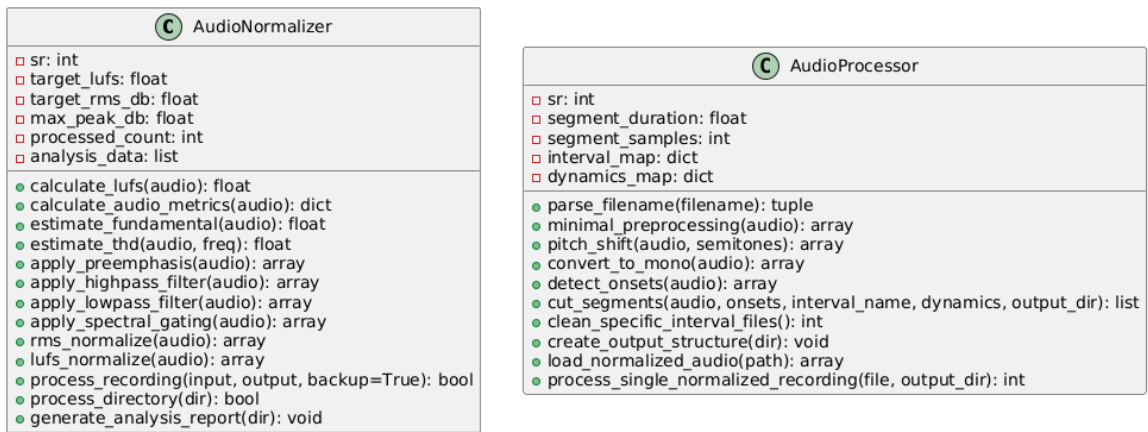


Рисунок 2.7 – Діаграма класів модуля нормалізації аудіо і модуля обробки та сегментації

Клас `AudioNormalizer` відповідає за стандартизацію акустичних параметрів вхідних записів (Додаток А). Основною метою цього компонента є приведення всіх аудіосигналів до єдиних стандартів гучності та динамічного діапазону, що критично важливо для забезпечення консистентності навчальних даних. Параметр `target_lufs` встановлений на рівні -23.0 дБ відповідно до рекомендацій EBU R 128 для цифрового аудіо, що забезпечує стандартизовану перцептивну гучність всіх записів.

Клас `AudioProcessor` реалізує функціональність сегментації та аугментації аудіоданих. Параметр `segment_duration` встановлений на 2.0 секунди, що представляє оптимальний компроміс між інформативністю сегмента (достатня тривалість для аналізу гармонічної структури інтервалу) та ефективністю обчислень (розумний розмір вхідних даних для нейронної мережі).

Процес нормалізації аудіосигналів базується на декількох фундаментальних математичних принципах обробки цифрових сигналів. Центральним поняттям є обчислення середньоквадратичного значення (RMS) сигналу, яке характеризує його енергетичну потужність (2.1) [23].

$$RMS = \sqrt{\frac{1}{N} \sum_{i=1}^N x[i]^2}, \quad (2.1)$$

де  $N$  — загальна кількість дискретних відліків сигналу;

$x[i]$  — амплітудне значення  $i$ -того відліку.

RMS-значення надає більш точну оцінку перцептивної гучності порівняно з піковими значеннями, оскільки враховує енергетичний вміст всього сигналу. Конвертація RMS у логарифмічну шкалу децибел здійснюється за формулою (2.2):

$$RMS_{dB} = 20 \cdot \log_{10}(RMS + \varepsilon), \quad (2.2)$$

де  $\varepsilon = 1 \times 10^{-10}$  — мала константа для запобігання логарифму від нуля.

Логарифмічна шкала відповідає нелінійному характеру сприйняття гучності людським слухом та спрощує подальші операції нормалізації. Реалізація обчислення аудіометрик у методі `calculate_audio_metrics()` включає розрахунок додаткових параметрів, що характеризують динамічні властивості сигналу:

```
def calculate_audio_metrics(self, audio):
    """Розрахунок всіх метрик аудіо"""
    rms = np.sqrt(np.mean(audio**2))
    peak = np.max(np.abs(audio))

    rms_db = 20 * np.log10(rms + 1e-10)
    peak_db = 20 * np.log10(peak + 1e-10)

    crest_factor = peak / (rms + 1e-10)
    dynamic_range = peak_db - rms_db

    fundamental_freq = self.estimate_fundamental(audio)
    thd = self.estimate_thd(audio, fundamental_freq) if fundamental_freq else 0

    lufs = self.calculate_lufs(audio)

    return {
        'rms': rms,
        'peak': peak,
        'rms_db': rms_db,
        'peak_db': peak_db,
```

```

        'crest_factor': crest_factor,
        'dynamic_range': dynamic_range,
        'thd': thd,
        'lufs': lufs,
        'duration': len(audio) / self.sr
    }

```

Параметр `crest_factor` характеризує співвідношення між піковими та середньоквадратичними значеннями сигналу. Високі значення ( $> 10$ ) можуть вказувати на наявність імпульсних компонентів або кліпування, тоді як низькі значення ( $< 3$ ) свідчать про надмірну компресію або обмеження динамічного діапазону.

Динамічний діапазон (`dynamic_range`) визначається як різниця між піковим та RMS рівнями у децибелах. Цей параметр критично важливий для оцінки якості запису, оскільки занадто малий динамічний діапазон ( $< 6$  дБ) може вказувати на проблеми з аудіообладнанням або неправильні налаштування запису.

Спектральна обробка аудіосигналів включає застосування цифрових фільтрів для видалення небажаних частотних компонентів та підвищення якості сигналу. Високочастотна фільтрація реалізується через фільтр Баттерворта четвертого порядку з частотою зрізу 80 Гц [24]. Математична модель цифрового фільтра Баттерворта базується на передавальній функції (2.3):

$$H(z) = \frac{\sum_{k=0}^M b[k] \cdot z^{-k}}{1 + \sum_{k=1}^N a[k] \cdot z^{-k}}, \quad (2.3)$$

де  $H(z)$  — передавальна функція в  $z$ -області;

$b[k]$  — коефіцієнти чисельника (feedforward);

$a[k]$  — коефіцієнти знаменника (feedback);

$M, N$  — порядок фільтра;

$z^{-k}$  — часові зсуви у  $z$ -просторі (затримки).

Для фільтра Баттерворта коефіцієнти  $a[k]$ ,  $b[k]$  отримуються з білатерального перетворення аналогового фільтра.

Реалізація високочастотного фільтра у методі `apply_highpass_filter()` використовує нормалізовану частоту зрізу (2.4) відносно частоти Найквіста, яка являє собою половину частоти дискретизації  $f_s$  і є максимально можливою частотою сигналу без спотворення (2.5):

$$f_{norm} = \frac{f_{cutoff}}{f_{Nyquist}} \quad (2.4)$$

$$f_{Nyquist} = \frac{f_s}{2} \quad (2.5)$$

```
def apply_highpass_filter(self, audio, cutoff=80, order=4):
    """High-pass фільтр"""
    nyquist = self.sr / 2
    normal_cutoff = cutoff / nyquist

    if normal_cutoff >= 1.0:
        return audio

    try:
        b, a = butter(order, normal_cutoff, btype='high', analog=False)
        return filtfilt(b, a, audio)
    except:
        return audio
```

Функція `filtfilt()` забезпечує нульовий фазовий зсув через застосування фільтра у прямому та зворотному напрямках. Це потрібно для збереження часових характеристик музичних інтервалів, оскільки фазові спотворення можуть вплинути на сприйняття гармонічних співвідношень.

Низькочастотна фільтрація з частотою зрізу 8000 Гц виконує функцію обмеження спектру до музично значущого діапазону. Більшість гармонік музичних інструментів сконцентровані нижче 8 кГц, тому обрізання вищих частот не впливає на музичну інформативність сигналу, але зменшує обчислювальне навантаження та мінімізує вплив високочастотних шумів.

Спектральний гейт реалізується через короткочасове перетворення Фур'є (STFT) з подальшим пороговим фільтруванням у частотно-часовій області:

```

def apply_spectral_gating(self, audio, gate_threshold=-70):
    """Спектральний гейт для зменшення шуму"""
    frame_length = 2048
    hop_length = 512

    stft = librosa.stft(audio, n_fft=frame_length, hop_length=hop_length)
    magnitude = np.abs(stft)
    phase = np.angle(stft)

    magnitude_db = 20 * np.log10(magnitude + 1e-10)

    gate_mask = magnitude_db > gate_threshold
    gated_magnitude = magnitude * gate_mask

    gated_stft = gated_magnitude * np.exp(1j * phase)
    gated_audio = librosa.istft(gated_stft, hop_length=hop_length)

    return gated_audio

```

Параметр `frame_length = 2048` визначає розмір вікна для STFT, що забезпечує компроміс між частотною (1024 частотних бінів) та часовою ( $\approx 46$  мс при частоті дискретизації 44100 Гц) роздільною здатністю. Параметр `hop_length = 512` встановлює крок перекриття вікон на рівні 75%, що забезпечує плавність реконструкції сигналу після обробки.

Поріг гейтування -70 дБ обрано як компроміс між ефективним видаленням шумової підлоги та збереженням слабких гармонічних компонентів музичних звуків. Компоненти спектру з амплітудою нижче цього порогу вважаються шумом та видаляються з сигналу.

Аугментація датасету здійснюється через транспозицію початкових записів на 1 півтон вище та нижче, що потроює кількість доступних навчальних прикладів. Математичною основою транспозиції є мультиплікативне зміщення всіх частотних компонентів сигналу на коефіцієнт, що відповідає музичному інтервалу півтону.

Коефіцієнт транспозиції для  $n$  півтонів обчислюється за формулою (2.6):

$$k = 2^{\frac{n}{12}}, \quad (2.6)$$

де  $n = \pm 1$  для підняття/опускання на один півтон.

Алгоритм транспозиції реалізований у методі `pitch_shift`, який дозволяє змінювати висоту звуку без зміни його тривалості:

```
def pitch_shift(self, audio, semitones):
    """Транспозиція"""
    if semitones == 0:
        return audio

    try:
        return librosa.effects.pitch_shift(
            audio, sr=self.sr, n_steps=semitones,
            bins_per_octave=12, res_type='scipy'
        )
    except Exception as e:
        print(f"Помилка транспозиції на {semitones} піввтонів: {e}")
        return audio
```

Параметр `bins_per_octave = 12` відповідає темперованому строю з 12 півтонами в октаві. Параметр `res_type = 'scipy'` визначає алгоритм ресемплінгу, що використовується для корекції тривалості після зміни висоти.

Клас модуля валідації зображений на рис. 2.8.

| AudioValidator                                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>sr: int = 44100</li> <li>stats: dict</li> </ul>                                                                                                                                                                                                                                                                                                                              |
| stats structure                                                                                                                                                                                                                                                                                                                                                                                                     |
| <ul style="list-style-type: none"> <li>total_files: int</li> <li>valid_files: int</li> <li>issues: defaultdict</li> <li>quality_stats: dict</li> <li>quiet_files: list</li> <li>problematic_files: list</li> </ul>                                                                                                                                                                                                  |
| Methods                                                                                                                                                                                                                                                                                                                                                                                                             |
| <ul style="list-style-type: none"> <li>_safe_json_convert(obj): any</li> <li>validate_file(path): tuple</li> <li>validate_dataset(path): bool</li> <li>_print_quality_statistics(): void</li> <li>_auto_fix_quiet_files(): bool</li> <li>_save_comprehensive_report(): void</li> <li>_calculate_quality_stats(): dict</li> <li>_generate_recommendations(): list</li> <li>_print_recommendations(): void</li> </ul> |

Рисунок 2.8 – Клас модуля валідації

Алгоритм виявлення початків звучання (onset detection) є ключовим компонентом системи автоматичної сегментації записів (Додаток Б). Він базується на аналізі спектральних змін у часі для ідентифікації моментів початку нових музичних подій.

Математичною основою виявлення початків є обчислення спектрального потоку (spectral flux) — міри зміни енергетичного розподілу у частотній області між сусідніми часовими кадрами (2.7):

$$SF(n) = \sum_{k=1}^K H(|X(n, k)| - |X(n - 1, k)|), \quad (2.7)$$

де  $SF(n)$  — значення спектрального потоку в кадрі  $n$ ;

$X(n, k)$  — спектральна амплітуда  $k$ -того частотного біна в  $n$ -тому кадрі (отримана після перетворення Фур'є);

$H(x) = \max(0, x)$  — функція півхвильової випрямленості (half-wave rectification) ;

$K$  — кількість частотних бінів у спектрі.

Реалізація виявлення початків у методі `detect_onsets()` використовує бібліотеку `librosa` з адаптивними параметрами:

```
def detect_onsets(self, audio, min_interval=1.2):
    """Виявлення початків"""
    try:
        onset_frames = librosa.onset.onset_detect(
            y=audio,
            sr=self.sr,
            hop_length=512,
            backtrack=True,
            units='time',
            pre_max=20,
            post_max=20,
            pre_avg=100,
            post_avg=100,
            delta=0.2,
            wait=10
```

)

Параметр `hop_length` визначає крок аналізу, що забезпечує достатню часову роздільність для виявлення швидких транзйєнтів. Параметр `backtrack = True` активує алгоритм зворотного відстеження для точного позиціонування початків на локальних мінімумах енергії.

Параметри `pre_max` та `post_max` визначають розмір локального максимуму для `peak picking`, тоді як `pre_avg` та `post_avg` встановлюють розмір вікна для обчислення локального середнього. Параметр `delta` визначає мінімальну висоту піка для його розпізнавання як початку.

Додаткова фільтрація початків за мінімальним інтервалом запобігає помилковому виявленню декількох початків для одного інтервалу:

```
filtered_onsets = []
for onset in onset_frames:
    if not filtered_onsets or onset - filtered_onsets[-1] >= min_interval:
        filtered_onsets.append(onset)

return np.array(filtered_onsets)
```

Процес сегментації перетворює континуальні аудіозаписи на дискретні фрагменти фіксованої тривалості, придатні для навчання нейронної мережі. Кожен сегмент має тривалість 2.0 секунди, що забезпечує достатню часову базу для аналізу гармонічної структури музичних інтервалів.

Алгоритм сегментації у методі `cut_segments()` здійснює вирізання фрагментів з урахуванням виявлених початків:

```
def cut_segments(self, audio, onsets, interval_name, dynamics, output_dir):
    """Нарізка аудіо на сегменти"""
    interval_dir = os.path.join(output_dir, interval_name)
    dynamics_dir = os.path.join(interval_dir, dynamics)
    Path(dynamics_dir).mkdir(parents=True, exist_ok=True)

    segments = []
    for i, onset in enumerate(onsets):
        start_sample = int(onset * self.sr)
```

```

end_sample = start_sample + self.segment_samples

if end_sample <= len(audio):
    segment = audio[start_sample:end_sample]

    filename = f"{interval_name}_{i:03d}_{dynamics}.wav"
    filepath = os.path.join(dynamics_dir, filename)

    try:
        sf.write(filepath, segment, self.sr)

        if os.path.exists(filepath) and os.path.getsize(filepath) > 0:
            segments.append((segment, filename))
            if i % 10 == 0:
                print(f"Збережено: {interval_name}/{dynamics}/{filename}")
            else:
                print(f"Помилка збереження: {filename}")

        except Exception as e:
            print(f"Помилка при збереженні {filename}: {e}")
            continue

    return segments

```

На рис. 2.9 зображено структуру класу модуля генерації FFT-датасету.

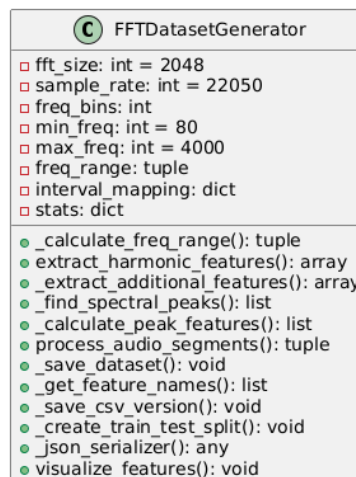


Рисунок 2.9 – Клас модуля генерації FFT-датасету

Основне перетворення у методі `extract_harmonic_features()` реалізує спектральний аналіз з використанням вікна Ханна для мінімізації спектральної витоку:

```
def extract_harmonic_features(self, audio):
    """Виділення гармонічних ознак з аудіо"""
    if len(audio) < self.fft_size:
        audio = np.pad(audio, (0, self.fft_size - len(audio)))
    elif len(audio) > self.fft_size:
        start = (len(audio) - self.fft_size) // 2
        audio = audio[start:start + self.fft_size]

    windowed_audio = audio * np.hanning(len(audio))

    fft = np.fft.fft(windowed_audio, n=self.fft_size)
    magnitude = np.abs(fft[:self.freq_bins])
    min_idx, max_idx = self.freq_range
    magnitude = magnitude[min_idx:max_idx]

    magnitude_db = 20 * np.log10(magnitude + 1e-10)
    magnitude_normalized = (magnitude_db - np.min(magnitude_db)) /
    (np.max(magnitude_db) - np.min(magnitude_db) + 1e-10)
    additional_features = self._extract_additional_features(magnitude, audio)
    all_features = np.concatenate([magnitude_normalized, additional_features])

    return all_features
```

Система додаткових ознак включає спектральні дескриптори, що характеризують тембральні властивості музичних звуків. Ці ознаки доповнюють інформацію з амплітудного спектру та підвищують дискримінаційну здатність моделі.

Спектральний центроїд визначає «центр ваги» спектрального розподілу енергії (2.8) [25]:

$$SC = \frac{\sum_{k=1}^K f_k \cdot |X_k|}{\sum_{k=1}^K |X_k|}, \quad (2.8)$$

де  $f_k$  — частота k-того спектрального біна;

$|X_k|$  — амплітуда k-того біна;

K — кількість бінів (частотних відліків).

Реалізація:

```
freqs = np.linspace(self.min_freq, self.max_freq, len(magnitude))
```

```
spectral_centroid = np.sum(freqs * magnitude) / (np.sum(magnitude) + 1e-10)
```

```
features.append(spectral_centroid / self.max_freq)
```

Нормалізація на self.max\_freq приводить значення до діапазону [0,1] та забезпечує стабільність навчання нейронної мережі.

Спектральна ширина (bandwidth) характеризує розосередженість енергії навколо спектрального центроїда (2.9) [26]:

$$SB = \sqrt{\frac{\sum_{k=1}^K (f_k - SC)^2 \cdot |X_k|}{\sum_{k=1}^K |X_k|}}, \quad (2.9)$$

де SC — спектральний центроїд.

Реалізація:

```
spectral_bandwidth = np.sqrt(np.sum((((freqs - spectral_centroid) ** 2) * magnitude) /  
(np.sum(magnitude) + 1e-10))
```

```
features.append(spectral_bandwidth / self.max_freq)
```

Спектральний rolloff визначає частоту, нижче якої зосереджено 85% спектральної енергії:

```
cumsum = np.cumsum(magnitude)
```

```
rolloff_idx = np.where(cumsum >= 0.85 * cumsum[-1])[0]
```

```
if len(rolloff_idx) > 0:
```

```
    rolloff_freq = freqs[rolloff_idx[0]]
```

```
    features.append(rolloff_freq / self.max_freq)
```

```
else:
```

```
    features.append(1.0)
```

Поріг 85% обраний як компроміс між інформативністю (врахування основної частини енергії) та стійкістю до шумів (ігнорування височастотного залишку).

Zero-crossing rate (ZCR) вимірює частоту переходів сигналу через нульове значення (2.10) [27]:

$$ZCR = \frac{1}{2N} \sum_{n=1}^{N-1} |\text{sign}(x_n) - \text{sign}(x_{n-1})|, \quad (2.10)$$

де  $x_n$  — значення аудіосигналу в n-й точці;

$\text{sign}(x)$  — знак функції (–1, 0, 1);

N — кількість дискретних точок сигналу.

Реалізація:

```
zcr = np.mean(np.abs(np.diff(np.sign(audio))))
features.append(zcr)
```

Виявлення та аналіз спектральних піків потрібен для розпізнавання музичних інтервалів, оскільки кожен інтервал характеризується специфічним набором гармонічних співвідношень. Система аналізу піків виділяє домінуючі частотні компоненти та обчислює їх відносні характеристики [28].

Алгоритм виявлення піків у методі `find_spectral_peaks()` використовує адаптивний поріг на основі `prominence`:

```
def _find_spectral_peaks(self, magnitude, prominence=0.1):
    """Знаходження піків в спектрі"""
    try:
        from scipy.signal import find_peaks
        peaks, _ = find_peaks(magnitude, prominence=prominence * np.max(magnitude))
        return peaks[:10]
    except ImportError:
        peaks = []
        for i in range(1, len(magnitude) - 1):
            if magnitude[i] > magnitude[i-1] and magnitude[i] > magnitude[i+1]:
                if magnitude[i] > prominence * np.max(magnitude):
                    peaks.append(i)
        if len(peaks) >= 10:
            break
        return peaks
```

Параметр `prominence = 0.1` встановлює мінімальну висоту піка відносно максимального значення спектру. Це забезпечує виявлення тільки значущих гармонік та відфільтровує спектральний шум. Обмеження на 10 піків запобігає надмірному ускладненню вектора ознак при збереженні інформації про основні гармоніки.

Аналіз гармонічних співвідношень реалізований у методі `calculate_peak_features()`:

```
def _calculate_peak_features(self, peak_indices, freqs, magnitude):
    """Розрахунок ознак піків"""
    features = []

    if len(peak_indices) == 0:
        return [0.0] * 15

    fundamental_freq = freqs[peak_indices[0]] if len(peak_indices) > 0 else 1.0
    features.append(fundamental_freq / self.max_freq)

    for i in range(1, min(5, len(peak_indices))):
        ratio = freqs[peak_indices[i]] / fundamental_freq if fundamental_freq > 0 else 0.0
        features.append(ratio)

    while len(features) < 6:
        features.append(0.0)

    max_magnitude = np.max(magnitude)
    for i in range(min(5, len(peak_indices))):
        amplitude = magnitude[peak_indices[i]] / max_magnitude
        features.append(amplitude)

    while len(features) < 11:
        features.append(0.0)

    for i in range(1, min(5, len(peak_indices))):
```

```
interval = (freqs[peak_indices[i]] - freqs[peak_indices[i-1]]) / self.max_freq
features.append(interval)
```

```
while len(features) < 15:
    features.append(0.0)
```

```
return features[:15]
```

Сформований набір ознак поєднує спектральну, гармонічну та тембральну інформацію, необхідну для надійної ідентифікації музичних інтервалів, що забезпечує якісну основу для навчання моделі розпізнавання.

## 2.4 Реалізація створення моделі розпізнавання

Нейронна мережа для розпізнавання музичних інтервалів спроектована як глибока багатошарова архітектура з урахуванням специфіки спектральних ознак та вимог до точності класифікації.

Структура класу моделі класифікації зображена на рис. 2.10

| FFTIntervalClassifier                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>fft_size: int = 2048</li> <li>sample_rate: int = 22050</li> <li>freq_bins: int</li> <li>model: keras.Sequential</li> <li>scaler: StandardScaler</li> <li>class_names: list</li> <li>class_to_index: dict</li> <li>num_classes: int</li> <li>min_freq: int = 80</li> <li>max_freq: int = 4000</li> <li>freq_range: tuple</li> </ul>                                                                                                               |
| <ul style="list-style-type: none"> <li>_calculate_freq_range(): tuple</li> <li>extract_harmonic_features(): array</li> <li>extract_additional_features(): array</li> <li>_find_spectral_peaks(): list</li> <li>_calculate_peak_ratios(): list</li> <li>prepare_dataset_from_pickle(): tuple</li> <li>create_model(input_dim): model</li> <li>train(): history</li> <li>evaluate(): float</li> <li>predict_interval_from_audio(): dict</li> <li>save_model_and_scaler(): void</li> </ul> |

Рисунок 2.10 – Клас моделі класифікації

Архітектура мережі включає п'ять прихованих шарів з поступовим зменшенням розмірності (рис. 2.11). Ця конфігурація забезпечує ієрархічне виділення ознак від низькорівневих спектральних деталей до високорівневих музичних концепцій (Додаток В).

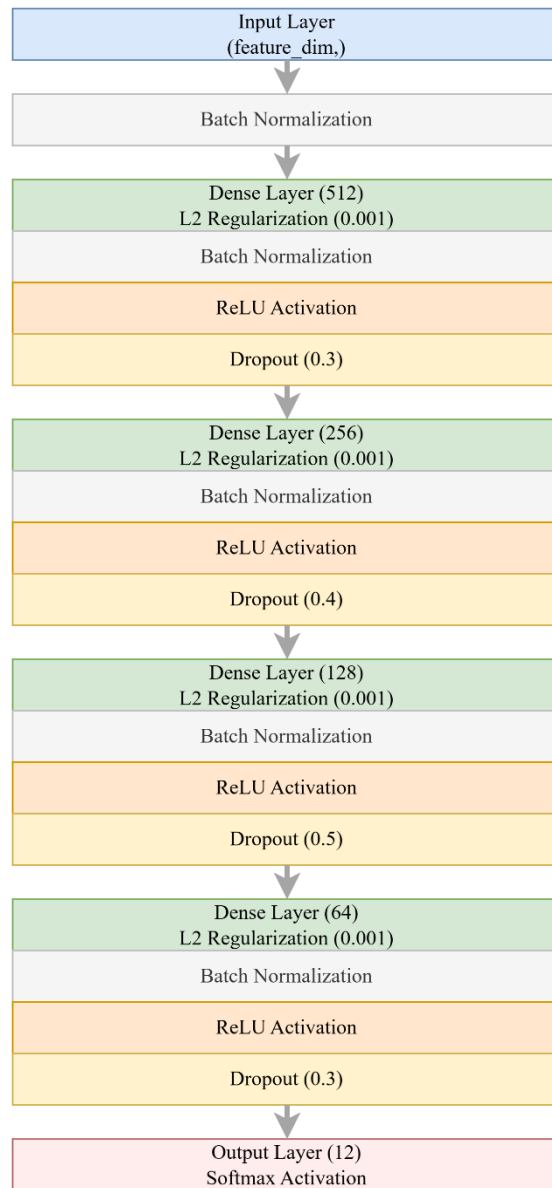


Рисунок 2.11 – Архітектура нейронної мережі

Реалізація моделі у методі `create_model()`:

```

def create_model(self, input_dim):
    """Створення моделі"""
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),

        layers.BatchNormalization(),

        layers.Dense(512, kernel_regularizer=keras.regularizers.l2(0.001)),
        layers.BatchNormalization(),
        layers.Activation('relu'),

```

```

layers.Dropout(0.3),

layers.Dense(256, kernel_regularizer=keras.regularizers.l2(0.001)),
layers.BatchNormalization(),
layers.Activation('relu'),
layers.Dropout(0.4),

layers.Dense(128, kernel_regularizer=keras.regularizers.l2(0.001)),
layers.BatchNormalization(),
layers.Activation('relu'),
layers.Dropout(0.5),

layers.Dense(64, kernel_regularizer=keras.regularizers.l2(0.001)),
layers.BatchNormalization(),
layers.Activation('relu'),
layers.Dropout(0.3),

layers.Dense(self.num_classes, activation='softmax')
)

```

Batch Normalization застосовується перед кожним прихованим шаром для стабілізації навчання та прискорення конвергенції. L2 регуляризація з коефіцієнтом  $\lambda = 0.001$  додається до ваг кожного Dense шару для запобігання перенавчанню. Dropout регуляризація з градуйованими забезпечує адаптивне запобігання перенавчанню. Вищі коефіцієнти у середніх шарах відповідають більшій схильності до перенавчання у шарах з максимальною кількістю параметрів.

ReLU активація обрана для прихованих шарів завдяки її обчислювальній ефективності та здатності пом'якшувати проблему згасаючих градієнтів [29]. Softmax активація у вихідному шарі забезпечує перетворення логітів (вихідних значень останнього шару нейронної мережі) у розподіл ймовірностей.

Процес навчання нейронної мережі використовує оптимізатор Adam з адаптивною швидкістю навчання та функції зворотного виклику (callbacks) для моніторингу якості тренування і раннього завершення навчання [30].

Конфігурація оптимізатора:

```
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy', keras.metrics.SparseCategoricalAccuracy(name='accuracy')]
)
```

Для оптимізації часу відгуку системи розпізнавання було реалізовано асинхронну ініціалізацію моделі при запуску сервера. Ця архітектурна особливість дозволяє уникнути затримок при першому запиті до API та забезпечує постійну готовність системи до обробки аудіоданих.

Структура класу системи інференсу зображена на рис. 2.12.

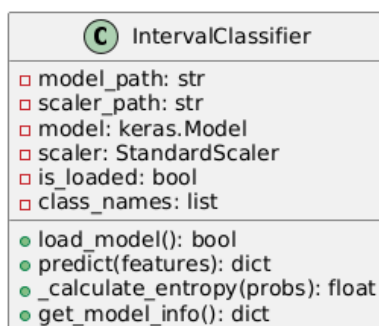


Рисунок 2.12 – Клас системи інференсу

Асинхронна ініціалізація реалізована через механізм Django app configuration:

```
def ready(self):
    try:
        from api.recognition_views import initialize_model
        logger.info("Запуск ініціалізації моделі...")

    import threading

    def init_models_thread():
```

```

try:
    success = initialize_model()
    if success:
        logger.info("Модель успішно ініціалізована при запуску сервера")
    else:
        logger.warning("Помилка ініціалізації моделі при запуску")
except Exception as e:
    logger.error(f"Критична помилка ініціалізації моделі: {e}")

```

```

timer = threading.Timer(2.0, init_models_thread)
timer.start()

```

```

except Exception as e:
    logger.error(f"Помилка при ініціалізації моделі в apps.py: {e}")

```

Завантаження натренованої моделі у класі IntervalClassifier включає завантаження як самої нейронної мережі, так і scaler'a для нормалізації ознак:

```

def load_model(self) -> bool:
    """Завантаження моделі"""
    try:
        if os.path.exists(self.scaler_path):
            self.scaler = joblib.load(self.scaler_path)
            logger.info("Scaler завантажено успішно")
        else:
            logger.error(f"Scaler не знайдено: {self.scaler_path}")
            return False

        if os.path.exists(self.model_path):
            try:
                import tensorflow as tf

                tf.get_logger().setLevel('ERROR')

                self.model = tf.keras.models.load_model(self.model_path, compile=False)

```

```

self.model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

logger.info("Модель завантажена")
logger.info(f"Вхідна форма: {self.model.input_shape}")
logger.info(f"Вихідна форма: {self.model.output_shape}")

self.is_loaded = True
return True

```

Запропонована архітектура моделі класифікації та механізм її ініціалізації створюють технічне підґрунтя для надійної та ефективної роботи системи розпізнавання інтервалів. Поєднання глибокої структури мережі, оптимізації параметрів навчання та попереднього завантаження компонентів забезпечує не лише стабільну точність, але й готовність системи до інтерактивної обробки аудіоданих без затримок у роботі.

### Висновки до другого розділу

У другому розділі було проаналізовано основні технічні рішення, що лягли в основу реалізації системи для розвитку гармонічного слуху. Класична клієнт-серверна архітектура з REST API виявилася оптимальним варіантом з огляду на обсяг задач, специфіку обробки аудіоданих і необхідність подальшого розширення. Вибір Django як серверного фреймворка забезпечив тісну інтеграцію з Python-екосистемою та дозволив уникнути зайвих накладних витрат на взаємодію з модулями машинного навчання. React.js, у свою чергу, забезпечив гнучкий інтерфейс із швидким реагуванням, що критично важливо в роботі з аудіоконтентом у режимі реального часу.

Окрему увагу приділено модулю розпізнавання музичних інтервалів. Детально описаний алгоритм демонструє глибоку інтеграцію цифрової обробки

сигналів і штучного інтелекту, що дозволяє забезпечити як точність результату, так і швидкість обробки. Впровадження механізмів аналізу якості вхідного сигналу, нормалізації, сегментації та виділення спектральних ознак дозволило адаптувати модель до умов реального використання без втрати продуктивності.

Обрана структура не обмежує подальший розвиток системи: у майбутньому можливий перехід до серверної інфраструктури з розподіленими сервісами, інтеграцією аналітики та підтримкою багатокористувацького режиму.

Реалізовані технічні рішення виглядають збалансованими з огляду на поставлену задачу, враховують реальні обмеження середовища та закладають технічну основу для подальшого розвитку функціоналу.

## РОЗДІЛ 3. ІНТЕРФЕЙС ТА ПОРЯДОК РОБОТИ З СИСТЕМОЮ ДОДАТКУ

### 3.1 Структура інтерфейсу та порядок взаємодії з системою

З огляду на специфіку обробки аудіоданих і потреби користувача, інтерфейс системи було спроектовано з акцентом на інтерактивність, простоту навігації та візуальну інформативність.

Сторінка реєстрації (рис. 3.1) передбачає створення нового облікового запису й містить два основні поля — для введення логіна та пароля. Під полем пароля розміщено коротке пояснення щодо мінімальної довжини (не менше п'яти символів), що допомагає орієнтуватися у вимогах безпеки вже на етапі введення даних. Кнопка «Зареєструватися» має візуальне виділення й доповнена іконкою користувача з плюсом, що підсилює асоціацію з дією створення нового облікового запису. Під формою є посилання для переходу до сторінки входу.

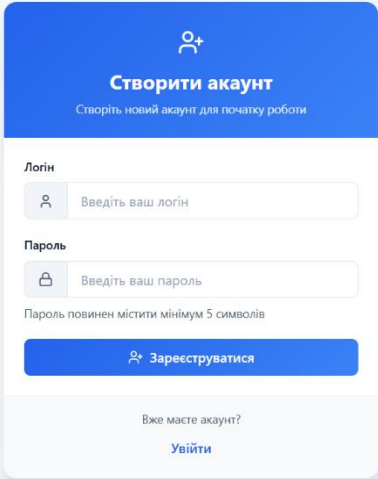
Скріншот мобільного інтерфейсу сторінки реєстрації. Вгорі синій заголовок з іконкою людини та плюсом і текстом «Створити акаунт» та «Створіть новий акаунт для початку роботи». Нижче два поля введення: «Логін» з іконкою людини та «Пароль» з іконкою замка. Під полем пароля текст: «Пароль повинен містити мінімум 5 символів». Внизу синя кнопка «Зареєструватися» з іконкою людини та плюсом. На самому дні посилання «Увійти».

Рисунок 3.1 – Сторінка реєстрації

Сторінка авторизації (рис. 3.2) оформлена у подібному стилі, однак супроводжується заголовком «Вітаємо знову», що створює відчуття персоналізованого повернення. Користувачеві пропонується ввести логін і пароль, після чого натиснути кнопку «Увійти», позначену іконкою стрілки входу. Як і у формі реєстрації, нижче розташоване альтернативне посилання — цього разу для користувачів, які ще не мають акаунта.

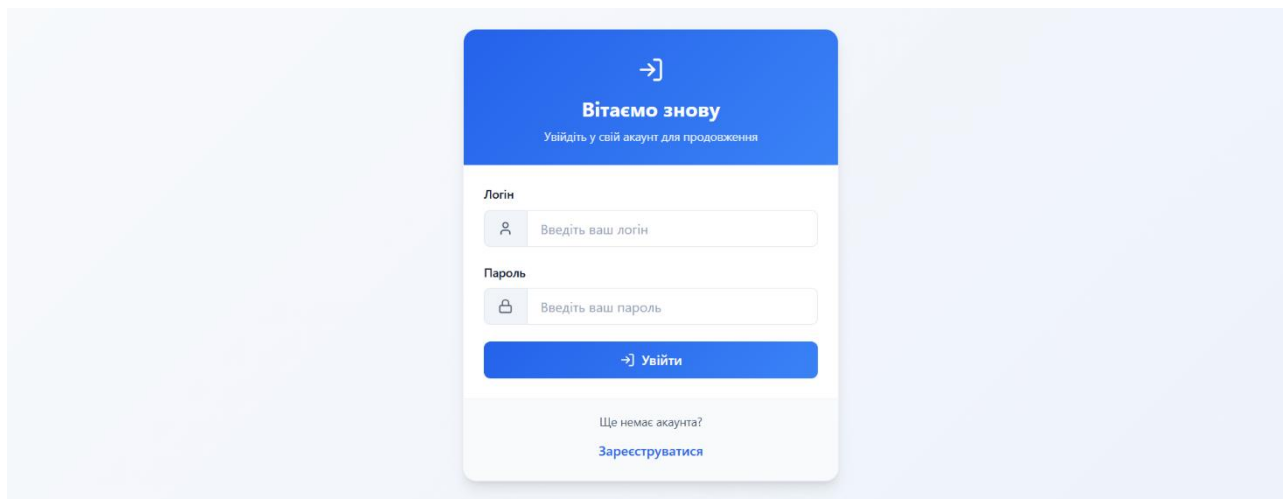


Рисунок 3.2 – Сторінка авторизації

На головній сторінці (рис. 3.1) користувач отримує коротке ознайомлення з можливостями платформи: розпізнавання 12 типів інтервалів, висока точність моделі (92%) та активна база користувачів. Основна навігація винесена в хедер і дозволяє швидко перейти до ключових модулів — Розпізнавання, Тренування та Тестування, кожен з яких має власний функціонал і візуальне представлення.

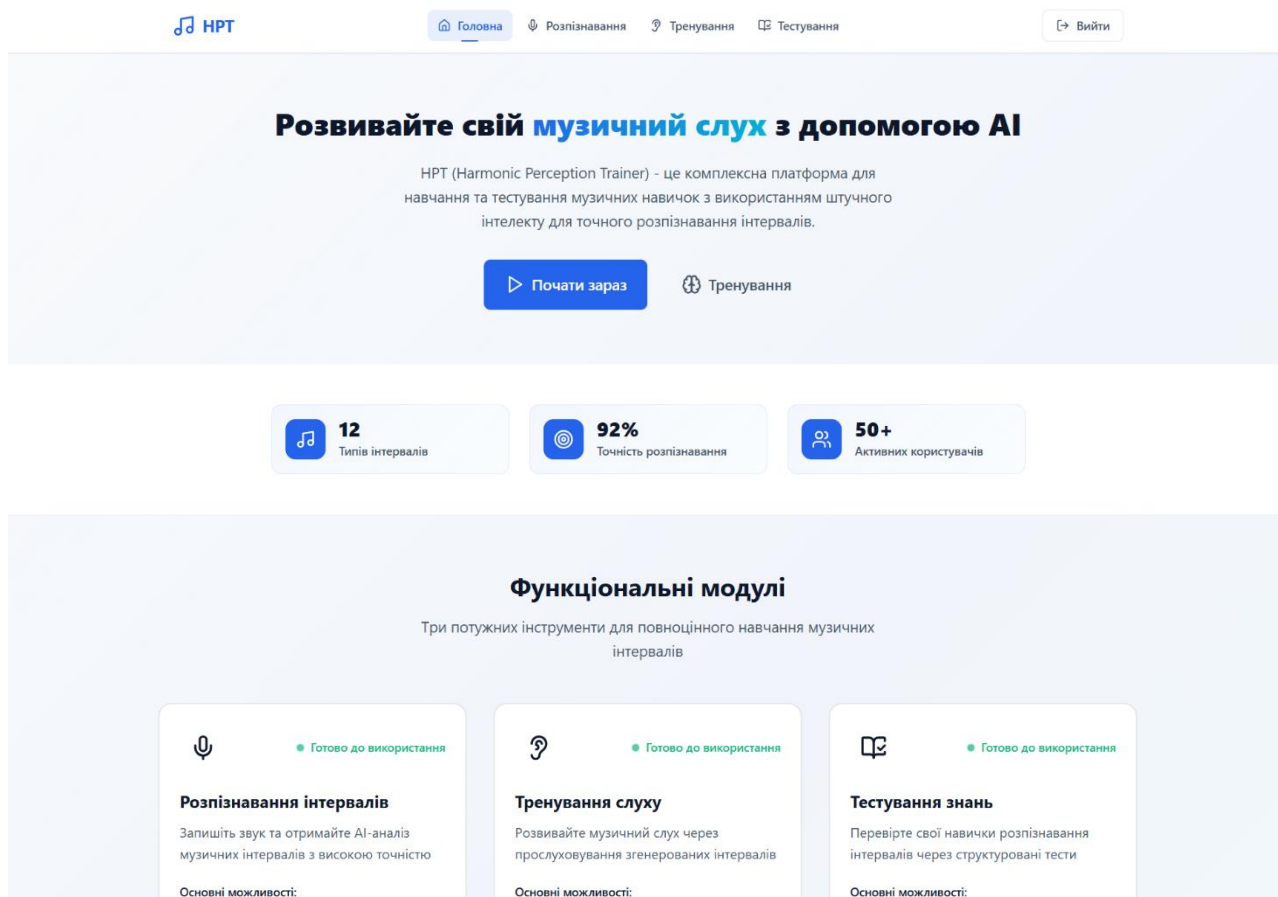


Рисунок 3.3 – Головна сторінка

Після успішної автентифікації користувач може перейти до вкладки «Розпізнавання», де на нього очікує центральний елемент інтерфейсу — модуль запису звуку. Інтерфейс інтуїтивно зрозумілий: єдина велика кнопка із мікрофоном пропонує натиснути «Почати запис» для ініціації процесу (рис. 3.4).

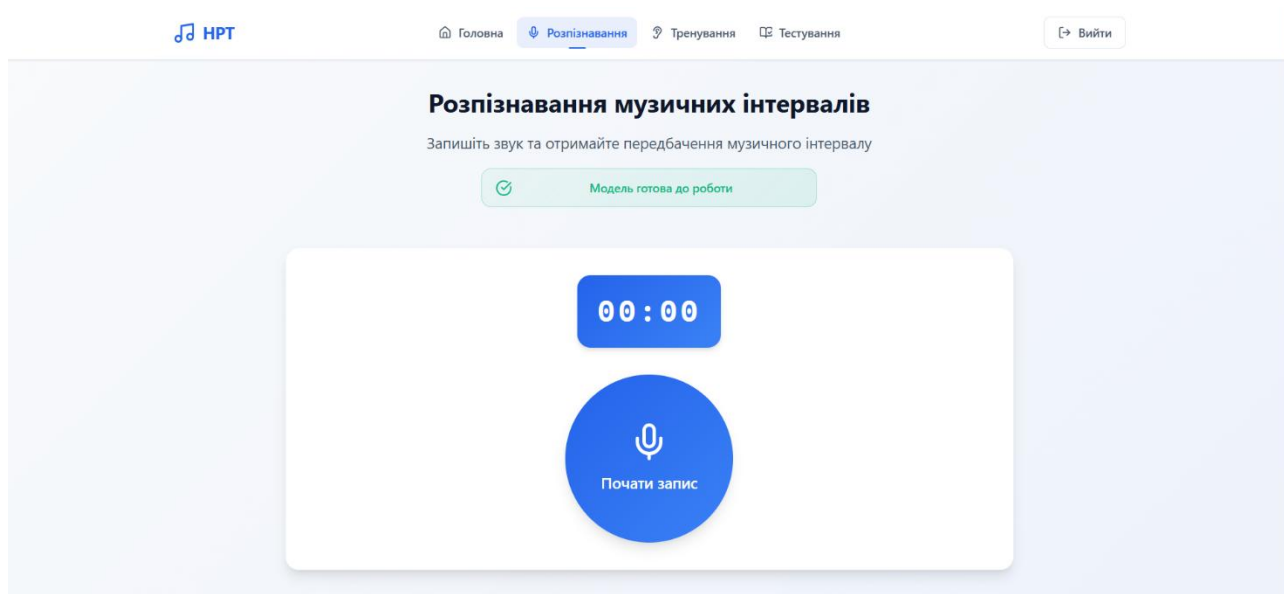


Рисунок 3.4 – Інтерфейс модуля розпізнавання музичних інтервалів до початку запису

Після активації запису інтерфейс динамічно оновлюється: візуалізується таймер і кнопка набуває червоного кольору з написом «Зупинити запис» (рис. 3.5). Користувач має можливість контролювати тривалість запису відповідно до інструкцій або інтуїтивно, орієнтуючись на тривалість інтервалу, але достатньо буде записати не більше 2 – 3 секунд.

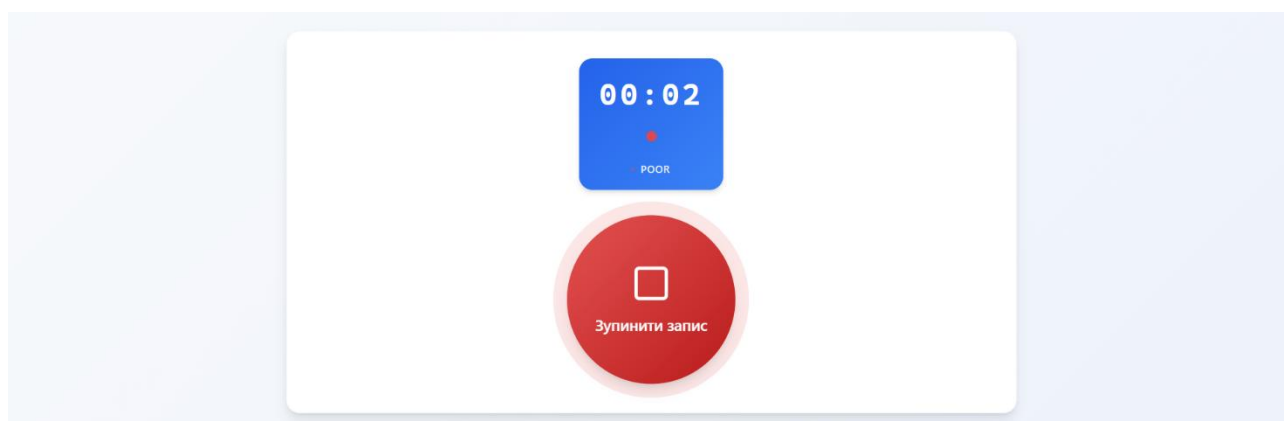


Рисунок 3.5 – Процес активного запису звуку з індикацією таймера та можливістю зупинки

Після завершення запису з'являється оновлений блок керування: кнопка змінюється на зелений індикатор з написом «Новий запис», а нижче стають доступними дії «Прослухати», «Розпізнати інтервал» та «Скинути» (рис. 3.6). Користувач може одразу перевірити якість запису, прослухавши його (рис. 3.7), або перейти до етапу розпізнавання.

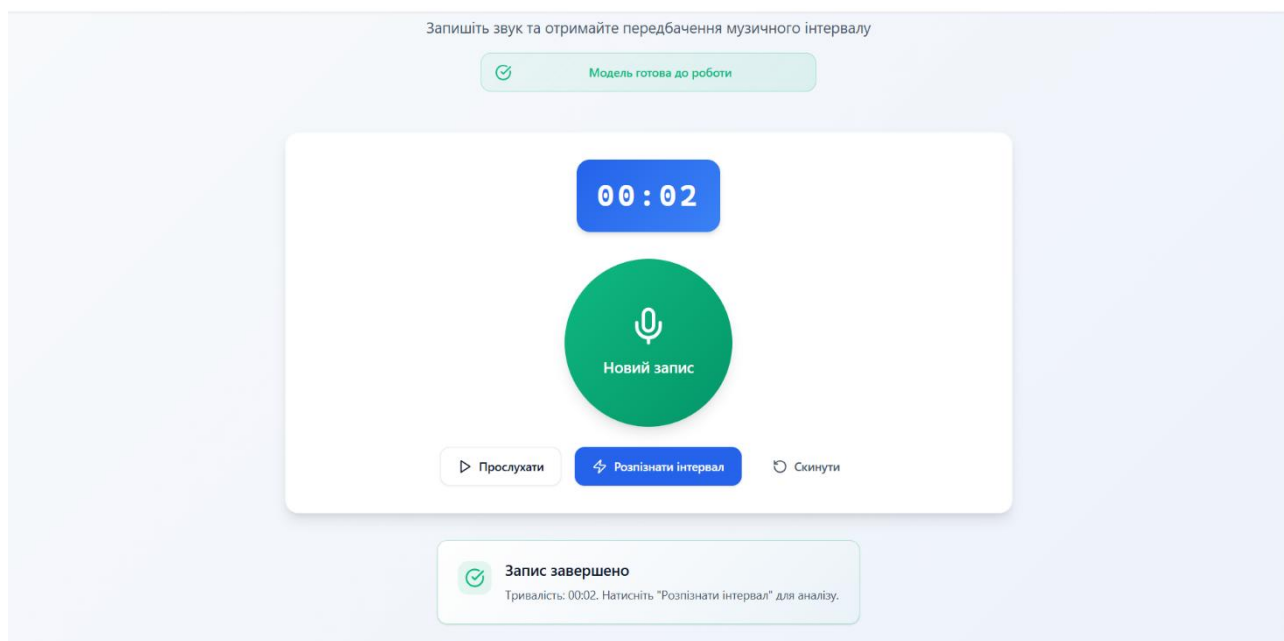


Рисунок 3.6 – Стан після завершення запису: доступні дії прослуховування, розпізнавання або повторного запису

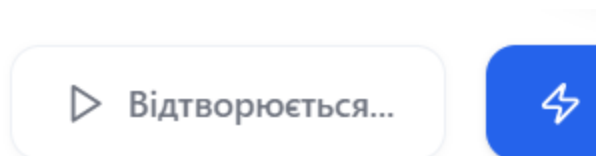


Рисунок 3.7 – Відтворення записаного аудіо для перевірки якості

Після натискання кнопки «Розпізнати інтервал» відбувається передача аудіоданих на сервер, де здійснюється аналіз із використанням нейронної мережі. У результаті користувач отримує детальну відповідь у форматі, що включає основний розпізнаний інтервал (наприклад, «Чиста квінта» або «Велика терція») (рис. 3.8 – 3.9), рівень впевненості моделі у відсотках, два альтернативні варіанти із відповідними ймовірностями та додаткові технічні характеристики (тривалість запису та час обробки).

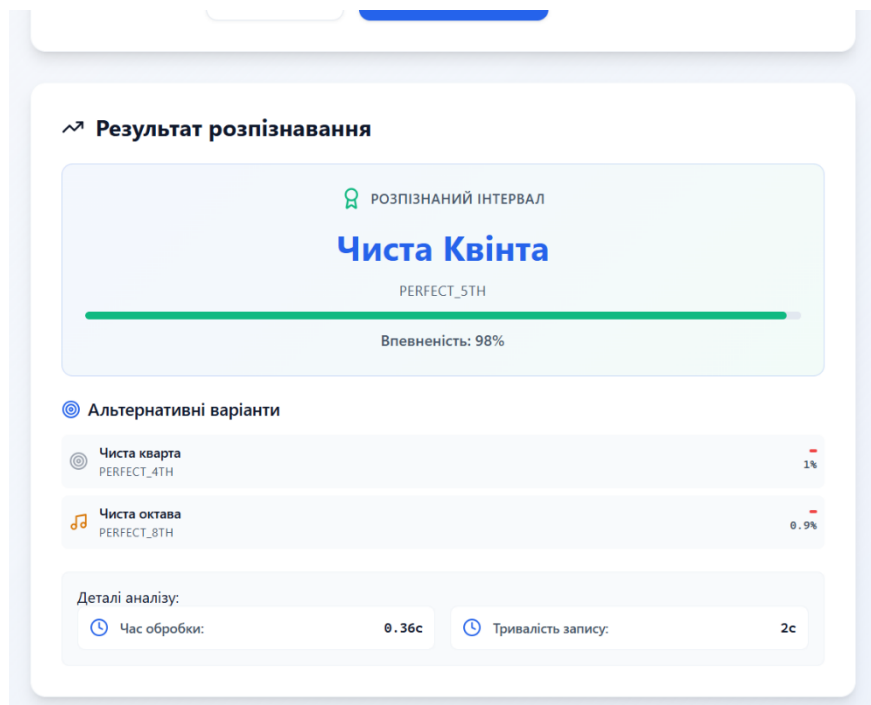


Рисунок 3.8 – Результат розпізнавання інтервалу

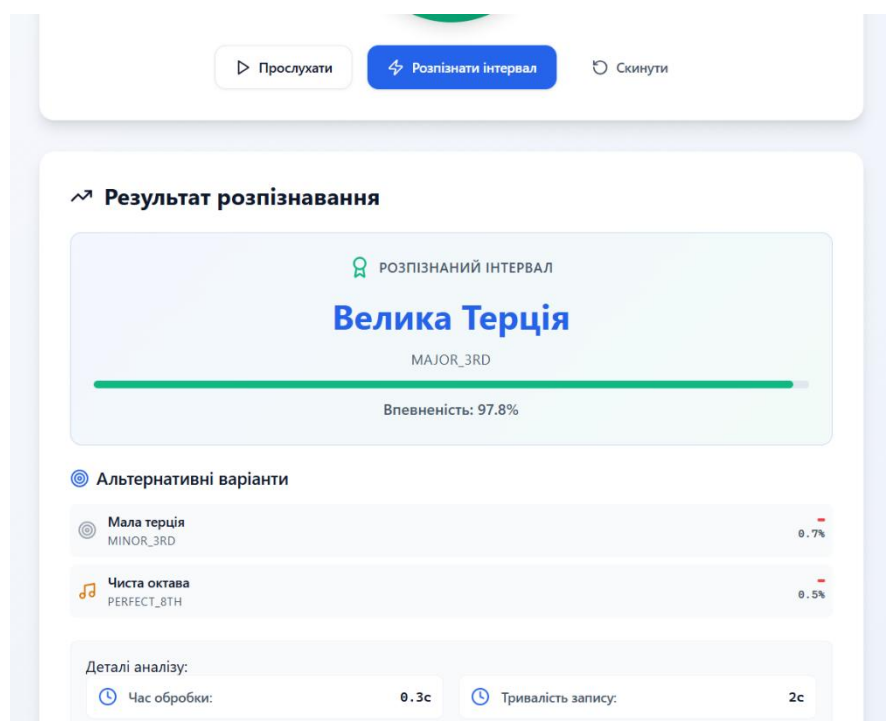


Рисунок 3.9 – Альтернативний приклад результату з відмінним основним інтервалом

На рис. 3.10 представлено інтерфейс налаштування параметрів тренування, в якому користувач обирає базову ноту від С до В та зазначає інтервали, з якими планується працювати. Система підтримує дванадцять інтервалів у межах однієї октави, кожен з яких має відповідну кількість півтонів, позначену в дужках.

Передбачено можливість вибору всіх інтервалів одразу або очищення раніше зазначених.

Налаштування тренування

Базова нота

C C# D D# E F F# G G# A A# B

Інтервали для тренування

Мала секунда (1 пт) Велика секунда (2 пт) Мала терція (3 пт) Велика терція (4 пт)

Чиста кварта (5 пт) Тритон (6 пт) Чиста квінта (7 пт) Мала секста (8 пт)

Велика секста (9 пт) Мала септима (10 пт) Велика септима (11 пт) Чиста октава (12 пт)

Згенерувати інтервали

Фортепіано

Рисунок 3.10 – Інтерфейс вибору базової ноти та переліку доступних інтервалів для тренування

На рисунку 3.11 зображено приклад конкретного налаштування для подальшого генерування аудіоприкладів. При виборі хоча б одного інтервалу розблоковується кнопка «Згенерувати інтервал».

Налаштування тренування

Базова нота

C C# D D# E F F# G G# A A# B

Інтервали для тренування

Мала секунда (1 пт) Велика секунда (2 пт) Мала терція (3 пт) Велика терція (4 пт)

Чиста кварта (5 пт) Тритон (6 пт) Чиста квінта (7 пт) Мала секста (8 пт)

Велика секста (9 пт) Мала септима (10 пт) Велика септима (11 пт) Чиста октава (12 пт)

Згенерувати інтервали

Рисунок 3.11 – Приклад обирання базової ноти та інтервалів

Після формування налаштувань користувач натискає кнопку генерації, і на екрані з'являється список інтервалів (рис. 3.12). Для кожного інтервалу виводиться його назва, напрям переходу від базової до цільової ноти, а також дві кнопки, які відповідають за прослуховування інтервалу з цілісним (одночасним) та поступовим відтворенням нот конструкції.

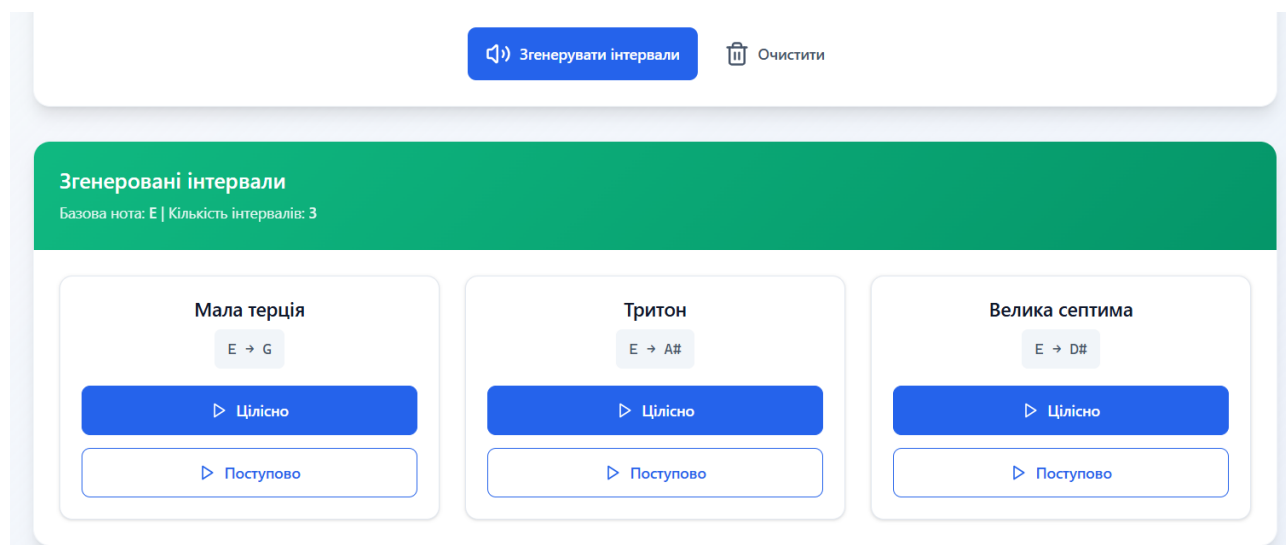


Рисунок 3.12 – Згенеровані інтервали після налаштування

На рис. 3.13 та рис. 3.14 продемонстровано взаємодію з елементами інтерфейсу під час відтворення звуку. Кнопка активного інтервалу змінює вигляд, що вказує на фазу відтворення. Система також повідомляє про поточний статус аудіо за допомогою повідомлення «Відтворюється...» та дозволяє зупинити програвання за потреби.

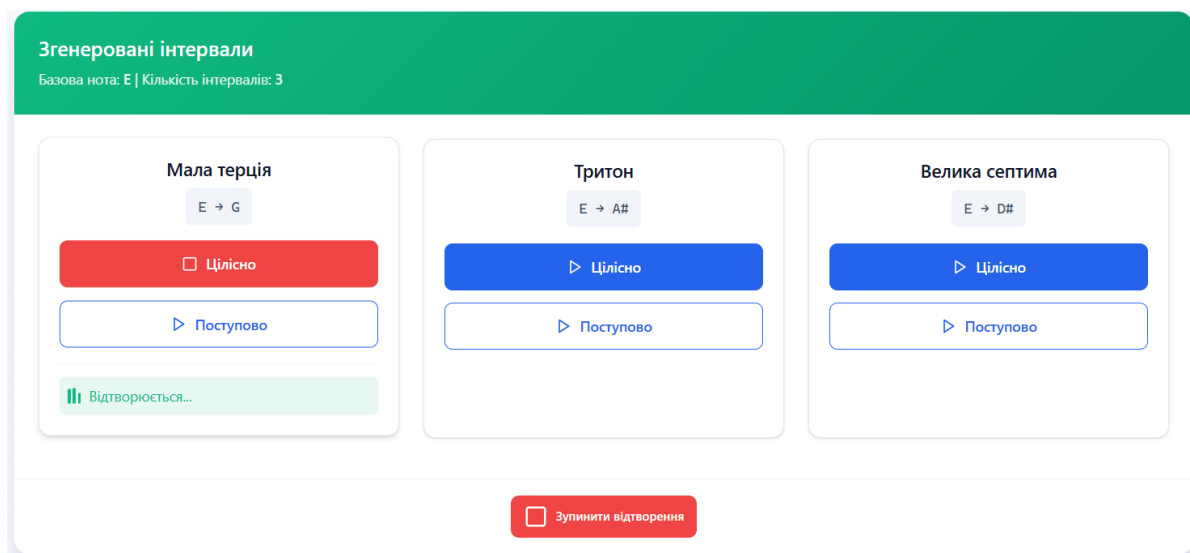


Рисунок 3.13 – Відтворення інтервалу «цілісно»

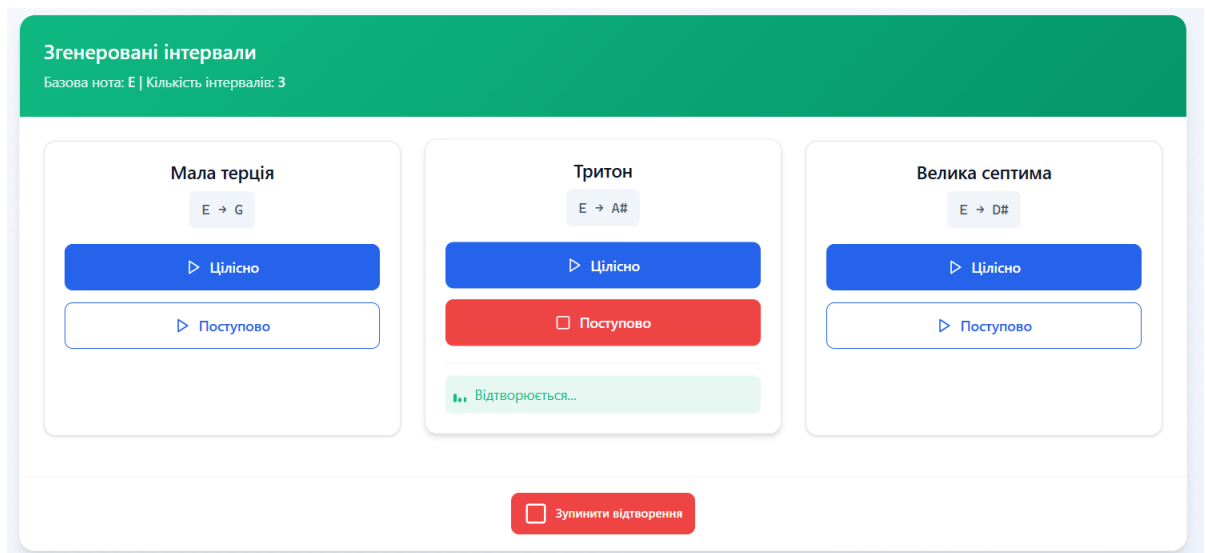


Рисунок 3.14 – Відтворення інтервалу «поступово»

Одним із важливих інструментів інтерфейсу є віртуальне фортепіано, розташоване в нижній частині сторінки (рис. 3.16). Воно розгортається за потреби користувача за допомогою спеціальної кнопки (рис. 3.15) і слугує як засіб для самостійного дослідження звукових інтервалів.

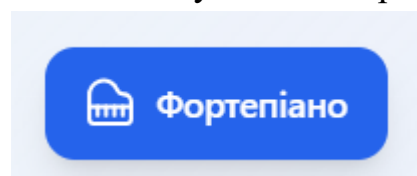


Рисунок 3.15 – Кнопка для розгортання віртуального фортепіано

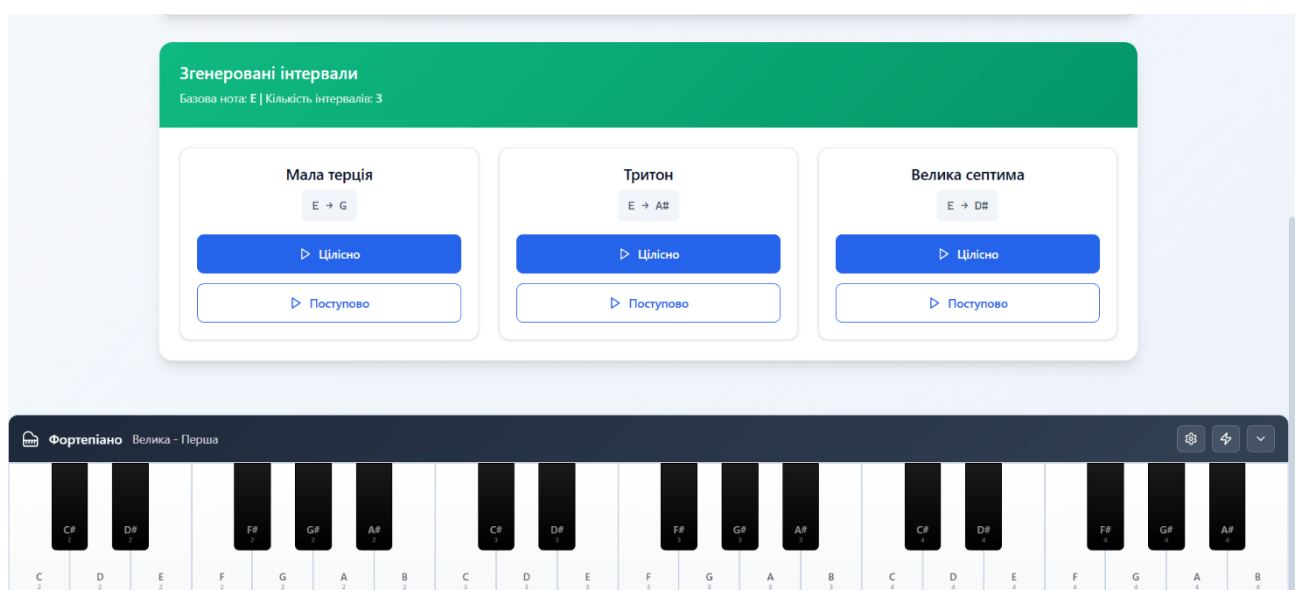


Рисунок 3.16 – Інтерфейс навчального блоку згенерованих інтервалів та відкрите фортепіано

Фортепіано підтримує повний діапазон — від субконтроктави (C0) до п'ятої октави (C8) (рис. 3.17), при цьому крайні октави реалізовані частково: субконтроктава представлена лише першими трьома клавішами, а п'ята — лише нотою C8. Користувач може самостійно налаштувати межі діапазону (рис. 3.18), обравши початкову та кінцеву октави. Також доступні перемикачі для відображення назв нот і номерів октав.

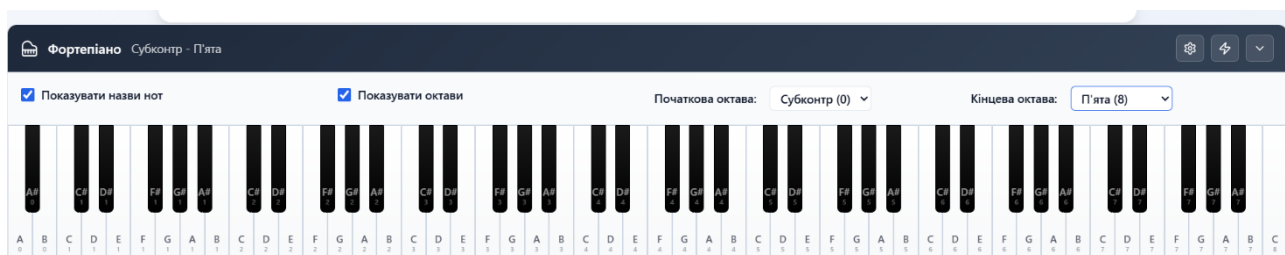


Рисунок 3.17 – Повністю розгорнуте фортепіано з діапазоном від субконтроктави до п'ятої октави

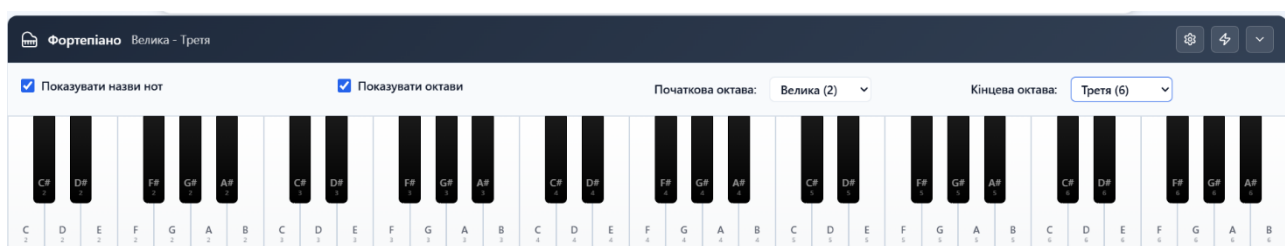


Рисунок 3.18 – Віртуальне фортепіано з активованими назвами нот та октав з діапазоном від великої до третьої октави

При натисканні (рис. 3.19) клавіша програвє звук тривалістю приблизно 1 секунду. Якщо клавішу утримувати, звук подовжується до 3,5 секунд. Для подовження звучання передбачено кнопку віртуальної педалі (рис. 3.20), при її активації усі натиснуті або утримувані ноти звучать до 7 секунд, а відпускання педалі миттєво зупиняє звучання. Також існує спеціальна кнопка для примусового припинення всіх звуків, яка з'являється лише під час активного звучання.

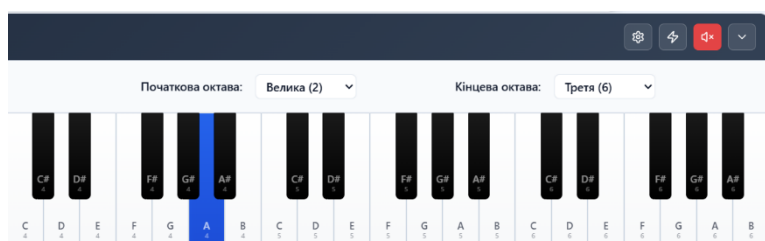


Рисунок 3.19 – Активована клавіша фортепіано (тривале натискання)

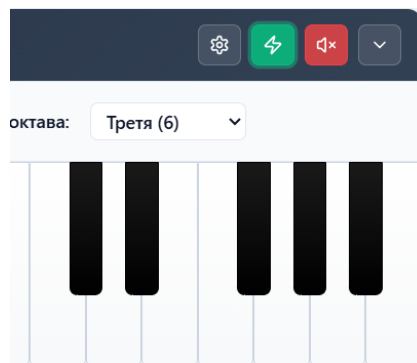


Рисунок 3.20 – Активована віртуальна педаль

Використання віртуального фортепіано дозволяє не лише слухати приклади інтервалів, але й порівнювати їх на слух з індивідуально натиснутими нотами, створюючи асоціативні зв'язки між теоретичним матеріалом і практичним звуковим досвідом (рис. 3.21). Цей інструмент може використовуватися як допоміжний під час навчання, або ж як окремий самостійний модуль для практики.

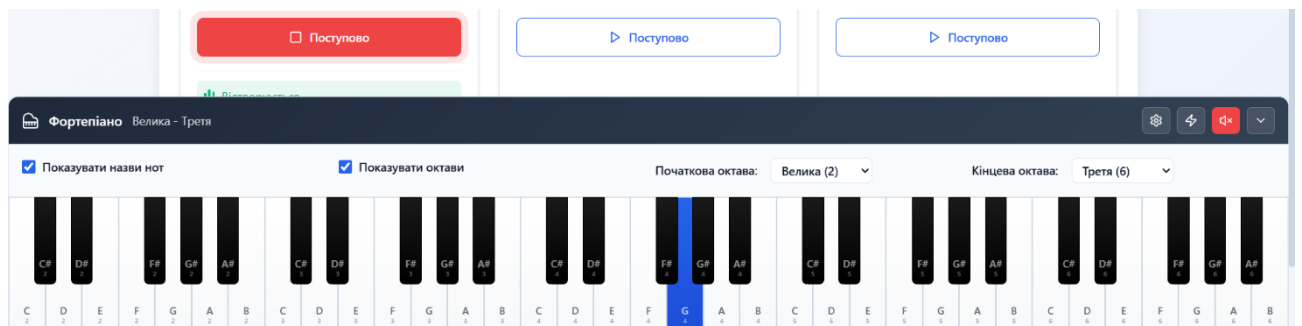


Рисунок 3.21 – Розгорнуте фортепіано у процесі порівняння з інтервалами

Інтерфейс фортепіано закріплений внизу сторінки, що забезпечує швидкий доступ незалежно від поточного положення користувача на сторінці.

### 3.2 Тестування датасету та моделі

Для забезпечення високої якості підготовленого датасету було розроблено спеціалізований модуль валідації, що дозволяє комплексно перевірити усі етапи обробки аудіоданих та оцінити готовність датасету до використання в задачах машинного навчання (рис. 3.22). Архітектура модуля побудована за принципом розділення відповідальності, де кожен компонент відповідає за валідацію конкретного типу даних або етапу обробки.

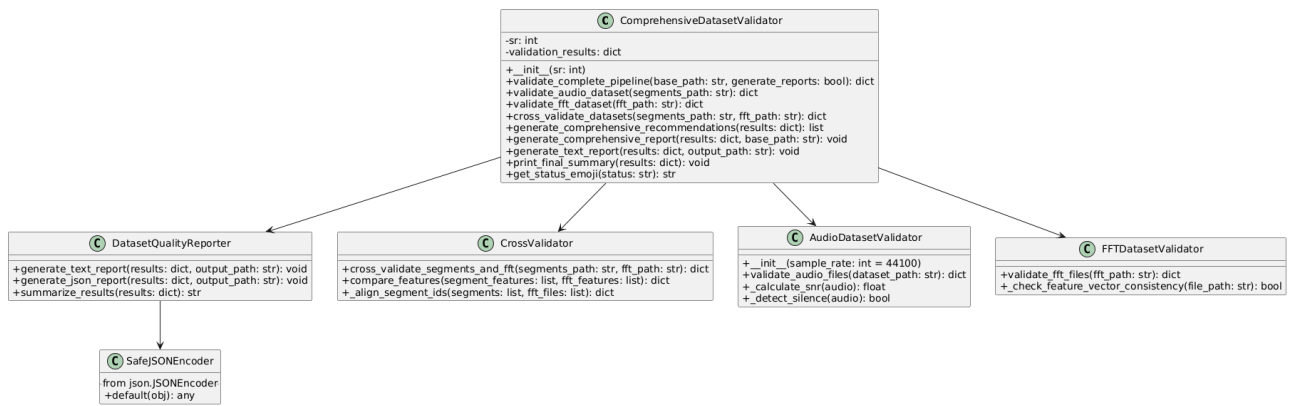


Рисунок 3.22 – Діаграма класів системи валідації датасету

Центральним елементом системи валідації є клас ComprehensiveDatasetValidator, який виконує роль координатора та забезпечує послідовне виконання всіх етапів перевірки.

Основний метод validate\_complete\_pipeline приймає шлях до базової директорії датасету та булевий параметр, що визначає необхідність генерації детальних звітів.

```

def validate_complete_pipeline(self, base_path, generate_reports=True):
    """Валідація всього пайплайну обробки"""
    print("КОМПЛЕКСНА ВАЛІДАЦІЯ ДАТАСЕТУ")
    print("=" * 60)

    results = {
        'timestamp': time.strftime('%Y-%m-%d %H:%M:%S'),
        'base_path': str(base_path)
    }

    try:
        segments_path = os.path.join(base_path, "processed-segments")
        if os.path.exists(segments_path):
            print("\nВАЛІДАЦІЯ АУДІОСГМЕНТІВ")
            print("-" * 40)
            audio_results = self.validate_audio_dataset(segments_path)
            results['audio_validation'] = audio_results
  
```

Процес валідації розділений на три основні етапи. Перший етап включає валідацію аудіосегментів, де перевіряється структура директорій, якість окремих аудіофайлів та збалансованість класів. Другий етап присвячений валідації FFT датасету, включаючи аналіз якості ознак, перевірку наявності некоректних значень та оцінку сепарабельності класів. Третій етап виконує крос-валідацію між аудіо та FFT датасетами для перевірки їх консистентності.

Валідація аудіодатасету здійснюється спеціалізованим класом `AudioDatasetValidator`, який аналізує структуру сегментованих аудіофайлів. Клас містить методи для перевірки директорійної структури, аналізу якості окремих файлів та оцінки балансу класів. Метод `check_single_audio_file` виконує детальний аналіз кожного аудіофайлу, перевіряючи тривалість запису, рівні RMS та пікових значень, наявність кліпування та співвідношення сигналу до тиші.

```
def check_single_audio_file(self, file_path):
    """Перевірка одного аудіофайлу"""
    issues = []
    self.stats['total_files'] += 1

    try:
        audio, sr = librosa.load(file_path, sr=self.sr, mono=True)
        duration = len(audio) / sr
        rms = np.sqrt(np.mean(audio**2))
        peak = np.max(np.abs(audio))

        if rms > 0:
            rms_db = 20 * np.log10(rms)
        else:
            rms_db = -100
        issues.append('silent_file')

    self.quality_metrics['rms_db'].append(rms_db)
    self.quality_metrics['duration'].append(duration)
    self.quality_metrics['peak'].append(peak)
```

```

if duration < 1.8:
    issues.append('too_short')
elif duration > 3.0:
    issues.append('too_long')

```

Критерії валідації аудіофайлів включають перевірку тривалості сегментів, які повинні знаходитися в діапазоні від 1.8 до 3.0 секунд, контроль RMS рівня в межах від -60 до -6 дБ, виявлення кліпування при перевищенні 1% семплів порогового значення 0.98, та аналіз співвідношення тиші, яке не повинно перевищувати 80% від загальної тривалості сегменту. Додатково перевіряється консистентність тривалості сегментів, де стандартне відхилення не повинно перевищувати 10% від середнього значення.

Валідація FFT датасету реалізована в класі `FFTDatasetValidator`, який підтримує завантаження даних як у форматі `pickle`, так і у форматі `NPZ`. Основний метод `analyze_fft_data` виконує комплексний аналіз завантажених ознак, включаючи перевірку якості ознак, аналіз балансу класів та оцінку сепарабельності. Метод `analyze_feature_quality` здійснює пошук некоректних значень, аналізує варіацію ознак та виявляє високо корельовані пари ознак.

```

def analyze_feature_quality(self, X):
    """Аналіз якості ознак"""
    nan_count = int(np.sum(np.isnan(X)))
    inf_count = int(np.sum(np.isinf(X)))

    feature_vars = np.var(X, axis=0)
    low_variance_features = int(np.sum(feature_vars < 1e-8))

    try:
        feature_corr = np.corrcoef(X.T)
        high_corr_pairs = int(np.sum(np.abs(feature_corr) > 0.95) - X.shape[1])
    except:
        high_corr_pairs = 0

```

Особливої уваги заслуговує метод аналізу сепарабельності класів, який використовує алгоритм `Random Forest` для швидкої оцінки здатності ознак до

розрізнення класів. Для оптимізації часу виконання, якщо датасет містить більше 1000 зразків, аналіз проводиться на випадковій вибірці.

```
def analyze_class_separability(self, X, y):
    """Аналіз сепарабельності класів"""
    if not SKLEARN_AVAILABLE:
        return {'error': 'sklearn недоступний', 'separable': False}

    try:
        le = LabelEncoder()
        y_encoded = le.fit_transform(y)

        if len(X) > 1000:
            indices = np.random.choice(len(X), 1000, replace=False)
            X_sample = X[indices]
            y_sample = y_encoded[indices]
        else:
            X_sample = X
            y_sample = y_encoded

        rf = RandomForestClassifier(n_estimators=10, random_state=42, n_jobs=1)
        scores = cross_val_score(rf, X_sample, y_sample, cv=3, scoring='accuracy')

        separability_score = float(np.mean(scores))
```

Крос-валідація між датасетами виконується класом CrossDatasetValidator, який порівнює кількість файлів в аудіодатасеті з кількістю зразків у FFT датасеті. Метод compare\_datasets розраховує коефіцієнт конверсії, аналізує спільні та відсутні класи, і обчислює загальний показник консистентності. Консистентність вважається задовільною при значенні показника понад 0.85.

Для генерації якісного показника FFT датасету використовується зважена система оцінки, де базові критерії (кількість зразків та класів) складають 40% загальної оцінки, якість ознак – 30%, баланс класів – 20%, та сепарабельність – 10%.

```
def calculate_fft_quality_score(self, analysis):
    """Розрахунок загального показника якості FFT датасету"""
    score = 0.0

    if analysis['total_samples'] >= 1000:
        score += 0.2
    elif analysis['total_samples'] >= 500:
        score += 0.1

    if analysis['num_classes'] >= 8:
        score += 0.2
    elif analysis['num_classes'] >= 5:
        score += 0.1
```

Система валідації також використовує JSON енкодер, який забезпечує коректну серіалізацію складних структур даних без циклічних посилань. Клас `SafeJSONEncoder` рекурсивно обробляє об'єкти різних типів, включаючи `numpy` масиви, `defaultdict` та користувацькі об'єкти, перетворюючи їх у формат, придатний для JSON серіалізації.

Результати валідації представляються у вигляді комплексних звітів, які включають JSON файл з детальними метриками, текстовий звіт з основними показниками та HTML звіт з візуалізацією. Клас `DatasetQualityReporter` генерує інтерактивні HTML звіти з таблицями та графіками розподілу класів для аудіо та FFT датасетів. Модуль також включає систему рекомендацій, яка на основі результатів валідації генерує конкретні поради щодо покращення якості датасету. Рекомендації враховують показники успішності аудіовалідації, якість FFT датасету та рівень консистентності між компонентами системи.

Результати виконання комплексної валідації датасету демонструють високу якість підготовлених даних та їх готовність до використання в задачах машинного навчання (рис. 3.23). Система валідації успішно завершила всі три етапи перевірки, включаючи аналіз аудіосегментів, валідацію FFT датасету та крос-валідацію консистентності між компонентами.

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS  POSTMAN CONSOLE

ЗАПУСК КОМПЛЕКСНОЇ ВАЛІДАЦІЇ
Базовий шлях: dataset-preparation

КОМПЛЕКСНА ВАЛІДАЦІЯ ДАТАСЕТУ

=====

ВАЛІДАЦІЯ АУДІОСЕМЕНТІВ
-----
Аналіз структури аудіосегментів...

ВАЛІДАЦІЯ FFT ДАТАСЕТУ
-----
Аналіз FFT датасету...

КРОС-ВАЛІДАЦІЯ ДАТАСЕТІВ
-----
Крос-валідація датасетів...

Звіти збережено:
  JSON: dataset-preparation\comprehensive_validation_report.json
  Text: dataset-preparation\validation_summary.txt

=====

ФІНАЛЬНЕ РЕЗЮМЕ
=====
Аудіодатасет:  ✓ success
FFT датасет:   ✓ success
Крос-валідація: ✓ success

КЛЮЧОВІ РЕКОМЕНДАЦІЇ:
  Хороша якість аудіодатасету
  FFT датасет готовий для ML тренування
  Відмінна консистентність між датасетами

Створення детального звіту...
Детальний звіт створено: dataset-preparation\quality_reports\dataset_quality_report.html

Валідація завершена!

```

Рисунок 3.23 – Результат виконання комплексної валідації датасету в терміналі

Аналіз розподілу класів в аудіодатасеті показує високий рівень збалансованості між різними типами музичних інтервалів (рис. 3.24). Кількість аудіофайлів для кожного класу коливається в межах від 348 до 360 файлів, що забезпечує рівномірне представлення всіх інтервалів у навчальній вибірці.

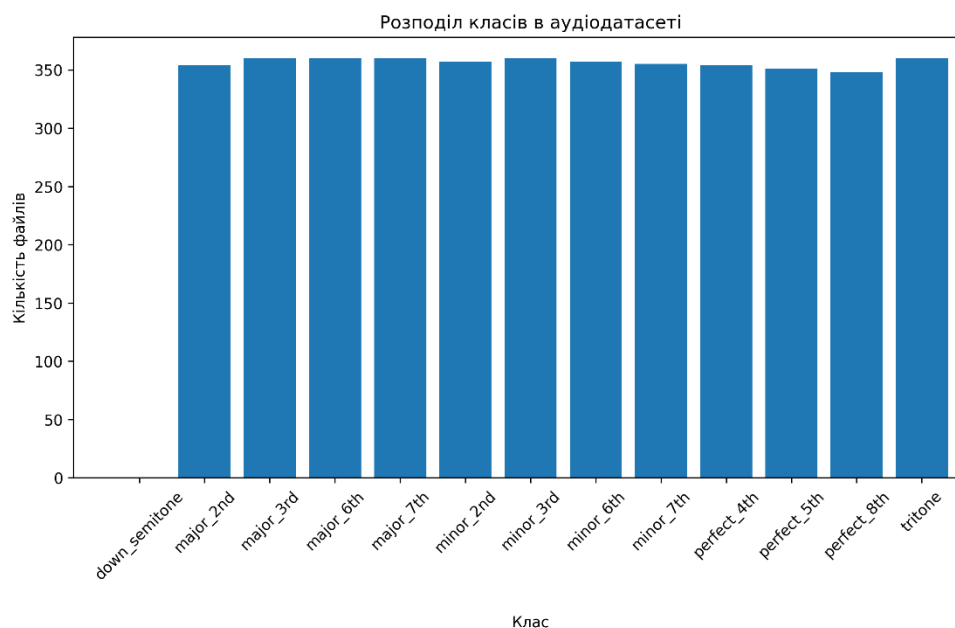


Рисунок 3.24 – Розподіл класів музичних інтервалів в аудіодатасеті за кількістю файлів

Незначні відхилення в кількості файлів між класами не перевищують 2%, що свідчить про ефективність процедури сегментації та рівномірність розподілу вихідного матеріалу.

Аналіз FFT датасету підтверджує успішну конверсію аудіосегментів у векторне представлення ознак (рис. 3.25). Розподіл класів у FFT датасеті демонструє ще вищий рівень збалансованості порівняно з вихідним аудіодатасетом. Кількість зразків для кожного класу варіює від 348 до 360, при цьому стандартне відхилення становить менше 2% від середнього значення. Загальна кількість зразків у FFT датасеті складає 4276 векторів ознак, що забезпечує достатній обсяг даних для тренування та валідації моделей машинного навчання.

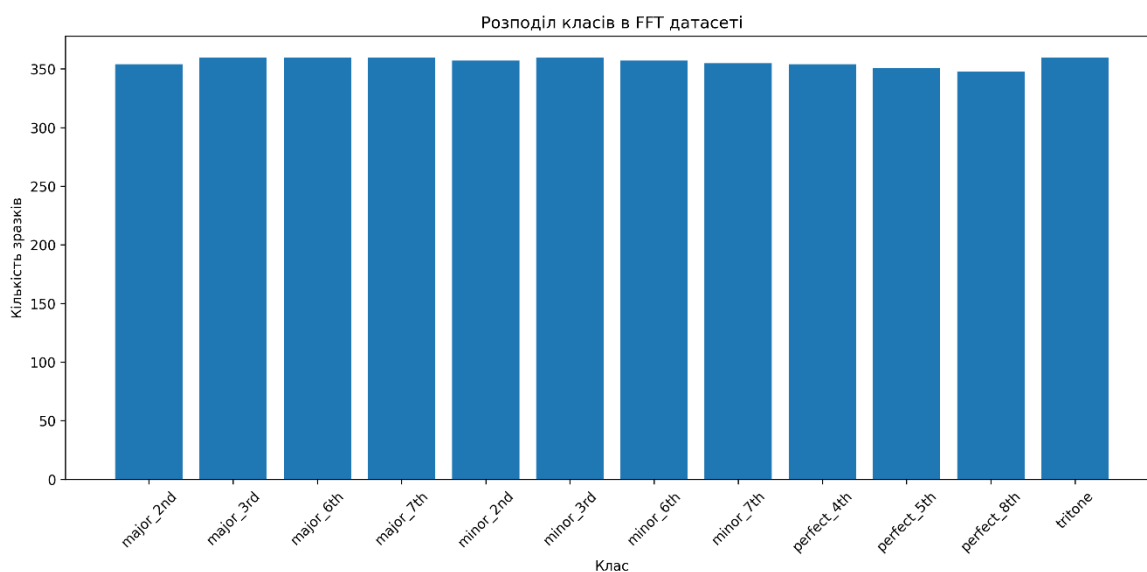


Рисунок 3.25 – Розподіл класів музичних інтервалів в FFT датасеті за кількістю зразків

Система валідації згенерувала комплексні рекомендації щодо якості підготовленого датасету. Оцінка «Хороша якість аудіодатасету» базується на високому показнику успішності валідації аудіофайлів, що перевищує 85% порогове значення. Рекомендація «FFT датасет готовий для ML тренування» підтверджує, що якісний показник перевищує 0.9, вказуючи на відсутність критичних проблем з ознаками та достатню сепарабельність класів. Особливо важливою є оцінка «Відмінна консистентність між датасетами» з показником

понад 0.95, що гарантує цілісність та узгодженість всього пайплайну обробки даних.

Після завершення тренування моделі розпізнавання музичних інтервалів виникає необхідність у комплексному тестуванні її продуктивності на незалежному наборі даних. Для цієї мети було розроблено спеціалізований клас `FFTModelTester` (рис. 3.26), який забезпечує завантаження навченої моделі, підготовку тестових даних та детальний аналіз результатів класифікації.

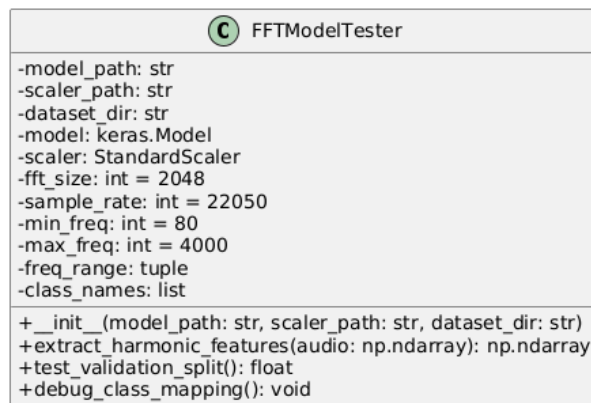


Рисунок 3.26 – Клас тестування моделі

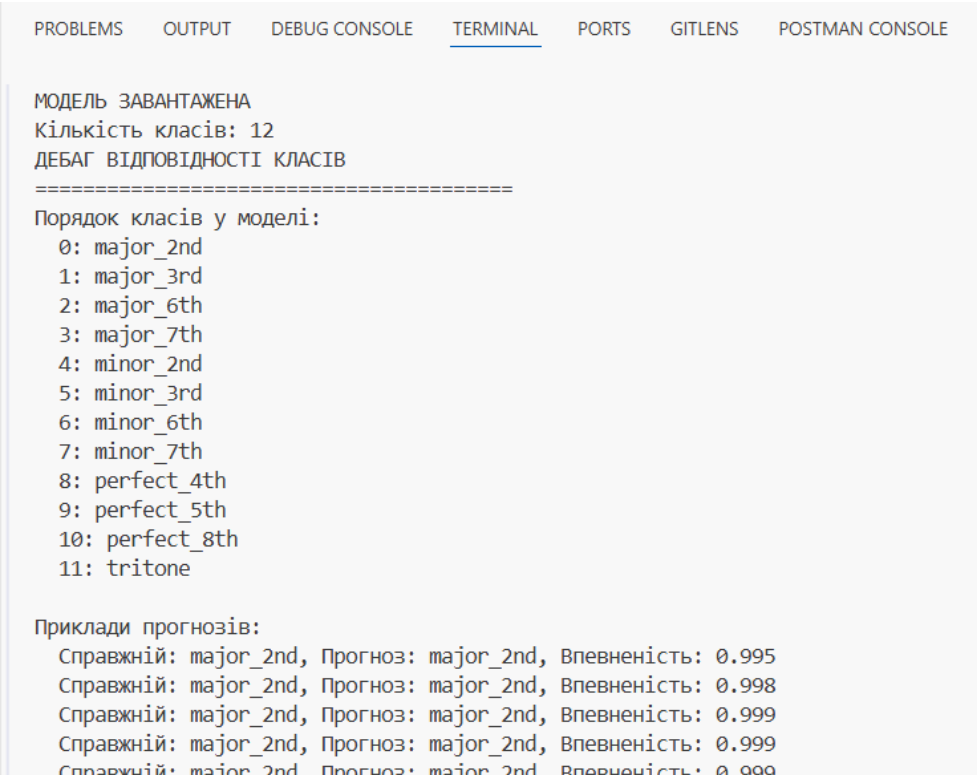
Клас `FFTModelTester` ініціалізується шляхами до збереженої моделі, файлу нормалізатора та директорії з датасетом. При завантаженні модель автоматично відновлює свою архітектуру та ваги з файлу формату Keras, а нормалізатор ознак завантажується з файлу `joblib`. Особливістю реалізації є автоматичне відновлення порядку класів з оригінального датасету, що гарантує коректність інтерпретації результатів прогнозування.

Для забезпечення консистентності векторного представлення аудіосигналів методи `extract_harmonic_features` та `_extract_additional_features` здійснюють віконне перетворення Фур'є, нормалізацію спектральних ознак та додавання аудіоописаторів, таких як спектральний центроїд, ширина смуги та характеристики піків.

Метод `test_validation_split` виконує тестування моделі на заздалегідь підготовленому тестовому наборі або створює новий розподіл даних з основного датасету.

Для візуалізації результатів тестування використовується метод `_plot_confusion_matrix`, який будує теплокарту (heatmap) матриці. Матриця помилок надає детальну інформацію про співвідношення між прогнозованими та фактичними класами, дозволяючи виявити інтервали, які найчастіше плутаються між собою. Така візуалізація є корисною для глибшого аналізу помилок класифікації та виявлення потенційних проблем у навчанні моделі.

Результати тестування демонструють високу ефективність розробленої моделі розпізнавання музичних інтервалів (рис. 3.27 – 3.28). Загальна точність класифікації на тестовому наборі з 856 зразків склала 92.06%, що свідчить про успішність обраного підходу до виділення ознак та архітектури нейронної мережі. Особливо важливо відзначити рівномірно високі показники для всіх класів інтервалів, що підтверджує збалансованість навчального процесу.



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS  POSTMAN CONSOLE

МОДЕЛЬ ЗАВАНТАЖЕНА
Кількість класів: 12
ДЕБАГ ВІДПОВІДНОСТІ КЛАСІВ
=====
Порядок класів у моделі:
0: major_2nd
1: major_3rd
2: major_6th
3: major_7th
4: minor_2nd
5: minor_3rd
6: minor_6th
7: minor_7th
8: perfect_4th
9: perfect_5th
10: perfect_8th
11: tritone

Приклади прогнозів:
Справжній: major_2nd, Прогноз: major_2nd, Впевненість: 0.995
Справжній: major_2nd, Прогноз: major_2nd, Впевненість: 0.998
Справжній: major_2nd, Прогноз: major_2nd, Впевненість: 0.999
Справжній: major_2nd, Прогноз: major_2nd, Впевненість: 0.999
Справжній: major_2nd, Прогноз: major_2nd, Впевненість: 0.999
  
```

Рисунок 3.27 – Верифікація класів моделі та демонстрація точності прогнозів

Аналіз детальних метрик по класах (рис. 3.28) показує, що найвищі показники точності досягнуті для тритону (0.99), великої терції (0.94) та малої

секунди (0.94). Тритон, як найбільш характерний інтервал з унікальними гармонічними властивостями, демонструє практично ідеальне розпізнавання. Дещо нижчі показники спостерігаються для чистої квінти (0.88) та чистої октави (0.91), що може бути пов'язано з більшою варіативністю їх спектральних характеристик в різних тембральних контекстах.

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS  POSTMAN CONSOLE

СТВОРЕННЯ ТЕСТОВОГО НАБОРУ З ОСНОВНОГО ДАТАСЕТУ
=====
Тестовий набір: 856 зразків
27/27  0s 2ms/step

Точність на тестовому наборі: 0.9206 (92.06%)

Звіт по класах:
      precision    recall  f1-score   support

major_2nd      0.87      0.96      0.91        71
major_3rd      0.94      0.93      0.94        72
major_6th      0.92      0.96      0.94        72
major_7th      0.90      0.90      0.90        72
minor_2nd      0.94      0.87      0.91        71
minor_3rd      0.91      0.93      0.92        72
minor_6th      0.93      0.94      0.94        72
minor_7th      0.94      0.90      0.92        71
perfect_4th     0.93      0.92      0.92        71
perfect_5th     0.88      0.90      0.89        70
perfect_8th     0.91      0.84      0.87        70
tritone        0.99      0.99      0.99        72

accuracy              0.92      856
macro avg      0.92      0.92      0.92      856
weighted avg   0.92      0.92      0.92      856

Матрицю помилок збережено: confusion_matrix.png
PS D:\HarmonicPerceptionTrainer\backend>

```

Рисунок 3.28 – Метрики якості класифікації музичних інтервалів по класах

Показники recall варіюють від 0.84 для чистої октави до 0.99 для тритону, при цьому середнє значення становить 0.92. F1-міра, як збалансована метрика, також демонструє стабільно високі значення в діапазоні від 0.87 до 0.99, що підтверджує надійність класифікації як з точки зору точності, так і повноти.

Матриця помилок надає детальну картину характеру помилок класифікації (рис. 3.29). Діагональні елементи матриці відображають кількість правильно класифікованих зразків для кожного інтервалу, при цьому найяскравіші значення спостерігаються для тритону (71 правильна класифікація), великої секунди (68)

та великої терції (67). Недіагональні елементи показують конкретні випадки неправильної класифікації між парами інтервалів.

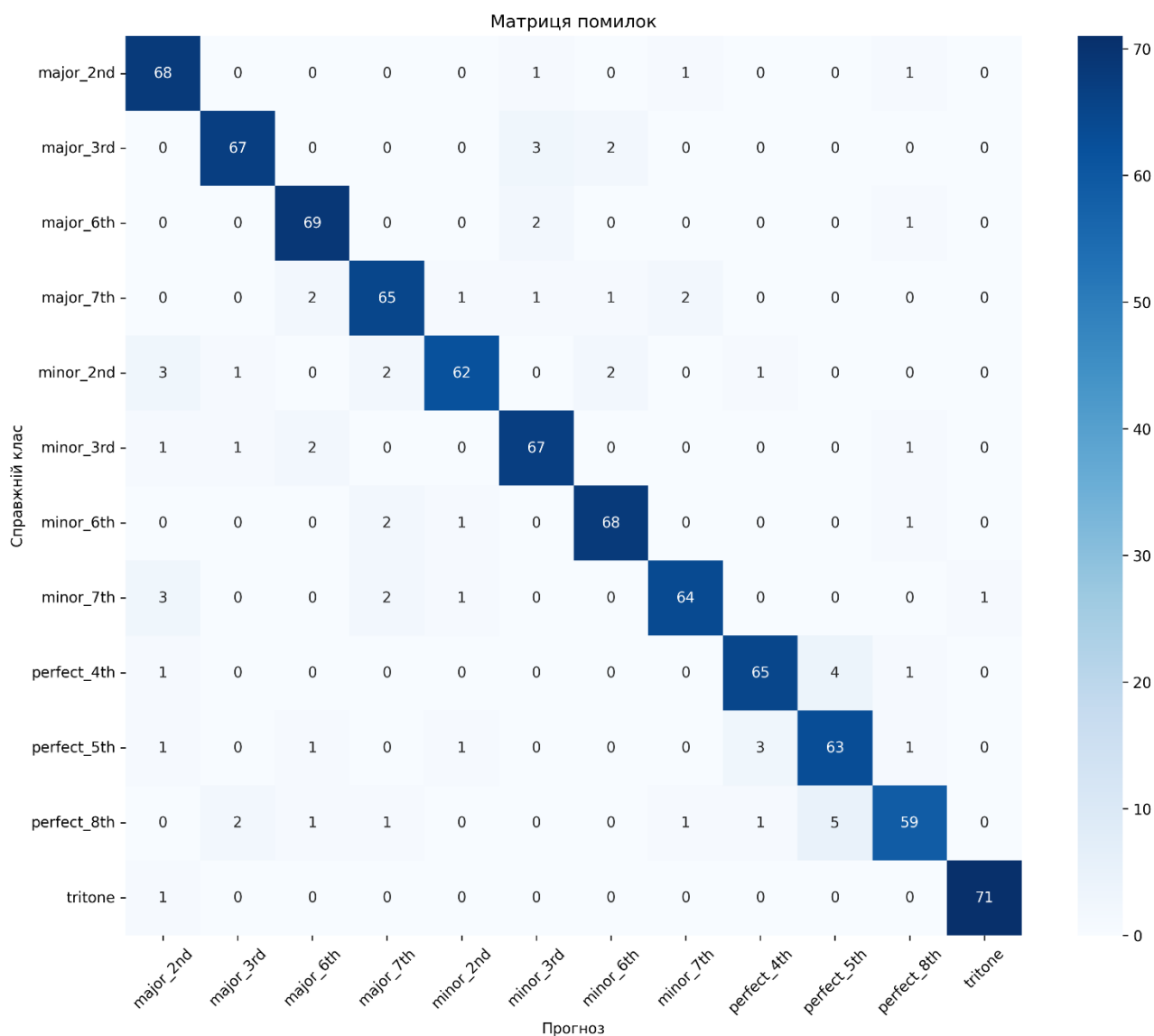


Рисунок 3.29 – Матриця помилок для класифікації музичних інтервалів

Найчастіші помилки спостерігаються між акустично близькими інтервалами. Зокрема, чиста кварта іноді плутається з чистою квінтою (4 випадки), що може бути пов'язано з їх гармонічною спорідненістю як консонансних інтервалів. Малі та великі інтервали одного типу також демонструють взаємну плутанину, наприклад, між малою та великою секундою (3 випадки), що відображає об'єктивну складність їх розрізнення на основі спектральних характеристик.

Додатковим фактором, що впливає на характер помилок, є явище інверсії інтервалів у музичній теорії. Інтервали, які є оберненнями один одного, можуть мати подібні гармонічні властивості. Наприклад, секунда (до – ре) та септима (ре – до наступної октави) містять ті самі звуки, але в різному порядку, що може призводити до схожих спектральних компонентів у FFT представленні. Це пояснює деякі випадки плутанини між віддаленими за номером, але близькими за гармонічним змістом інтервалами.

Особливо цікавим є факт мінімальної кількості помилок для тритону, який демонструє лише поодинокі випадки неправильної класифікації. Це підтверджує унікальність його акустичних властивостей, оскільки тритон є єдиним інтервалом, який при інверсії утворює ідентичний за розміром інтервал, що забезпечує йому особливу гармонічну стабільність та відсутність "подвійника" у спектральному просторі. Чисті інтервали (кварта, квінта, октава) показують дещо більшу взаємну плутанину, що може вказувати на необхідність додаткових ознак для їх надійного розрізнення.

Загалом, аналіз матриці помилок підтверджує, що основні помилки класифікації відбуваються між акустично близькими та інверсійними інтервалами, що є очікуваним результатом і не вказує на системні проблеми моделі. Високі показники по діагоналі матриці свідчать про успішність навчання та готовність моделі до практичного використання в задачах автоматичного розпізнавання музичних інтервалів.

### 3.3 Тестування роботи програмного продукту

Етап тестування розробленої системи для розвитку гармонічного слуху проводився згідно з комплексною методологією, що охоплювала функціональне тестування.

Функціональне тестування проводилося за методологією чорної скриньки з використанням розроблених тест-кейсів (табл. 3.1 – 3.3). Кожен тест-кейс включав вхідні дані, очікуваний результат та критерії успішного проходження.

Тестування охоплювало основні функціональні модулі системи: автентифікацію користувачів, аудіозапис та режим тренування з генерацією інтервалів.

Таблиця 3.1

Тест-кейси модуля автентифікації

| № з/п | Назва тесту                      | Вхідні дані                                       | Очікуваний результат                                     | Статус   |
|-------|----------------------------------|---------------------------------------------------|----------------------------------------------------------|----------|
| 1     | Успішний вхід з валідними даними | username: "testuser",<br>password: "validpass123" | Перенаправлення на головну сторінку, JWT токен збережено | Пройдено |
| 2     | Вхід з пустим логіном            | username: "",<br>password: "validpass123"         | Попередження «Заповніть це поле»                         | Пройдено |
| 3     | Вхід з пустим паролем            | username: "testuser",<br>password: ""             | Попередження «Заповніть це поле»                         | Пройдено |
| 4     | Вхід з коротким паролем          | username: "testuser",<br>password: "123"          | Помилка «Пароль повинен містити мінімум 5 символів»      | Пройдено |
| 5     | Вхід з невірними даними          | username: "wronguser",<br>password: "wrongpass"   | Помилка «Невірний логін або пароль»                      | Пройдено |

Таблиця 3.2

Тест-кейси модуля аудіозапису

| № з/п | Назва тесту                 | Вхідні дані                  | Очікуваний результат                             | Статус   |
|-------|-----------------------------|------------------------------|--------------------------------------------------|----------|
| 1     | Запуск запису з мікрофона   | Дозвіл на мікрофон надано    | Індикатор запису активний, таймер працює         | Пройдено |
| 2     | Зупинка запису користувачем | Натискання кнопки «Зупинити» | Аудіофайл створений, кнопка «Прослухати» активна | Пройдено |

Продовження табл. 3.2

|   |                                               |                             |                                                                                                                             |          |
|---|-----------------------------------------------|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------|----------|
| 3 | Відтворення записаного аудіо                  | Натискання «Прослухати»     | Звук відтворюється, індикатор відтворення                                                                                   | Пройдено |
| 4 | Запис нового аудіо без видалення попереднього | Натискання «Новий запис»    | Індикатор запису активний, таймер працює                                                                                    | Пройдено |
| 5 | Видалення аудіо                               | Натискання кнопки «Скинути» | Видалення запису, очищення інформації про запис з екрану та приховання кнопок відтворення, розпізнавання та видалення аудіо | Пройдено |

Таблиця 3.3

## Тест-кейси модуля тренування

| № з/п | Назва тесту                                  | Вхідні дані                                               | Очікуваний результат                                     | Статус   |
|-------|----------------------------------------------|-----------------------------------------------------------|----------------------------------------------------------|----------|
| 1     | Генерація інтервалів з валідними параметрами | note: "C",<br>intervals: ["major_third", "perfect_fifth"] | 2 аудіофайли створено, кнопки відтворення активні        | Пройдено |
| 2     | Спроба генерації без вибору інтервалів       | note: "C",<br>intervals: []                               | Заблокування кнопки генерації інтервалів                 | Пройдено |
| 3     | Цілісне відтворення інтервалу                | Натискання «Цілісно» для мажорної терції                  | Ноти звучать одночасно та утворюють правильний інтервал  | Пройдено |
| 4     | Поступове відтворення інтервалу              | Натискання «Поступово» для квінти                         | Ноти звучать послідовно та утворюють правильний інтервал | Пройдено |
| 5     | Відтворення аудіо під час звучання іншого    | Натискання «Цілісно» під час звучання інтервалу           | Відтворення звучання з зупинкою попереднього аудіо       | Пройдено |

Результати функціонального тестування, представлені в табл. 3.1 – 3.3, підтверджують коректність роботи основних модулів системи. Всі 15 тест-кейсів пройшли успішно, що свідчить про надійність реалізованого функціоналу. Модуль автентифікації забезпечує належну валідацію користувацьких даних та безпечний доступ до системи. Компонент аудіозапису стабільно працює з різними сценаріями використання. Тренувальний модуль коректно генерує та відтворює музичні інтервали, забезпечуючи якісний навчальний досвід. Виявлена відсутність критичних дефектів у функціональності дозволяє стверджувати про готовність системи до використання для розвитку гармонічного слуху.

### **Висновки до третього розділу**

У третьому розділі було представлено результати реалізації системи для розвитку гармонічного слуху та проведено тестування її функціональності.

Реалізований інтерфейс включає модуль автентифікації з валідацією даних користувачів та JWT токенизацією, компонент розпізнавання з можливістю запису аудіо та отримання результатів аналізу, тренувальний модуль для генерації аудіоприкладів інтервалів. Окремо впроваджено віртуальне фортепіано з діапазоном від субконтроктави до п'ятої октави, що надає можливість практичного порівняння звучання інтервалів.

Валідація датасету здійснювалася через систему перевірки якості аудіосегментів, аналізу FFT представлення та крос-валідації консистентності. Отримані показники складають 85% успішності валідації аудіофайлів, 0.9 якісного індексу FFT датасету та 0.95 консистентності між компонентами. Розподіл класів демонструє збалансованість з відхиленнями до 2% між типами інтервалів.

Тестування моделі на наборі з 856 зразків показало точність 92.06%. Найвищі результати отримано для тритону з precision та recall 0.99. Матриця помилок засвідчила, що неточності класифікації виникають переважно між акустично близькими інтервалами.

Функціональне тестування охопило 15 тест-кейсів за методологією чорної скриньки для модулів автентифікації, аудіозапису та тренування. Успішне проходження всіх тестів підтвердило коректність валідації даних, стабільність аудіообробки та належне функціонування навчальних компонентів.

Результати свідчать про готовність системи до практичного застосування. Досягнута точність розпізнавання, функціональність інтерфейсу та відсутність критичних дефектів дозволяють рекомендувати систему до використання в процесі розвитку музичного слуху.

## ВИСНОВКИ

У результаті виконання кваліфікаційної роботи було успішно розроблено та реалізовано систему для розвитку гармонічного слуху, яка інтегрує сучасні технології цифрової обробки аудіосигналів, машинного навчання та веб-розробки для забезпечення ефективного навчання користувачів розпізнаванню музичних інтервалів.

Проведений детальний аналіз існуючих програмних рішень виявив критичні обмеження наявних аналогів: відсутність підтримки розпізнавання одночасного звучання кількох нот у додатку Simply Sing, обмежений функціонал віртуального фортепіано на платформі Musissa без можливості поліфонії та педалі сустейну, а також брак комплексних систем оцінювання прогресу користувачів. Розроблена система усуває ці недоліки завдяки впровадженню спеціалізованого модуля розпізнавання музичних конструкцій, повнофункціонального віртуального фортепіано з діапазоном від субконтроктави до п'ятої октави та інтегрованої системи тестування з нарахуванням балів досвіду.

Обрана клієнт-серверна архітектура з REST API на основі Django та React.js забезпечила оптимальний баланс між функціональністю та масштабованістю. Django забезпечив тісну інтеграцію з Python-екосистемою машинного навчання та вбудовані механізми безпеки, тоді як React.js гарантує швидку інтерактивну роботу з аудіоконтентом без затримок, критичних для музичних додатків.

Центральним технічним аспектом роботи є створення нейронної мережі для автоматичного розпізнавання музичних інтервалів. Багатошарова архітектура з повнозв'язними шарами забезпечила точність класифікації 92,06% на тестовому наборі з 856 зразків. Особливо високі результати досягнуто для тритону (точність та повнота 0,99), що підтверджує ефективність підходу до виділення спектральних ознак через швидке перетворення Фур'є з аналізом гармонічної структури.

Створення репрезентативного датасету включало систематичний збір та обробку 24 базових аудіофайлів з подальшою автоматизованою сегментацією, що забезпечило 4276 якісних навчальних зразків. Розроблена система валідації продемонструвала високі показники: коефіцієнт успішності валідації аудіофайлів 0,85, індекс якості FFT датасету 0,9 та консистентність між компонентами 0,95. Збалансованість класів з відхиленнями до 2% гарантує рівномірне представлення всіх 12 типів інтервалів.

Модуль тренування реалізує адитивний синтез звуку з використанням ADSR огинаючої та підтримкою гармонік, що забезпечує природну динаміку звучання. Система підтримує цілісний та поступовий режими відтворення інтервалів для розвитку різних аспектів музичного сприйняття. Віртуальне фортепіано з підтримкою поліфонії, педалі сустейну та налаштовуваними параметрами відображення служить ефективним інструментом для практичного порівняння звучання.

Підсистема тестування забезпечує об'єктивну оцінку прогресу через структуровані тести з адаптивною складністю (легкий, середній та складний рівні) та детальну аналітику результатів з персоналізованими рекомендаціями. Система автоматично генерує тестові завдання з рівномірним розподілом інтервалів та забезпечує збереження статистики прогресу.

Комплексне функціональне тестування охопило 15 тест-кейсів за методологією чорної скриньки для модулів автентифікації, аудіозапису та тренування. Успішне проходження всіх тестів підтвердило коректність валідації даних, стабільність аудіообробки та належне функціонування навчальних компонентів. Аналіз матриці помилок показав, що неточності класифікації виникають переважно між акустично близькими інтервалами (чиста кварта/квінта, малі/великі варіанти), що є очікуваним результатом і не вказує на системні проблеми моделі.

Практична значущість роботи полягає у створенні доступного інструменту для розвитку музичних здібностей, який у подальшому може використовуватися

як самостійно, так і в освітніх закладах. Система забезпечує персоналізований підхід до навчання незалежно від початкового рівня користувача та географічного розташування, що вирішує проблему доступності якісної музичної освіти.

Наукова новизна визначається комплексним підходом до розвитку гармонічного слуху через інтеграцію машинного навчання з інтерактивними веб-технологіями, розробкою ефективної архітектури нейронної мережі для розпізнавання музичних інтервалів на основі спектральних ознак та створенням адаптивної системи навчання з об'єктивним оцінюванням прогресу.

Розроблена система демонструє високу ефективність поєднання традиційних методів цифрової обробки сигналів з сучасними технологіями штучного інтелекту у сфері музичної освіти. Досягнуті результати підтверджують перспективність використання нейронних мереж для автоматизації музичного аналізу.

Перспективи розвитку включають розширення функціональності на розпізнавання акордів та складніших гармонічних прогресій, впровадження адаптивних алгоритмів машинного навчання для персоналізації навчального процесу та створення мобільних версій для забезпечення повсюдного доступу до навчальних матеріалів.

## СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Simply Ltd. Simply Sing: My Singing App. App Store. URL: [https://apps.apple.com/us/app/simply-sing-my-singing-app/id1592900217?pid=website&wf\\_device\\_id=206505b9-7a15-40f1-a785-1fed36bd312f](https://apps.apple.com/us/app/simply-sing-my-singing-app/id1592900217?pid=website&wf_device_id=206505b9-7a15-40f1-a785-1fed36bd312f).
2. Musicca. URL: <https://www.musicca.com/uk>.
3. Django REST framework. URL: <https://www.django-rest-framework.org/>.
4. Single Page App (SPA). React Router. URL: <https://reactrouter.com/how-to/spa>.
5. Discrete Short-Time Fourier Transform (STFT). AudioLabs. URL: [https://www.audiolabs-erlangen.de/resources/MIR/FMP/C2/C2\\_STFT-Basic.html](https://www.audiolabs-erlangen.de/resources/MIR/FMP/C2/C2_STFT-Basic.html).
6. Django documentation. Django Project. URL: <https://docs.djangoproject.com/en/5.2/>.
7. FastAPI. URL: <https://fastapi.tiangolo.com/>.
8. Welcome to Flask. Flask Documentation. URL: <https://flask.palletsprojects.com/en/stable/>.
9. React. URL: <https://react.dev/>.
10. Vue.js. URL: <https://vuejs.org/>.
11. Tone.js. URL: <https://tonejs.github.io/>.
12. Angular. URL: <https://angular.dev/>.
13. Vite. URL: <https://vite.dev/>.
14. TensorFlow. URL: <https://www.tensorflow.org/>.
15. PyTorch. URL: <https://pytorch.org/>.
16. Keras documentation: Keras 3 API documentation. Keras. URL: <https://keras.io/api/>.
17. Documentation scikit-learn: machine learning in Python. Scikit-learn. URL: <https://scikit-learn.sourceforge.net/stable/documentation.html>.
18. Librosa documentation. Librosa. URL: <https://librosa.org/doc/latest/index.html>.

19. PyPI Docs. URL: <https://docs.pypi.org/>.
20. NumPy documentation. NumPy. URL: <https://numpy.org/doc/stable/>.
21. SciPy documentation. Scipy. URL: <https://docs.scipy.org/doc/scipy/>.
22. Django CORS Guide: What It Is and How to Enable It. StackHawk, Inc. URL: <https://www.stackhawk.com/blog/django-cors-guide/>.
23. Mean and Standard Deviation. The Scientist and Engineer's Guide to Digital Signal Processing. URL: <https://www.dspguide.com/ch2/2.htm>.
24. Butterworth Filter Design and Low Pass Butterworth Filters. Basic Electronics Tutorials. URL: [https://www.electronics-tutorials.ws/filter/filter\\_8.html](https://www.electronics-tutorials.ws/filter/filter_8.html).
25. Schubert E., Wolfe J., Tarnopolsky A. Spectral centroid and timbre in complex, multiple instrumental textures. Wayback Machine. URL: <https://web.archive.org/web/20110810004531/http://icmpc8.umn.edu/proceedings/ICMPC8/PDF/AUTHOR/MP040215.PDF>.
26. What Is Spectral Bandwidth within UV-Vis Spectroscopy?. SpectroKnowledge. URL: <https://knowledge.cphnano.com/en/pages/what-is-spectral-bandwidth-within-uv-vis-spectroscopy>.
27. Zero-crossing rate. Introduction to Speech Processing. URL: [https://speechprocessingbook.aalto.fi/Representations/Zero-crossing\\_rate.html](https://speechprocessingbook.aalto.fi/Representations/Zero-crossing_rate.html).
28. Alrashidi K., Khatri K. Musical Chord Classification. Kaggle. URL: <https://www.kaggle.com/code/khaledalrashidi1/musical-chord-classification/notebook>.
29. ReLU – PyTorch 2.7 documentation. PyTorch. URL: <https://docs.pytorch.org/docs/stable/generated/torch.nn.ReLU.html>.
30. Adam – PyTorch 2.7 documentation. PyTorch. URL: <https://docs.pytorch.org/docs/stable/generated/torch.optim.Adam.html>.

# ДОДАТКИ

|  |  |  |  |  |                           |     |
|--|--|--|--|--|---------------------------|-----|
|  |  |  |  |  | ІІЗ.КР.Б – 121 – 25 – ІІЗ |     |
|  |  |  |  |  |                           |     |
|  |  |  |  |  |                           | 105 |

## Лістинг коду файлу audio\_normalization.py

```
import librosa
import soundfile as sf
import numpy as np
import os
from pathlib import Path
import matplotlib.pyplot as plt
from scipy.signal import butter, filtfilt
from scipy.ndimage import median_filter

class AudioNormalizer:
    def __init__(self, sr=44100, target_lufs=-23.0):
        self.sr = sr
        self.target_lufs = target_lufs
        self.target_rms_db = -18.0
        self.max_peak_db = -3.0

        self.processed_count = 0
        self.analysis_data = []

    def calculate_lufs(self, audio):
        """Розрахунок LUFS (стандарт гучності)"""
        try:
            import pyloudnorm as pyln
            meter = pyln.Meter(self.sr)
            loudness = meter.integrated_loudness(audio)
            return loudness
        except ImportError:
            return None

    def calculate_audio_metrics(self, audio):
        """Розрахунок всіх метрик аудіо"""
        rms = np.sqrt(np.mean(audio**2))
        peak = np.max(np.abs(audio))
```

```

rms_db = 20 * np.log10(rms + 1e-10)
peak_db = 20 * np.log10(peak + 1e-10)

crest_factor = peak / (rms + 1e-10)
dynamic_range = peak_db - rms_db

# THD+N наближення
fundamental_freq = self.estimate_fundamental(audio)
thd = self.estimate_thd(audio, fundamental_freq) if fundamental_freq else 0

lufs = self.calculate_lufs(audio)

return {
    'rms': rms,
    'peak': peak,
    'rms_db': rms_db,
    'peak_db': peak_db,
    'crest_factor': crest_factor,
    'dynamic_range': dynamic_range,
    'thd': thd,
    'lufs': lufs,
    'duration': len(audio) / self.sr
}

def estimate_fundamental(self, audio):
    """Оцінка основної частоти"""
    try:
        pitches, magnitudes = librosa.piptrack(y=audio, sr=self.sr)
        pitches = pitches[magnitudes > np.percentile(magnitudes, 85)]
        if len(pitches) > 0:
            return np.median(pitches[pitches > 0])
    except:
        pass
    return None

```

```

def estimate_thd(self, audio, fundamental_freq):
    """Оцінка коефіцієнта гармонічних спотворень"""
    if not fundamental_freq:
        return 0

    try:
        fft = np.fft.fft(audio)
        freqs = np.fft.fftfreq(len(audio), 1/self.sr)

        fundamental_bin = np.argmin(np.abs(freqs - fundamental_freq))
        fundamental_magnitude = np.abs(fft[fundamental_bin])

        harmonic_magnitudes = []
        for harmonic in range(2, 6):
            harmonic_freq = fundamental_freq * harmonic
            if harmonic_freq < self.sr / 2:
                harmonic_bin = np.argmin(np.abs(freqs - harmonic_freq))
                harmonic_magnitudes.append(np.abs(fft[harmonic_bin]))

        if harmonic_magnitudes:
            thd = np.sqrt(sum(h**2 for h in harmonic_magnitudes)) /
fundamental_magnitude
            return min(thd, 1.0)
        except:
            pass
        return 0

def apply_preemphasis(self, audio, alpha=0.97):
    """Пре-емфазис для підвищення високих частот"""
    return np.append(audio[0], audio[1:] - alpha * audio[:-1])

def apply_highpass_filter(self, audio, cutoff=80, order=4):
    """High-pass фільтр"""
    nyquist = self.sr / 2

```

```
normal_cutoff = cutoff / nyquist
```

```
if normal_cutoff >= 1.0:
```

```
    return audio
```

```
try:
```

```
    b, a = butter(order, normal_cutoff, btype='high', analog=False)
```

```
    return filtfilt(b, a, audio)
```

```
except:
```

```
    return audio
```

```
def apply_lowpass_filter(self, audio, cutoff=8000, order=4):
```

```
    """Low-pass фільтр"""
```

```
    nyquist = self.sr / 2
```

```
    normal_cutoff = cutoff / nyquist
```

```
if normal_cutoff >= 1.0:
```

```
    return audio
```

```
try:
```

```
    b, a = butter(order, normal_cutoff, btype='low', analog=False)
```

```
    return filtfilt(b, a, audio)
```

```
except:
```

```
    return audio
```

```
def apply_spectral_gating(self, audio, gate_threshold=-70):
```

```
    """Спектральний гейт для зменшення шуму"""
```

```
    frame_length = 2048
```

```
    hop_length = 512
```

```
    stft = librosa.stft(audio, n_fft=frame_length, hop_length=hop_length)
```

```
    magnitude = np.abs(stft)
```

```
    phase = np.angle(stft)
```

```
    magnitude_db = 20 * np.log10(magnitude + 1e-10)
```

```

gate_mask = magnitude_db > gate_threshold
gated_magnitude = magnitude * gate_mask

gated_stft = gated_magnitude * np.exp(1j * phase)
gated_audio = librosa.istft(gated_stft, hop_length=hop_length)

return gated_audio

```

```

def rms_normalize(self, audio, target_rms_db=-18):

```

```

    """RMS нормалізація"""

```

```

    current_rms = np.sqrt(np.mean(audio**2))

```

```

    if current_rms == 0:

```

```

        return audio

```

```

    current_rms_db = 20 * np.log10(current_rms)

```

```

    gain_db = target_rms_db - current_rms_db

```

```

    gain_linear = 10**(gain_db / 20)

```

```

    normalized = audio * gain_linear

```

```

    peak = np.max(np.abs(normalized))

```

```

    if peak > 0.95:

```

```

        normalized = normalized * (0.95 / peak)

```

```

    return normalized

```

```

def lufs_normalize(self, audio, target_lufs=-23):

```

```

    """LUFS нормалізація (професійний стандарт)"""

```

```

    try:

```

```

        import pyloudnorm as pyn

```

```

        meter = pyn.Meter(self.sr)

```

```

        loudness = meter.integrated_loudness(audio)

```

```

    if np.isfinite(loudness):

```

```

        normalized_audio = pyln.normalize.loudness(audio, loudness, target_lufs)

        peak = np.max(np.abs(normalized_audio))
        if peak > 0.95:
            normalized_audio = normalized_audio * (0.95 / peak)

        return normalized_audio
    except ImportError:
        pass

    return self.rms_normalize(audio, self.target_rms_db)

def process_recording(self, input_path, output_path, backup=True):
    """Повна обробка одного запису"""
    if backup:
        backup_path = str(input_path).replace('.wav', '_original_backup.wav')
        if not os.path.exists(backup_path):
            import shutil
            shutil.copy2(input_path, backup_path)

    try:
        audio, sr = librosa.load(input_path, sr=self.sr, mono=True)
        original_metrics = self.calculate_audio_metrics(audio)

        processed_audio = audio.copy()

        processed_audio = processed_audio - np.mean(processed_audio)

        processed_audio = self.apply_highpass_filter(processed_audio, cutoff=80)
        processed_audio = self.apply_lowpass_filter(processed_audio, cutoff=8000)

        processed_audio = self.apply_spectral_gating(processed_audio, gate_threshold=-

```

```

processed_audio = np.tanh(processed_audio * 0.95) * 0.95

final_metrics = self.calculate_audio_metrics(processed_audio)

sf.write(output_path, processed_audio, self.sr, subtype='PCM_24')

processing_data = {
    'file': os.path.basename(input_path),
    'original': original_metrics,
    'processed': final_metrics,
    'gain_applied': final_metrics['rms_db'] - original_metrics['rms_db']
}

self.analysis_data.append(processing_data)
self.processed_count += 1

print(f'Оброблено: {os.path.basename(input_path)}')
print(f"  RMS: {original_metrics['rms_db']:.1f} → {final_metrics['rms_db']:.1f}
дБ")

print(f"  Пік: {original_metrics['peak_db']:.1f} → {final_metrics['peak_db']:.1f}
дБ")

if original_metrics['lufs'] and final_metrics['lufs']:
    print(f"  LUFs: {original_metrics['lufs']:.1f} → {final_metrics['lufs']:.1f}")
print()

return True

except Exception as e:
    print(f"Помилка обробки {input_path}: {e}")
    return False

def process_directory(self, input_dir, output_dir=None):
    """Обробка всієї папки"""
    if output_dir is None:

```

```

        output_dir = input_dir

    Path(output_dir).mkdir(parents=True, exist_ok=True)

    wav_files = [f for f in os.listdir(input_dir)
                  if f.endswith('.wav') and not f.endswith('_original_backup.wav')]

    if not wav_files:
        print("Не знайдено WAV файлів для обробки")
        return False

    print(f"НОРМАЛІЗАЦІЯ АУДІО ФАЙЛІВ")
    print(f"Знайдено {len(wav_files)} файлів")
    print(f"Цільові параметри:")
    print(f"  LUFS: {self.target_lufs} дБ")
    print(f"  RMS: {self.target_rms_db} дБ")
    print(f"  Максимальний пік: {self.max_peak_db} дБ")
    print("=" * 50)

    for filename in wav_files:
        input_path = os.path.join(input_dir, filename)
        output_path = os.path.join(output_dir, filename)

        self.process_recording(input_path, output_path)

    self.generate_analysis_report(output_dir)

    print(f"Нормалізація завершена. Оброблено {self.processed_count} файлів")
    return True

def generate_analysis_report(self, output_dir):
    """Генерація звіту аналізу"""
    if not self.analysis_data:
        return

```

```
import json
```

```
original_rms = [d['original']['rms_db'] for d in self.analysis_data]
```

```
processed_rms = [d['processed']['rms_db'] for d in self.analysis_data]
```

```
original_peaks = [d['original']['peak_db'] for d in self.analysis_data]
```

```
processed_peaks = [d['processed']['peak_db'] for d in self.analysis_data]
```

```
report = {
```

```
    'summary': {
```

```
        'files_processed': len(self.analysis_data),
```

```
        'target_lufs': self.target_lufs,
```

```
        'target_rms_db': self.target_rms_db
```

```
    },
```

```
    'original_stats': {
```

```
        'rms_db_mean': np.mean(original_rms),
```

```
        'rms_db_std': np.std(original_rms),
```

```
        'rms_db_min': np.min(original_rms),
```

```
        'rms_db_max': np.max(original_rms),
```

```
        'peak_db_mean': np.mean(original_peaks),
```

```
        'peak_db_max': np.max(original_peaks)
```

```
    },
```

```
    'processed_stats': {
```

```
        'rms_db_mean': np.mean(processed_rms),
```

```
        'rms_db_std': np.std(processed_rms),
```

```
        'rms_db_min': np.min(processed_rms),
```

```
        'rms_db_max': np.max(processed_rms),
```

```
        'peak_db_mean': np.mean(processed_peaks),
```

```
        'peak_db_max': np.max(processed_peaks)
```

```
    },
```

```
    'consistency_improvement': {
```

```
        'original_rms_std': np.std(original_rms),
```

```
        'processed_rms_std': np.std(processed_rms),
```

```
        'improvement_factor': np.std(original_rms) / np.std(processed_rms)
```

```
    }
```

```

    }

    report_path = os.path.join(output_dir, 'normalization_report.json')
    with open(report_path, 'w') as f:
        json.dump(report, f, indent=2, default=lambda x: float(x) if isinstance(x,
np.floating) else x)

    print(f"Звіт збережено: {report_path}")
    print(f"Покращення                консистентності                RMS:
{report['consistency_improvement']['improvement_factor']:.2f}x")

def normalize_desktop_recordings(input_dir, output_dir=None):
    """Нормалізація записів"""
    normalizer = AudioNormalizer(
        sr=44100,
        target_lufs=-23.0
    )

    return normalizer.process_directory(input_dir, output_dir)

if __name__ == "__main__":
    input_directory = "dataset-preparation/raw-recordings"
    output_directory = "dataset-preparation/normalized-recordings"

    if os.path.exists(input_directory):
        wav_files = [f for f in os.listdir(input_directory) if f.endswith('.wav')]
        if wav_files:
            print(f"Знайдено {len(wav_files)} файлів для нормалізації")
            success = normalize_desktop_recordings(input_directory, output_directory)
            if success:
                print("\nНормалізація завершена. Файли готові для нарізки сегментів.")
            else:
                print(f"Відсутні WAV файли")

```

else:

```
print(f"Папка не найдена")
```

```
Path(input_directory).mkdir(parents=True, exist_ok=True)
```

```
print(f"Створено папку: {input_directory}")
```

## Лістинг коду файлу audio\_processing.py

```
import librosa
import soundfile as sf
import numpy as np
import os
from scipy.signal import butter, filtfilt
from pathlib import Path
import shutil

class AudioProcessor:
    def __init__(self, sr=44100, segment_duration=2.0):
        self.sr = sr
        self.segment_duration = segment_duration
        self.segment_samples = int(segment_duration * sr)

        self.interval_map = {
            'min2': 'minor_2nd',
            'maj2': 'major_2nd',
            'min3': 'minor_3rd',
            'maj3': 'major_3rd',
            'perf4': 'perfect_4th',
            'tritone': 'tritone',
            'perf5': 'perfect_5th',
            'min6': 'minor_6th',
            'maj6': 'major_6th',
            'min7': 'minor_7th',
            'maj7': 'major_7th',
            'perf8': 'perfect_8th'
        }

        self.dynamics_map = {
            'soft': 'piano',
            'hard': 'forte'
        }
```

```

def parse_filename(self, filename):
    base_name = filename.replace('.wav', '')
    parts = base_name.split('_')

    if len(parts) >= 2:
        interval_code = parts[0]
        dynamics_code = parts[1]

        interval_name = self.interval_map.get(interval_code, interval_code)
        dynamics_name = self.dynamics_map.get(dynamics_code, dynamics_code)

        return interval_name, dynamics_name

    return None, None

def minimal_preprocessing(self, audio):
    """Мінімальний препроцесинг для нормалізованого аудіо"""
    audio = audio - np.mean(audio)

    audio = np.clip(audio, -0.98, 0.98)

    audio = np.tanh(audio * 0.95) * 0.95

    return audio

def pitch_shift(self, audio, semitones):
    """Транспозиція"""
    if semitones == 0:
        return audio

    try:
        return librosa.effects.pitch_shift(
            audio, sr=self.sr, n_steps=semitones,
            bins_per_octave=12, res_type='scipy'

```

```

    )

except Exception as e:
    print(f"Помилка транспозиції на {semitones} піввтонів: {e}")
    return audio

def convert_to_mono(self, audio):
    """Конвертування стерео в моно"""
    if audio.ndim > 1:
        return np.mean(audio, axis=0)
    return audio

def detect_onsets(self, audio, min_interval=1.2):
    """Виявлення початків"""
    try:
        onset_frames = librosa.onset.onset_detect(
            y=audio,
            sr=self.sr,
            hop_length=512,
            backtrack=True,
            units='time',
            pre_max=20,
            post_max=20,
            pre_avg=100,
            post_avg=100,
            delta=0.2,
            wait=10
        )

        filtered_onsets = []
        for onset in onset_frames:
            if not filtered_onsets or onset - filtered_onsets[-1] >= min_interval:
                filtered_onsets.append(onset)

        return np.array(filtered_onsets)
    except Exception as e:

```

```

        print(f"Помилка onset detection: {e}")
        duration = len(audio) / self.sr
        num_segments = max(int(duration / 2), 30)
        return np.linspace(1, duration - self.segment_duration - 1, num_segments)

def cut_segments(self, audio, onsets, interval_name, dynamics, output_dir):
    """Нарізка аудіо на сегменти"""
    interval_dir = os.path.join(output_dir, interval_name)
    dynamics_dir = os.path.join(interval_dir, dynamics)
    Path(dynamics_dir).mkdir(parents=True, exist_ok=True)

    segments = []
    for i, onset in enumerate(onsets):
        start_sample = int(onset * self.sr)
        end_sample = start_sample + self.segment_samples

        if end_sample <= len(audio):
            segment = audio[start_sample:end_sample]

            filename = f"{interval_name}_{i:03d}_{dynamics}.wav"
            filepath = os.path.join(dynamics_dir, filename)

            try:
                sf.write(filepath, segment, self.sr)

                if os.path.exists(filepath) and os.path.getsize(filepath) > 0:
                    segments.append((segment, filename))
                    if i % 10 == 0:
                        print(f"Збережено: {interval_name}/{dynamics}/{filename}")
                else:
                    print(f"Помилка збереження: {filename}")

            except Exception as e:
                print(f"Помилка при збереженні {filename}: {e}")
                continue

```

```

return segments

def clean_specific_interval_files(self, output_base_dir, interval_name, dynamics):
    """Очистити файли конкретного інтервалу"""
    dirs_to_check = [
        os.path.join(output_base_dir, "original", interval_name, dynamics),
        os.path.join(output_base_dir, "original", interval_name, f"{dynamics}_up"),
        os.path.join(output_base_dir, "original", interval_name, f"{dynamics}_down"),
        os.path.join(output_base_dir, "up_semitone", interval_name, dynamics),
        os.path.join(output_base_dir, "up_semitone", interval_name, f"{dynamics}_up"),
        os.path.join(output_base_dir, "down_semitone", interval_name, dynamics),
        os.path.join(output_base_dir, "down_semitone", interval_name,
f"{dynamics}_down")
    ]

    import glob
    removed_count = 0

    for dir_path in dirs_to_check:
        if os.path.exists(dir_path):
            wav_files = glob.glob(os.path.join(dir_path, "*.wav"))
            for file_path in wav_files:
                try:
                    if os.path.exists(file_path):
                        os.remove(file_path)
                        removed_count += 1
                except Exception as e:
                    print(f"Помилка видалення {file_path}: {e}")

    if removed_count > 0:
        print(f"Видалено {removed_count} старих файлів для {interval_name}_{dynamics}")

    return removed_count

```

```

def create_output_structure(self, output_base_dir):
    """Створення структури папок"""
    dirs_to_create = [
        os.path.join(output_base_dir, "original"),
        os.path.join(output_base_dir, "up_semitone"),
        os.path.join(output_base_dir, "down_semitone")
    ]

    for dir_path in dirs_to_create:
        Path(dir_path).mkdir(parents=True, exist_ok=True)

def load_normalized_audio(self, file_path):
    """Завантаження нормалізованого аудіо"""
    try:
        audio, sr = librosa.load(file_path, sr=self.sr, mono=True)
        print(f"        Завантажено:    {os.path.basename(file_path)},    тривалість: {len(audio)/sr:.2f}c")
        return audio
    except Exception as e:
        print(f" Помилка завантаження {file_path}: {e}")
        return None

def process_single_normalized_recording(self, input_file, output_base_dir):
    """Обробка одного нормалізованого запису"""
    print(f"\nОбробка {os.path.basename(input_file)}")

    filename = os.path.basename(input_file)
    interval_name, dynamics = self.parse_filename(filename)

    if not interval_name or not dynamics:
        print(f" Неможливо розпарсити: {filename}")
        return 0

    print(f" Інтервал: {interval_name}, Динаміка: {dynamics}")

```

```

if not os.path.exists(input_file):
    print(f" Файл не існує: {input_file}")
    return 0

try:
    self.create_output_structure(output_base_dir)
    self.clean_specific_interval_files(output_base_dir, interval_name, dynamics)

    audio = self.load_normalized_audio(input_file)
    if audio is None:
        return 0

    print(f" Мінімальний препроцесинг...")
    audio = self.minimal_preprocessing(audio)

    print(f" Створення транспозицій...")
    audio_up = self.pitch_shift(audio, 1.0)
    audio_down = self.pitch_shift(audio, -1.0)

    onsets = self.detect_onsets(audio)
    print(f" Знайдено {len(onsets)} onset'ів")

    if len(onsets) < 10:
        print(f" УВАГА: Мало onset'ів ({len(onsets)})")

    print(f" Нарізка сегментів...")
    original_segments = self.cut_segments(
        audio, onsets, interval_name, dynamics,
        os.path.join(output_base_dir, "original")
    )

    up_segments = self.cut_segments(
        audio_up, onsets, interval_name, f"{dynamics}_up",
        os.path.join(output_base_dir, "up_semitone")
    )

```

```

    )

    down_segments = self.cut_segments(
        audio_down, onsets, interval_name, f"{dynamics}_down",
        os.path.join(output_base_dir, "down_semitone")
    )

    total_segments = len(original_segments) + len(up_segments) +
len(down_segments)
    print(f" Результат: {total_segments} сегментів")

    return total_segments

except Exception as e:
    print(f" Помилка обробки {filename}: {e}")
    return 0
finally:
    import gc
    gc.collect()

def clear_dataset_safely(output_dir):
    """Очищення датасету"""
    if not os.path.exists(output_dir):
        return True

    removed_count = 0
    failed_count = 0

    for root, dirs, files in os.walk(output_dir, topdown=False):
        for file in files:
            file_path = os.path.join(root, file)
            try:
                os.remove(file_path)
                removed_count += 1

```

```
except PermissionError:
    failed_count += 1
except Exception as e:
    print(f"Помилка видалення {file_path}: {e}")
    failed_count += 1
```

```
for dir_name in dirs:
    dir_path = os.path.join(root, dir_name)
    try:
        if not os.listdir(dir_path):
            os.rmdir(dir_path)
    except:
        pass
```

```
print(f"Видалено {removed_count} файлів, пропущено {failed_count} файлів")
return failed_count == 0
```

```
def process_normalized_recordings(input_dir, output_dir, clear_existing=False):
    """Обробка нормалізованих записів"""
    processor = AudioProcessor()
```

```
    if clear_existing:
        print("ОЧИЩЕННЯ ІСНУЮЧОГО ДАТАСЕТУ")
        print("=" * 50)
        if os.path.exists(output_dir):
            try:
                shutil.rmtree(output_dir)
                print(f"Очищено: {output_dir}")
            except Exception as e:
                print(f"Помилка очищення: {e}")
                clear_dataset_safely(output_dir)
```

```
    print("ОБРОБКА НОРМАЛІЗОВАНИХ ЗАПИСІВ")
    print("=" * 50)
```

```

Path(output_dir).mkdir(parents=True, exist_ok=True)

wav_files = [f for f in os.listdir(input_dir)
              if f.endswith('.wav') and not f.endswith('_original_backup.wav')]
wav_files.sort()

if not wav_files:
    print("Не знайдено нормалізованих wav файлів!")
    return

print(f"Знайдено {len(wav_files)} нормалізованих файлів:")
for file in wav_files:
    interval, dynamics = processor.parse_filename(file)
    if interval and dynamics:
        print(f" {file} → {interval} ( {dynamics} )")
    else:
        print(f" {file} → неправильний формат")

print("\nПочинаємо обробку нормалізованих файлів...")

total_processed = 0
successful_files = 0

for filename in wav_files:
    input_file = os.path.join(input_dir, filename)

    try:
        segments_count = processor.process_single_normalized_recording(input_file,
output_dir)
        if segments_count > 0:
            total_processed += segments_count
            successful_files += 1
    except Exception as e:
        print(f"Критична помилка при обробці {filename}: {e}")

```

```

print(f"\nОБРОБКА ЗАВЕРШЕНА")
print(f"=" * 50)
print(f"Успішно оброблено: {successful_files}/{len(wav_files)} файлів")
print(f"Загалом створено: {total_processed} сегментів")
print(f"Середньо сегментів на файл: {total_processed/max(successful_files,1):.1f}")

if total_processed >= 3000:
    print(f"Датасет готовий для генерації спектрограм")
else:
    print(f"УВАГА: Мало сегментів для надійного тренування")

if __name__ == "__main__":
    normalized_directory = "dataset-preparation/normalized-recordings"
    output_directory = "dataset-preparation/processed-segments"

    if os.path.exists(normalized_directory):
        wav_files = [f for f in os.listdir(normalized_directory)
                     if f.endswith('.wav') and not f.endswith('_original_backup.wav')]
        if wav_files:
            print(f"\nЗнайдено {len(wav_files)} нормалізованих файлів")
            process_normalized_recordings(normalized_directory, output_directory,
            clear_existing=True)

        else:
            print(f"У папці {normalized_directory} відсутні WAV файли")
    else:
        print(f"\nПапка не знайдена: {normalized_directory}")

```

## Лістинг коду файлу interval\_classifier.py

```
import numpy as np
import librosa
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import os
import json
from pathlib import Path
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.utils.class_weight import compute_class_weight
import matplotlib.pyplot as plt

class FFTIntervalClassifier:
    def __init__(self, fft_size=2048, sample_rate=22050):
        self.fft_size = fft_size
        self.sample_rate = sample_rate
        self.freq_bins = fft_size // 2 + 1
        self.model = None
        self.scaler = StandardScaler()

        self.class_names = []
        self.class_to_index = {}
        self.num_classes = 0

        self.min_freq = 80
        self.max_freq = 4000
        self.freq_range = self._calculate_freq_range()

    def _calculate_freq_range(self):
        """Розрахунок індексів частотного діапазону"""
        freqs = np.fft.fftfreq(self.fft_size, 1/self.sample_rate)[:self.freq_bins]
        min_idx = np.argmax(freqs >= self.min_freq)
```

```

max_idx = np.argmax(freqs >= self.max_freq)
if max_idx == 0:
    max_idx = len(freqs)
return min_idx, max_idx

```

```

def extract_harmonic_features(self, audio):

```

```

    """Виділення гармонічних ознак з аудіо"""

```

```

    if len(audio) < self.fft_size:

```

```

        audio = np.pad(audio, (0, self.fft_size - len(audio)))

```

```

    elif len(audio) > self.fft_size:

```

```

        start = (len(audio) - self.fft_size) // 2

```

```

        audio = audio[start:start + self.fft_size]

```

```

    windowed_audio = audio * np.hanning(len(audio))

```

```

    fft = np.fft.fft(windowed_audio, n=self.fft_size)

```

```

    magnitude = np.abs(fft[:self.freq_bins])

```

```

    min_idx, max_idx = self.freq_range

```

```

    magnitude = magnitude[min_idx:max_idx]

```

```

    magnitude_db = 20 * np.log10(magnitude + 1e-10)

```

```

    magnitude_normalized = (magnitude_db - np.min(magnitude_db)) /
    (np.max(magnitude_db) - np.min(magnitude_db) + 1e-10)

```

```

    additional_features = self._extract_additional_features(magnitude, audio)

```

```

    all_features = np.concatenate([magnitude_normalized, additional_features])

```

```

    return all_features

```

```

def _extract_additional_features(self, magnitude, audio):

```

```

    """Додаткові музичні ознаки"""

```

```

    features = []

```

```

freqs = np.linspace(self.min_freq, self.max_freq, len(magnitude))
spectral_centroid = np.sum(freqs * magnitude) / (np.sum(magnitude) + 1e-10)
features.append(spectral_centroid / self.max_freq)

spectral_bandwidth = np.sqrt(np.sum(((freqs - spectral_centroid) ** 2) * magnitude)
/ (np.sum(magnitude) + 1e-10))
features.append(spectral_bandwidth / self.max_freq)

cumsum = np.cumsum(magnitude)
rolloff_idx = np.where(cumsum >= 0.85 * cumsum[-1])[0]
if len(rolloff_idx) > 0:
    rolloff_freq = freqs[rolloff_idx[0]]
    features.append(rolloff_freq / self.max_freq)
else:
    features.append(1.0)

rms = np.sqrt(np.mean(audio ** 2))
features.append(rms)

zcr = np.mean(np.abs(np.diff(np.sign(audio))))
features.append(zcr)

peak_indices = self._find_spectral_peaks(magnitude)
peak_ratios = self._calculate_peak_ratios(peak_indices, freqs)
features.extend(peak_ratios)

return np.array(features)

def _find_spectral_peaks(self, magnitude, prominence=0.1):
    """Знаходження піків в спектрі"""
    from scipy.signal import find_peaks

    peaks, _ = find_peaks(magnitude, prominence=prominence * np.max(magnitude))
    return peaks[:10]

```

```

def _calculate_peak_ratios(self, peak_indices, freqs):
    """Розрахунок співвідношень між піками"""
    if len(peak_indices) < 2:
        return [0.0] * 10

    ratios = []
    base_freq = freqs[peak_indices[0]] if len(peak_indices) > 0 else 1.0

    for i, peak_idx in enumerate(peak_indices[:5]):
        ratio = freqs[peak_idx] / base_freq if base_freq > 0 else 0.0
        ratios.append(ratio)

    while len(ratios) < 10:
        ratios.append(0.0)

    return ratios[:10]

def prepare_dataset_from_pickle(self, dataset_dir):
    """Завантаження датасету"""
    import pickle

    pickle_path = os.path.join(dataset_dir, 'fft_dataset.pkl')

    if not os.path.exists(pickle_path):
        raise FileNotFoundError(f"Датасет не знайдено: {pickle_path}")

    print("ЗАВАНТАЖЕННЯ ДАТАСЕТУ")

    with open(pickle_path, 'rb') as f:
        dataset = pickle.load(f)

    X = dataset['features']
    y_labels = dataset['labels']

```

```
self.class_names = sorted(list(set(y_labels)))
self.num_classes = len(self.class_names)
self.class_to_index = {name: idx for idx, name in enumerate(self.class_names)}
```

```
y = np.array([self.class_to_index[label] for label in y_labels])
```

```
print(f"Завантажено {len(X)} зразків")
print(f"Розмірність ознак: {X.shape[1]}")
print(f"Класи ({self.num_classes}):")
```

```
for i, class_name in enumerate(self.class_names):
    count = np.sum(y == i)
    print(f" {class_name}: {count} зразків")
```

```
return X, y
```

```
def create_model(self, input_dim):
    """Створення моделі"""
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),

        layers.BatchNormalization(),

        layers.Dense(512, kernel_regularizer=keras.regularizers.l2(0.001)),
        layers.BatchNormalization(),
        layers.Activation('relu'),
        layers.Dropout(0.3),

        layers.Dense(256, kernel_regularizer=keras.regularizers.l2(0.001)),
        layers.BatchNormalization(),
        layers.Activation('relu'),
        layers.Dropout(0.4),

        layers.Dense(128, kernel_regularizer=keras.regularizers.l2(0.001)),
        layers.BatchNormalization(),
```

```

        layers.Activation('relu'),
        layers.Dropout(0.5),

        layers.Dense(64, kernel_regularizer=keras.regularizers.l2(0.001)),
        layers.BatchNormalization(),
        layers.Activation('relu'),
        layers.Dropout(0.3),

        layers.Dense(self.num_classes, activation='softmax')
    ])

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=0.001),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy', keras.metrics.SparseCategoricalAccuracy(name='accuracy')]
    )

    self.model = model
    return model

    def train(self, dataset_dir, epochs=100, batch_size=32, validation_split=0.2,
save_path='fft_interval_model.keras'):
    """Тренування моделі"""
    X, y = self.prepare_dataset_from_pickle(dataset_dir)

    X_scaled = self.scaler.fit_transform(X)

    X_train, X_val, y_train, y_val = train_test_split(
        X_scaled, y, test_size=validation_split, random_state=42, stratify=y
    )

    print(f"\nТренувальний набір: {len(X_train)} зразків")
    print(f"Валідаційний набір: {len(X_val)} зразків")

    self.create_model(X.shape[1])

```

```
print(f'Параметрів моделі: {self.model.count_params():,}')
```

```
class_weights = compute_class_weight('balanced', classes=np.unique(y), y=y)
```

```
class_weight_dict = {i: weight for i, weight in enumerate(class_weights)}
```

```
callbacks = [  
    keras.callbacks.ModelCheckpoint(  
        save_path,  
        monitor='val_accuracy',  
        save_best_only=True,  
        mode='max',  
        verbose=1  
    ),  
    keras.callbacks.ReduceLROnPlateau(  
        monitor='val_loss',  
        factor=0.5,  
        patience=10,  
        min_lr=1e-7,  
        verbose=1  
    ),  
    keras.callbacks.EarlyStopping(  
        monitor='val_accuracy',  
        patience=20,  
        restore_best_weights=True,  
        verbose=1,  
        min_delta=0.001  
    )  
]
```

```
self.history = self.model.fit(  
    X_train, y_train,  
    validation_data=(X_val, y_val),  
    epochs=epochs,  
    batch_size=batch_size,
```

```
callbacks=callbacks,  
class_weight=class_weight_dict,  
verbose=1  
)
```

```
try:
```

```
    self.model = keras.models.load_model(save_path)
```

```
    print(f"\nТренування завершено. Найкраща модель: {save_path}")
```

```
except Exception as e:
```

```
    print(f"Помилка завантаження моделі: {e}")
```

```
return self.history
```

```
def evaluate(self, segments_dir):
```

```
    """Оцінка моделі"""
```

```
    X, y = self.prepare_dataset_from_segments(segments_dir)
```

```
    X_scaled = self.scaler.transform(X)
```

```
    predictions = self.model.predict(X_scaled)
```

```
    predicted_classes = np.argmax(predictions, axis=1)
```

```
    accuracy = np.mean(predicted_classes == y)
```

```
    print(f"\nТочність на тестових даних: {accuracy:.4f} ({accuracy*100:.2f}%)")
```

```
    return accuracy
```

```
def predict_interval_from_audio(self, audio_path):
```

```
    """Прогнозування інтервалу з аудіофайлу"""
```

```
    try:
```

```
        audio, sr = librosa.load(audio_path, sr=self.sample_rate, mono=True)
```

```
        features = self.extract_harmonic_features(audio)
```

```
        features_scaled = self.scaler.transform([features])
```

```
        predictions = self.model.predict(features_scaled, verbose=0)
```

```

predicted_probs = predictions[0]

predicted_index = np.argmax(predicted_probs)
confidence = float(predicted_probs[predicted_index])
predicted_interval = self.class_names[predicted_index]

top3_indices = np.argsort(predicted_probs)[-3:][::-1]
top3_predictions = [
    {
        "interval": self.class_names[i],
        "confidence": float(predicted_probs[i])
    }
    for i in top3_indices
]

return {
    "interval": predicted_interval,
    "confidence": confidence,
    "top3_predictions": top3_predictions
}

except Exception as e:
    return {"error": f"Помилка прогнозування: {str(e)}"}

def save_model_and_scaler(self, model_path, scaler_path):
    """Збереження моделі та scaler"""
    if self.model:
        self.model.save(model_path)

    import joblib
    joblib.dump(self.scaler, scaler_path)

    metadata = {
        'class_names': self.class_names,
        'class_to_index': self.class_to_index,

```

```

        'num_classes': int(self.num_classes),
        'fft_size': int(self.fft_size),
        'sample_rate': int(self.sample_rate),
        'freq_range': [int(self.freq_range[0]), int(self.freq_range[1])]
    }

```

```

metadata_path = model_path.replace('.keras', '_metadata.json')
with open(metadata_path, 'w') as f:
    json.dump(metadata, f, indent=2)

```

```

print(f"Модель збережена: {model_path}")
print(f"Scaler збережено: {scaler_path}")
print(f"Метадані збережені: {metadata_path}")

```

```

def train_fft_classifier(dataset_dir, model_path, scaler_path, epochs=100):

```

```

    """Основна функція для тренування"""

```

```

    print("ЗАПУСК ТРЕНУВАННЯ")

```

```

    classifier = FFTIntervalClassifier()

```

```

    history = classifier.train(
        dataset_dir=dataset_dir,
        epochs=epochs,
        batch_size=32,
        save_path=model_path
    )

```

```

    classifier.save_model_and_scaler(model_path, scaler_path)

```

```

    return classifier, history

```

```

if __name__ == "__main__":

```

```

    dataset_directory = "../dataset-preparation/fft-dataset"

```

```
model_path = "fft_interval_model.keras"
scaler_path = "fft_scaler.joblib"

if os.path.exists(dataset_directory):
    pickle_file = os.path.join(dataset_directory, 'fft_dataset.pkl')
    if os.path.exists(pickle_file):
        classifier, history = train_fft_classifier(
            dataset_directory,
            model_path,
            scaler_path,
            epochs=100
        )

        print("Модель готова для використання")
    else:
        print(f"Датасет не знайдено: {pickle_file}")
else:
    print(f"Папка датасету не знайдена: {dataset_directory}")
```